

Herald — Specification V3

Herald — Specification V3

Field	Value
Revision	1
Created	2026-05-20
Last modified	2026-05-20
Status	active
Status summary	V3 r1 supersedes V2. Adds §31 project-integration contract, §32 inbound processing pipeline (30s poll + FIFO + worker stages + anti-spam), §33 LLM/agent dispatch (Claude Code session resolution), §34 tri-stage reply protocol (queued→processing→result), §35 versioned-report fan-out with Git linkage, §36 outbound multi-format attachment bundles (Markdown + HTML + PDF + DOCX). Expanded §18.2 Project Herald with Investigation-before-Fixing flow + criticality + type classification + attachment storage at <code>issues/users/attachments/WORKABLE_ITEM_ID/</code> .
Issues	none
Issues summary	First-implementation work tracked under HRD- prefix; V3 is the version consuming projects integrate against.
Fixed	V3-R1-01..V3-R1-14 (this revision); inherits closed: V2-R-01..V2-R-14 (V2 r2), V3-R-01..V3-R-12 (V2 r3), V1 R-01..R-22.
Fixed summary	Delivers the operator-product story V2 deferred: explicit consuming-project contract, full inbound-message lifecycle, first-class Claude Code integration with project-named session resolution, tri-stage user-visible reply protocol, versioned reports re-publish with Git commit link, multi-format outbound attachments (.md/.html/.pdf/.docx).
Continuation	V3 r2 (planned, same workstream): refine all flavor sections (§18.x) for richer per-channel interaction (interactive buttons, modals, reactions, slash commands, command discovery via <code>help</code>). V3 r3 (planned): polish + re-export V1+V2+V3 + commit + push EVERYTHING.

The **bi-directional event fan-out** system: Herald ingests events from heterogeneous sources and reliably fans them out to multiple notification channels so every alert reaches the right destination without confusion, and processes inbound replies/commands back from subscribers in a structured, security-validated way.

Table of contents

- [§1. Upstreams](#)
- [§2. Mission and scope](#)
 - [2.1 What Herald is](#)
 - [2.2 What Herald is NOT](#)
 - [2.3 Architecture diagram](#)
- [§3. Execution model](#)
 - [3.1 Single binary, two modes](#)
 - [3.2 Distribution + invocation surfaces](#)
 - [3.3 Configuration & credentials](#)
 - [3.4 Worker pools, concurrency, and hot-reload](#)
 - [3.5 Time / clock abstraction](#)
- [§4. Event model & wire format](#)
 - [4.1 CloudEvents v1.0 as canonical envelope](#)
 - [4.2 Herald event-type taxonomy](#)
 - [4.3 Idempotency keys](#)
- [§5. Architecture overview](#)
 - [5.1 Components](#)
 - [5.2 Internal routing \(Watermill\)](#)
 - [5.3 Queue backend \(Postgres + River default, NATS opt-in\)](#)
 - [5.4 Retries & dead-lettering](#)
 - [5.4.1 Outbound idempotency \(channel-side composition\)](#)
 - [5.5 Webhook ingestion](#)
 - [5.6 Orchestration of long-running operations \(Temporal, opt-in\)](#)
 - [5.7 Ingress API URLs \(HTTP surface\)](#)
- [§6. Channel addressing & routing](#)
 - [6.1 URL-scheme channel addresses \(Apprise-style\)](#)
 - [6.2 Tag-based fan-out](#)
 - [6.3 HeraldBranding \(per-flavor visual identity\)](#)
- [§7. Subscriber model](#)
 - [7.1 Identity & reconciliation \(per-channel-id + operator alias\)](#)
 - [7.2 Preferences \(Knock-style PreferenceSet\)](#)
 - [7.3 Quiet hours & throttling](#)
 - [7.4 Locale \(i18n\)](#)
 - [7.5 AI-agent subscribers \(distinct from human subscribers\)](#)
- [§8. Workable-item naming prefix](#)
 - [8.1 Static prefix HRD-](#)
 - [8.2 Derived 3-letter prefix algorithm](#)
 - [8.3 Workable-item lifecycle \(HRD-NNN flow\)](#)
- [§9. Technology stack](#)
 - [9.1 Go \(single-binary, multi-flavor\)](#)
 - [9.2 Postgres + Row-Level Security](#)
 - [9.3 Redis \(per-tenant ACL\)](#)
 - [9.4 Container ports \(24XXX\)](#)
 - [9.5 containers submodule](#)
 - [9.6 Database migration tooling](#)
- [§10. Commons \(architecture layers\)](#)

- §11. Channels — per-channel capabilities matrix
 - 11.0 Channel adapter contract (Go interface + value types)
 - 11.1 Telegram
 - 11.2 Max (max.ru)
 - 11.3 Slack
 - 11.4 Discord
 - 11.5 Microsoft Teams
 - 11.6 Lark / Feishu
 - 11.7 WhatsApp
 - 11.8 Viber
 - 11.9 Email (deep)
 - 11.10 ntfy / Gotify
 - 11.11 Generic outbound webhook
 - 11.12 Diary (Markdown + PDF + HTML)
 - 11.13 Feature matrix summary
 - 11.14 null:// sandbox channel (test-only)
- §12. Messaging flows
- §13. Templating & message composition
- §14. Localization (i18n)
- §15. Security model
 - 15.1 Transport-level (channel signature verification)
 - 15.2 Sender-level (allowlist + verified subscribers)
 - 15.3 Content-level (parsing & sanitization)
 - 15.4 Credential handling
 - 15.5 Webhook ingestion (defense-in-depth)
- §16. Multi-tenancy & isolation
 - 16.1 Data retention & privacy
- §17. Observability & SLOs
 - 17.1 OpenTelemetry pipeline
 - 17.2 Metrics catalogue
 - 17.3 Span model
 - 17.4 SLOs
 - 17.4.1 Per-channel SLO budgets
 - 17.5 Health probes (livez / readyz / startupz)
 - 17.6 doctor CLI
- §18. Flavors (the implementations)
 - 18.1 Common flavor contract
 - 18.2 Project Herald (pherald)
 - 18.2.1 Investigation-before-Fixing flow
 - 18.2.2 Criticality determination
 - 18.2.3 Type classification (Universal §11.4.16 mapping)
 - 18.2.4 Attachment validation + storage
 - 18.2.5 Claude Code project-session integration
 - 18.3 System Herald (sherald)
 - 18.4 Build Herald (bherald)
 - 18.5 Deploy Herald (dherald)
 - 18.6 Alert Herald (aherald)
 - 18.7 Schedule Herald (scherald)
 - 18.8 Incident Herald (iherald)
 - 18.9 Release Herald (rherald)
 - 18.10 Compliance Herald (cherald)
 - 18.11 Future flavors
- §19. Diary
- §20. Extensibility

- [§21. Supply chain & release engineering](#)
 - [§22. Constitution integration](#)
 - [§23. Specification documents \(change rule\)](#)
 - [§24. Documentation](#)
 - [24.1 Machine-readable API specifications](#)
 - [§25. Testing](#)
 - [§26. Operations](#)
 - [26.5 Operator quickstart \(5-minute Docker Compose\)](#)
 - [26.6 Disaster recovery](#)
 - [§27. Roadmap](#)
 - [27.3 Cost considerations](#)
 - [§28. Notes & open questions](#)
 - [§29. Changelog](#)
 - [29.1 V1 → V2 changes \(2026-05-19\)](#)
 - [29.2 V2 → V3 changes \(2026-05-20\)](#)
 - [§30. V2 self-review log](#)
 - [30.6 V3 r1 review log \(this revision\)](#)
 - [§31. Project integration contract](#)
 - [§32. Inbound processing pipeline](#)
 - [§33. LLM / agent dispatch](#)
 - [§34. Reply protocol \(queued → processing → result\)](#)
 - [§35. Reports + state-tracking documents \(versioned fan-out with Git linkage\)](#)
 - [§36. Outbound multi-format attachments \(.md + .html + .pdf + .docx\)](#)
-

§1. Upstreams

All existing project upstreams:

- **GitHub** (main repository): `git@github.com:vasic-digital/Herald.git`
- **GitLab**: `git@gitlab.com:vasic-digital/herald.git`
- **GitFlic**: `git@gitflic.ru:vasic-digital/herald.git`
- **GitVerse**: `git@gitverse.ru:vasic-digital/Herald.git`

The local origin remote is a fan-out (one fetch URL + four push URLs). A single `git push origin <branch>` propagates to all four hosts (Helix Constitution §2.1 / Herald Constitution §103).

§2. Mission and scope

2.1 What Herald is

Herald is the **bi-directional event fan-out mechanism**. It receives input from one or more **sources** and dispatches the resulting content to one or more **destinations** (channels). Depending on the implementation (Flavor) of Herald, sources and destinations are heterogeneous: a single input type or many; a single output channel or many.

For example, the input may be the result of a CI pipeline execution (a build report, a test summary, a security-scan finding). Herald enriches, normalizes, routes, and dispatches that input to messaging channels (Telegram, Slack, Max, Email, Markdown diary, etc.) so that human and machine subscribers can be informed and can interact back.

The possibilities are not limited. The structure of the system **MUST** be **hierarchical**: shared abstractions live in the **commons** layers (closest to the root); flavor-specific implementations live in `flavors/<flavor>/`.

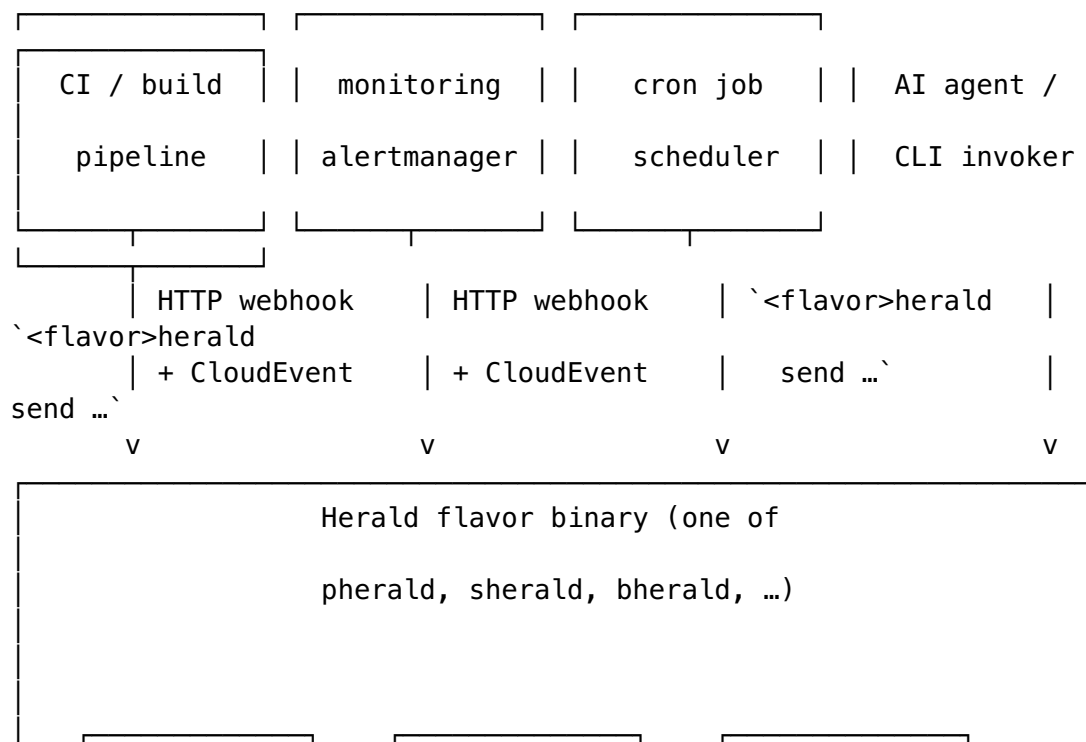
Note: We **MUST NOT** be obligated to follow this hierarchical structure rigidly when a parent project's specific custom flavor must exist privately or in a different location. Flexibility is mandatory and fully supported.

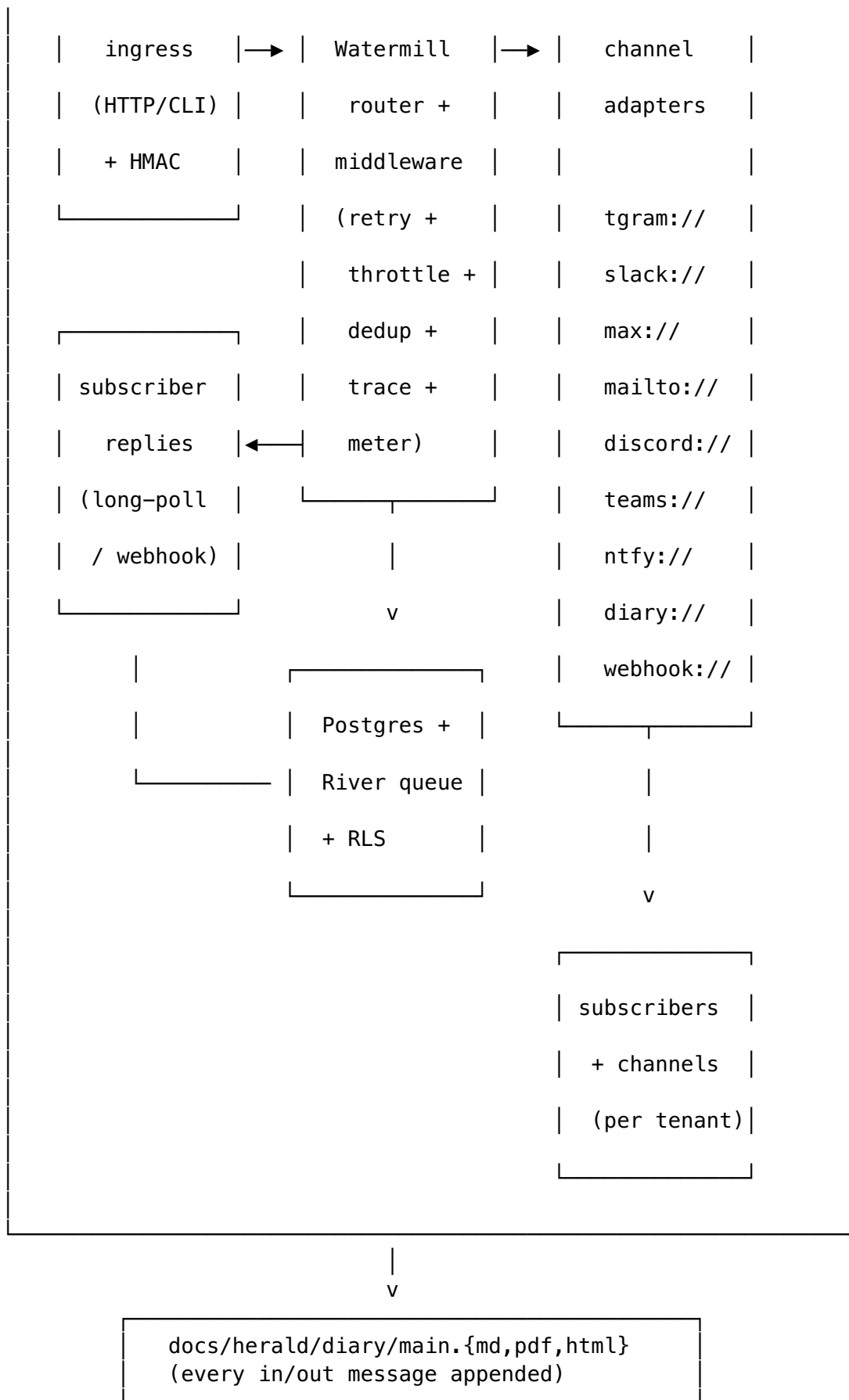
2.2 What Herald is NOT

To bound scope (§102 Herald Constitution — mission boundary):

- Herald is **not** a general-purpose chat application; it is an event-driven fan-out + inbound-reply processor.
- Herald is **not** a general-purpose monitoring/observability platform — it integrates *with* them via webhook ingestion (Prometheus Alertmanager, Grafana, Datadog, PagerDuty, OpsGenie) but does not replace them.
- Herald is **not** a transactional/marketing email service provider in itself — it integrates *with* ESPs (SendGrid, Resend, Postmark) as one of its delivery transports.
- Herald is **not** a workflow orchestration platform — Temporal/Argo/Step Functions remain the right tool for that; Herald can be triggered by them and can dispatch notifications about them.

2.3 Architecture diagram





§3. Execution model

3.1 Single binary, two modes

Every Herald flavor is a **single statically-linked Go binary** with **Cobra-style subcommands** (per R-02 — cleanest fit, lowest deployment footprint, no duplicate auth/config plumbing). The same binary serves both runtime modes:

- **One-shot mode** — `<flavor>herald send ...`: connects, transmits a single event, awaits delivery acknowledgement with a configurable deadline, exits non-zero on failure. Designed for CI integration, cron jobs, AI-agent invocation, pipeline steps.
- **Daemon mode** — `<flavor>herald serve`: long-running listener; blocks on (a) the HTTP ingress for webhook events, (b) per-channel subscriber-reply loops (Telegram long-poll, Slack Socket Mode, Email IMAP, Discord gateway, etc.), (c) scheduled jobs from the River queue.

Both modes share the same configuration loader, channel adapters, storage layer, and observability stack — only the entry point and process lifecycle differ.

Graceful shutdown semantics. Both modes **MUST**:

1. Trap `SIGTERM` and `SIGINT`.
2. Stop accepting new ingress requests immediately (`/readyz` returns 503).
3. Drain in-flight work with a configurable grace period (default 30 s, env `HERALD_SHUTDOWN_GRACE`).
4. Refuse `Send()` calls that arrive after grace begins (returning `context.Canceled`).
5. Flush the OpenTelemetry exporter (`tracerProvider.Shutdown(ctx)` with its own 5 s sub-deadline).
6. `Close()` the Postgres pool and Redis client.
7. Exit with code 0 if drain completed cleanly, 1 if grace expired with work still in flight.

A second `SIGTERM` within the grace window forces immediate exit. `SIGKILL` is uncatchable by definition; operators **MUST** size the grace period for their workload's tail latency.

Additional subcommands:

- `<flavor>herald doctor` — verifies environment health (Postgres ping, Redis ping, channel-credential validity, DKIM/SPF/DMARC records, port availability).
- `<flavor>herald migrate` — applies database migrations (idempotent).
- `<flavor>herald deadletter [list | replay <id> | purge]` — operate on the dead-letter table.
- `<flavor>herald subscriber [list | add | verify <token>]` — manage subscribers.
- `<flavor>herald digest ...` — generate scheduled summaries (digests, weekly reports).

3.2 Distribution + invocation surfaces

Herald applications are CLI binaries primarily designed for:

- **CI integration** — invoked from GitHub Actions / GitLab CI / Jenkins / CircleCI / Drone steps.
- **Pipelines** — invoked from build/deploy scripts.
- **AI CLI agents** — invoked by Claude Code, OpenCode, Cursor, Aider, etc., as a structured way to surface progress and accept human feedback.
- **cron** — scheduled checks and digests.
- **Webhook recipients** — running in daemon mode behind a reverse proxy.

Some Herald application names (illustrative — flavors are fully enumerated in §18):

- pherald for **Project Herald** (development-lifecycle events).
- sherald for **System Herald** (OS/host/service events).
- bherald for **Build Herald** (CI/CD build events).
- dherald for **Deploy Herald** (release/deploy events).
- aherald for **Alert Herald** (monitoring alert routing).
- scherald for **Schedule Herald** (cron/scheduled-task notifications).
- iherald for **Incident Herald** (incident management).
- rherald for **Release Herald** (release lifecycle).
- cherald for **Compliance Herald** (audit/compliance events).

3.3 Configuration & credentials

Configuration precedence (12-factor aligned, per R-04):

1. Explicit CLI flag (highest precedence).
2. Shell-exported environment variable (from `.bashrc` / `.zshrc` / `k8s env` / `Docker -e`).
3. Value from `.env` (fallback only — does NOT override exported shell vars).
4. Compiled default (lowest).

`.env` MUST BE git-ignored. `.env.example` is the documented template, committed. An opt-in `--env-file-override` flag maps to `godotenv.Overload()` for the rare case operators want the file to win (one-off rescues, debugging).

Configuration file layout (`config.toml`, optional, loaded after `.env`):

```
[herald]
flavor      = "pherald"
tenant_id   = "00000000-0000-0000-0000-000000000001" # default
              if multi-tenancy disabled
locale      = "en-US"
diary_root  = "" # blank →
              discover via parent-walk
admin_port  = 24090 # /
              livez, /readyz, /metrics, pprof
http_port   = 24091 # webhook
              ingress + reply webhooks
```

```
[postgres]
```



```

dsn          = "${HERALD_POSTGRES_DSN}"           # env
              interpolation
max_conns    = 20
rls_enforced = true

[redis]
addr          = "${HERALD_REDIS_ADDR}"
acl_user      = "${HERALD_REDIS_USER}"
acl_password  = "${HERALD_REDIS_PASSWORD}"

[channels]
# Apprise-style URLs (see §6.1)
default = [
    "tgram://${TELEGRAM_BOT_TOKEN}/${TELEGRAM_CHAT_ID}?
      tags=oncall,prod",
    "slack://${SLACK_TOKEN}/${SLACK_CHANNEL}?tags=oncall",
    "mailto://herald@example.com?tags=audit",
    "diary:///tags=*" # always log to diary
]

[observability]
otlp_endpoint = "http://otel-collector:4317"
log_level     = "info"

```

3.4 Worker pools, concurrency, and hot-reload

Worker pool sizing. `herald serve` runs three independently-sized worker pools:

Pool	Default size	Configurable	Workload
HTTP ingress	2 × NumCPU	<code>[server].http_workers</code>	accepts inbound CloudEvents + webhook deliveries; CPU-bound on signature verification + JSON parsing template render + preference filter + tag matching; mostly CPU + small Postgres lookups
Router/ dispatch	4 × NumCPU	<code>[router].workers</code>	I/O-bound HTTP calls to channel APIs (Telegram, Slack, ...);
River channel- delivery	8 × NumCPU (capped at <code>[river].max_workers</code> , default 64)	<code>[river].workers</code>	

Pool	Default size	Configurable	Workload
			bottlenecked by upstream rate limits

Operators tune via env vars (HERALD_HTTP_WORKERS, HERALD_ROUTER_WORKERS, HERALD_RIVER_WORKERS) or config.toml. Sizing guidance:

- **Single-tenant small** (≤ 1 alert/sec): defaults (~16 workers total on a 2-vCPU host).
- **Multi-tenant production** (~50 alerts/sec sustained): bump [river].max_workers to $\min(256, 8 \times \text{tenants})$; tune [postgres].max_conns to [router].workers + [river].workers + 4 (admin connections reserved).
- **High-burst CI fanout** (~500 alerts/min in 10-sec spikes): rely on River's backpressure rather than over-provisioning workers; River queues durably in Postgres and drains as channels free up.

Hot-reload via SIGHUP. herald serve traps SIGHUP and re-reads config.toml + .env + channel_addresses (database-backed) without dropping in-flight work. Reload is constrained to **safe-to-change** sections:

Reloadable on SIGHUP	NOT reloadable (require process restart)
[channels].* (channel addresses + tags)	[server].http_port / admin port
[router].workers (resized live)	[postgres].dsn (connection identity)
[river].workers / max_workers	[redis].addr
[observability].log_level	[observability].otlp_endpoint (exporter identity)
[security].allowlists.*	binary version / migrations
[rate_limits].* (token bucket caps)	RLS policy changes (DB-side)

Reload semantics:

1. Compute the diff between in-memory config and freshly-loaded config.
2. Reject if any "NOT reloadable" key changed — log ERROR config reload rejected: <key> requires restart; old config remains active.
3. For reloadable keys, apply atomically (worker pools resize via a lockless swap; channel adapters re-initialize against new addresses; rate-limit buckets refresh with new caps; in-flight work continues against the live config snapshot it captured).
4. Emit digital.vasic.herald.system.config.reloaded event with the diff for audit.

No hot-reload in one-shot mode: herald send reads config once at startup and exits; SIGHUP is ignored.

3.5 Time / clock abstraction

Herald never calls `time.Now()` directly outside of `commons/clock`. The clock abstraction lets tests fast-forward time (essential for quiet-hours, batching windows, retry backoff, idempotency TTLs, escalation chains).

```
package commons // L0

// Clock is the time-source abstraction. Production uses
//   RealClock;
// tests use FakeClock with controllable Now()/After()/NewTimer().
type Clock interface {
    Now() time.Time
    Since(t time.Time) time.Duration
    Sleep(d time.Duration)
    After(d time.Duration) <-chan time.Time
    NewTimer(d time.Duration) Timer
    NewTicker(d time.Duration) Ticker
}

// RealClock wraps the stdlib time package. Used in production.
type RealClock struct{

// FakeClock is the test implementation. Advance(d) moves the
//   clock
// forward by d and fires any pending timers/tickers whose
//   deadlines
// the advance crosses. Available in commons/clock/clocktest.
type FakeClock struct {
    // unexported state
}

// Default exports a process-global Clock – Herald's CLI bootstrap
// sets it to RealClock; tests swap it in TestMain.
var Default Clock = RealClock{}
```

Rationale: this is the same pattern that github.com/benbjohnson/clock popularised; `commons/clock` ships a minimal in-tree implementation rather than depending on a third-party module that has no maintainer activity since 2022. The interface surface is intentionally small — only what Herald needs.

Discipline: anyone calling `time.Now()` outside `commons/clock` is a bug — the lint rule `herald-no-direct-time-now` (custom `golangci-lint` analyzer, planned) flags violations.

§4. Event model & wire format

4.1 CloudEvents v1.0 as canonical envelope

Herald speaks **CloudEvents v1.0** (CNCF graduated, Jan 2024) natively on every external boundary — ingress, internal transport, audit/diary record. Inside the binary, events are passed as `cloudevents.Event` values from `github.com/cloudevents/sdk-go/v2`.

Required CloudEvents attributes for every Herald event:

Attribute	Source	Example
specversion	constant "1.0"	"1.0"
id	UUIDv7 (natural ordering by time)	"01931a7c-3f4e-7000-9abc-def012345678"
source	URI of the emitter	"https://ci.example.com/ pipelines/4231"
type	reverse-DNS event type	"digital.vasic.herald.ci.failed"
time	RFC 3339 timestamp	"2026-05-19T17:30:00Z"
datacontenttype	media type of data	"application/json"
subject	target hint (channel address, tag, route key)	"tag:oncall,prod" or "channel:slack-incidents"

Two wire modes accepted on HTTP ingress:

- **Structured mode** — single JSON object:
`{"specversion":"1.0","id":"...","type":"...","data":{"..."}}`.
- **Binary mode** — HTTP headers carry attributes (`ce-id`, `ce-type`, `ce-source`, ...), body is the raw data payload.

Herald-specific extension attributes:

- `heraldtenant` — tenant UUID (overrides default).
- `heraldidempotencykey` — explicit dedup key (overrides `id` if present).
- `heraldpriority` — low / normal / high / urgent (ntfy-style 1–5 mapping).

4.2 Herald event-type taxonomy

CloudEvents type is a reverse-DNS string. Herald's namespace is `digital.vasic.herald.*`.

Subtree	Used by	Examples
digital.vasic.herald.ci.*	bherald	.build.queued, .build.started, .build.
digital.vasic.herald.project.*	pherald	.task.opened, .task.assigned, .task.cl
digital.vasic.herald.system.*	sherald	.host.cpu_high, .host.disk_full, .serv
digital.vasic.herald.deploy.*	dherald	.deploy.started, .deploy.succeeded, .d
digital.vasic.herald.alert.*	aherald	.alert.firing, .alert.resolved, .alert
digital.vasic.herald.schedule.*	scherald	.job.started, .job.succeeded, .job.fai
digital.vasic.herald.incident.*	iherald	.incident.opened, .incident.escalated
digital.vasic.herald.release.*	rherald	.release.tagged, .release.published, .
digital.vasic.herald.compliance.*	cherald	.audit.access, .policy.violation, .cer
digital.vasic.herald.reply.*	all	.reply.received, .command.parsed, .com
digital.vasic.herald.delivery.*	internal	.delivery.accepted, .delivery.routed,

4.3 Idempotency keys

Per R-05 + research Topic 4: **at-least-once delivery with deterministic dedup**.

- Every ingress accepts an Herald-Idempotency-Key HTTP header (or heraldidempotencykey CE extension, or CLI flag `--idempotency-key`).
- If absent, Herald falls back to the CloudEvents id.
- Dedup is enforced via a Postgres `idempotency_keys` table:

```
CREATE TABLE idempotency_keys (
  tenant_id          UUID NOT NULL,
  idempotency_key    TEXT NOT NULL,
  request_hash       BYTEA NOT NULL,           -- SHA-256 of
  canonical_request_body
  response_id        UUID,                     -- pointer to
  first_response
  locked_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
  expires_at         TIMESTAMPTZ NOT NULL DEFAULT (now() +
  interval '24 hours'),
  PRIMARY KEY (tenant_id, idempotency_key)
) WITH (FILLFACTOR = 80);
```

```
ALTER TABLE idempotency_keys ENABLE ROW LEVEL SECURITY;
ALTER TABLE idempotency_keys FORCE ROW LEVEL SECURITY;
CREATE POLICY idem_isolation ON idempotency_keys
  USING (tenant_id = current_setting('app.tenant_id')::uuid);
```

- Dedup INSERT ... ON CONFLICT DO NOTHING is in the **same transaction** as the channel-fanout enqueue. Stripe-pattern (cite: Brandur's writeup).
- Redis is used as a **fast-path negative cache** (SET NX EX <ttl>) in front of Postgres, never as sole authority.
- TTL: 24h for synchronous API replies (matches Stripe); 7d for delivery-side dedup (matches reply-queue retention).
- Replay (same key, same body) returns the cached response; replay (same key, *different* body) returns HTTP 409 Conflict.

§5. Architecture overview

5.1 Components

A Herald flavor binary is composed of these in-process components:

1. **Ingress layer** — HTTP server (CloudEvents binding) + CLI command parser. Verifies signatures (§15.5), enforces idempotency (§4.3), and emits a Watermill message.
2. **Watermill router** — central message bus. Channel adapters, evaluators, and middleware (retry, dedup, throttle, trace, meter) are registered as `HandlerFuncs` on a single router.
3. **Channel adapters** — one per supported channel; implement the `Channel` interface (§10).
4. **Subscriber-reply consumers** — per-channel inbound loops (Telegram `getUpdates` long-poll or webhook, Slack Socket Mode / Events API webhook, Email IMAP, Discord gateway, etc.).
5. **Storage layer** — Postgres (durable state, RLS-isolated per tenant) + Redis (rate-limit token buckets, fast-path dedup, transient state).
6. **Queue backend** — Postgres + River as default; NATS JetStream as opt-in for multi-replica fan-out.
7. **Observability layer** — OpenTelemetry SDK (traces + metrics + logs), OTLP exporter, `/metrics` for Prometheus scrape, `/livez` / `/readyz` / `/startupz` probes.

5.2 Internal routing (Watermill)

Per research Topic 2: **Watermill** (github.com/ThreeDotsLabs/watermill, ~8.5k stars, v1.4+) is the routing kernel inside `commons_messaging`.

- Each channel adapter (Telegram, Slack, ...) is registered on a single `message.Router` as a `HandlerFunc`.
- Cross-cutting concerns (correlation ID, retry, poison-queue, metrics, throttling, tracing) are implemented as **Watermill middlewares** rather than per-channel code — eliminating ~60% of the plumbing the team would otherwise write.
- Pubsub backend is **pluggable**: Postgres adapter (default), NATS adapter (opt-in), in-memory adapter (tests).

5.3 Queue backend (Postgres + River default, NATS opt-in)

Per research Topic 3:

- **Default: Postgres + riverqueue/river** — transactional enqueue (jobs only run if the originating tx commits), built-in retries with backoff, uniqueness constraints. Zero new infrastructure since Postgres is already required.
- **LISTEN/NOTIFY** wakes consumers sub-millisecond between polls.
- **Opt-in alternative: NATS JetStream** — for deployments needing fan-out across multiple Herald instances, edge sites, or sub-millisecond cross-host latency (~200k–400k msg/s with persistence, built-in KV/Object stores).
- **Kafka is intentionally NOT the default** — overkill for Herald's alert-volume profile (dozens to low-thousands of events/sec per tenant). Document as Watermill-pluggable for operators who already run Kafka.

5.4 Retries & dead-lettering

Per research Topic 5 and AWS arch blog "Exponential Backoff and Jitter":

- **Backoff curve: decorrelated jitter** — `sleep = min(cap, random_between(base, prev_sleep * 3))` with `base = 1s`, `cap = 5min`.
- **Max attempts**: 8 per message for transient errors. Empirically: ~30min worst-case retry window, matching what humans expect of an alert system.
- **No retries on 4xx** from upstream channels — 400 / 401 / 403 / 404 from Telegram/Slack/etc. are permanent.
- **Implementation**: Watermill Retry middleware in front of channel handlers.
- **Dead letter sink: Postgres dead_letters table** (NOT a Redis list — operators need to query, triage, replay):

```
CREATE TABLE dead_letters (  
  id                UUID PRIMARY KEY DEFAULT uuidv7(),  
  tenant_id         UUID NOT NULL,  
  original_event_id UUID NOT NULL,  
  channel           TEXT NOT NULL,  
  attempt_count     INTEGER NOT NULL,  
  last_error        TEXT,  
  payload_jsonb     JSONB NOT NULL,  
  created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),  
  triaged_at        TIMESTAMPTZ,  
  triage_status     TEXT -- new | acknowledged | replayed |  
                        discarded  
);  
  
ALTER TABLE dead_letters ENABLE ROW LEVEL SECURITY;  
ALTER TABLE dead_letters FORCE ROW LEVEL SECURITY;  
CREATE POLICY dl_isolation ON dead_letters  
  USING (tenant_id = current_setting('app.tenant_id')::uuid);
```

- Every entry into `dead_letters` also emits a `digital.vasic.herald.delivery.dead_lettered` CloudEvent — Herald observes its own failures.
- CLI: `<flavor>herald deadletter list | replay <id> | purge`.

5.4.1 Outbound idempotency (channel-side composition)

The §4.3 idempotency table guards **ingress**: same event id arriving twice produces one logical delivery. It does NOT, however, guard against the case where Herald has already called `Channel.Send` once and the call's *response* was lost (network blip, container OOM, channel-API timeout). Under at-least-once semantics, the retry layer (§5.4) WILL re-call `Channel.Send` — and a naive channel adapter would re-post the message, producing visible duplicates.

Resolution: every `Channel.Send` invocation MUST pass an **outbound-idempotency key** derived deterministically from `(tenant_id, EventID, ChannelID, ChannelAddressID)`. Adapters that support upstream idempotency (Slack `idempotency_key` extension, Stripe-style `Idempotency-Key` header on REST channels, Email `Message-ID` header) MUST forward this key. Adapters whose upstream does NOT support idempotency (raw SMTP, Telegram Bot API, Discord) MUST consult a Postgres `outbound_dedup` table inside the same River job transaction:

```

CREATE TABLE outbound_dedup (
  tenant_id          UUID NOT NULL,
  outbound_key       TEXT NOT NULL,          --
  "<event_id>:<channel>:<channel_address_id>"
  sent_at            TIMESTAMPTZ NOT NULL DEFAULT now(),
  channel_msg_id     TEXT,
  "-- adapter's Receipt.ChannelMsgID"
  expires_at         TIMESTAMPTZ NOT NULL DEFAULT (now() +
    interval '24 hours'),
  PRIMARY KEY (tenant_id, outbound_key)
);
ALTER TABLE outbound_dedup ENABLE ROW LEVEL SECURITY;
ALTER TABLE outbound_dedup FORCE ROW LEVEL SECURITY;
CREATE POLICY od_isolation ON outbound_dedup
  USING (tenant_id = current_setting('app.tenant_id')::uuid)
  WITH CHECK (tenant_id =
    current_setting('app.tenant_id')::uuid);

```

The adapter wraps each send in: `INSERT ... ON CONFLICT DO NOTHING;` if row already existed, return cached Receipt without re-calling upstream. The window is short (24h) because we only need to cover the retry-storm window — far shorter than ingress idempotency.

5.5 Webhook ingestion

Per R-16 + research Topic 8. Every webhook source registers a per-source signing secret (rotatable) in the `webhook_sources` table.

The ingress handler enforces, in order:

1. **HMAC-SHA256 signature verification** with `crypto/hmac.Equal` (constant-time) against the source-configured header:
 - GitHub: `X-Hub-Signature-256`
 - Stripe: `Stripe-Signature` (sign timestamp.payload)
 - Generic CloudEvents source: `Webhook-Signature`
2. **Timestamp window** ≤ 5 min vs. server clock (replay protection).
3. **Delivery-ID dedup** via the idempotency table (GitHub's `X-GitHub-Delivery`, Stripe's event id).
4. **Optional IP allowlist** as L4 defense-in-depth.

Verified payloads are normalized into a CloudEvent and handed to the Watermill router.

webhook_sources schema:

```

CREATE TABLE webhook_sources (
  id                UUID PRIMARY KEY DEFAULT uuidv7(),
  tenant_id         UUID NOT NULL,
  name              TEXT NOT NULL,          -- "github-
  "push-prod", "stripe-webhook", ..."
  signature_kind    TEXT NOT NULL,         -- 'hmac-
  "sha256' | 'stripe' | 'telegram-secret-token' | 'dkim' |"
  "'none'"

```



```

signature_header TEXT NOT NULL,           -- 'X-Hub-
    Signature-256', 'Stripe-Signature', ...
secret_encrypted BYTEA NOT NULL,          -- pgcrypto
    pgp_sym_encrypt
secret_rotated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
ip_allowlist INET[],                      -- optional
    L4 defense-in-depth
replay_window_s INTEGER NOT NULL DEFAULT 300, -- 5 min
enabled BOOLEAN NOT NULL DEFAULT true,
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
UNIQUE (tenant_id, name)
);
CREATE INDEX webhook_sources_tenant_idx ON webhook_sources
    (tenant_id) WHERE enabled = true;
ALTER TABLE webhook_sources ENABLE ROW LEVEL SECURITY;
ALTER TABLE webhook_sources FORCE ROW LEVEL SECURITY;
CREATE POLICY ws_isolation ON webhook_sources
    USING (tenant_id = current_setting('app.tenant_id')::uuid)
    WITH CHECK (tenant_id =
        current_setting('app.tenant_id')::uuid);

```

Secrets are rotatable: `<flavor>herald webhook rotate <name>` generates a new secret, returns it once, and starts accepting the new signature while continuing to accept the old one for `replay_window_s × 4` (default 20 min) to drain in-flight deliveries.

Dev / behind-NAT support: `smee.io` and `gh webhook forward` are documented as supported proxy paths.

5.6 Orchestration of long-running operations (Temporal, opt-in)

Per research Topic 7:

- **Default routing** — Watermill handlers + River jobs cover the vast majority of fan-out cases.
- **Temporal sidecar (opt-in)** — required for operations that genuinely need durable state across hours/days, deterministic replay, and human-friendly timeline UIs:
 - **Incident escalation chains** (page A → wait 5min → escalate to B → wait 10min → page C). Used by `iherald`.
 - **Scheduled digest builders** with human-in-the-loop acknowledgment.
 - **Multi-channel orchestrated rollouts** (used by `dherald` / `rherald`).
- Temporal is explicitly **not** the default — its operational footprint (separate cluster, gRPC, server upgrades) violates Herald's "lightweight CLI" ethos.

5.7 Ingress API URLs (HTTP surface)

Herald's HTTP ingress (port 24091 default per §9.4) exposes a small, stable URL surface. All paths under `/v1/` are public; everything under `/admin/` is admin-port only (port 24090).

Method	Path	Purpose
POST	<code>/v1/events</code>	

Method	Path	Purpose
		Canonical CloudEvents ingest (binary + structured modes). Body MUST be a CloudEvent per §4.1.
POST	/v1/send	Convenience REST ingest for ad-hoc senders that don't speak CloudEvents — JSON body {type, source, data, tags?, idempotency_key?, priority?} is wrapped into a CloudEvent server-side.
POST	/webhooks/{source_id}	Verified webhook receiver — source_id is a UUID matching webhook_sources.id; the handler enforces §5.5 signature + replay + dedup before forwarding.
GET	/v1/events/{id}	Idempotent replay of a previously-accepted event (returns 200 with cached response if id matches, 409 if id matches but body differs).
GET	/v1/deadletters	List dead-lettered messages (RLS-isolated to caller's tenant).
POST	/v1/deadletters/{id}/replay	Replay a dead-lettered message.
GET	/v1/subscribers / POST /v1/subscribers	Subscriber CRUD (operator role required).
POST	/v1/subscribers/{id}/forget	GDPR right-to-erasure (§16.1).
GET	/v1/subscribers/{id}/export	GDPR right-to-portability — emits JSON bundle.
GET	/v1/channels	List configured channel addresses for the caller's tenant.
GET	/livez	Liveness probe (admin port — §17.5).
GET	/readyz	Readiness probe (admin port).
GET	/startupz	Startup probe (admin port).
GET	/metrics	Prometheus scrape (admin port).
GET	/admin/version	Build info (commit SHA, build time, Go version).
GET	/admin/pprof/*	Go runtime profiling (admin port; loopback-only by default; opt-in [admin].pprof_external=true).

Auth model:

- /v1/events and /v1/send accept either (a) HTTP Basic auth with a per-tenant ingest_token from the tenants table, or (b) signed bearer token in Authorization: Bearer <jwt> (Herald validates against JWKS configured in [auth.oidc]). Self-hosted operators MAY enable mTLS via a reverse proxy.
- /webhooks/{source_id} does its own signature verification (§5.5) — no token required at the HTTP layer.

- `/v1/subscribers*` requires a JWT with operator-role claim.
- Admin endpoints listen only on the admin port (default loopback or trusted-network only).

Versioning: the `/v1/` prefix is the stable contract; breaking changes ship as `/v2/` with at least 6 months of `/v1/` co-existence. The OpenAPI schema (`docs/api/openapi.v1.yaml`) is the source of truth — `<flavor>herald openapi` prints the embedded spec.

Rate limits: per-tenant token bucket (§16, default 1000 req/min per tenant on `/v1/` events); per-IP secondary bucket for unauthenticated webhook receivers (default 60 req/min/IP).

§6. Channel addressing & routing

6.1 URL-scheme channel addresses (Apprise-style)

Per research Topic 1 (notification-platforms): adopt Apprise's URL-scheme channel model. Each destination is a single string that fully captures credentials + endpoint + targeting:

```
tgram://<bot_token>/<chat_id>?tags=oncall,prod
slack://<token_a>/<token_b>/<token_c>/<channel>?tags=oncall
max://<bot_token>/<chat_id>?tags=ru,prod
mailto://<user>:<pass>@<smtp_host>:<port>?
tags=audit&from=herald@example.com
discord://<webhook_id>/<webhook_token>?tags=community
teams://<incoming_webhook_path>?tags=corp
lark://<app_id>:<app_secret>@<chat_id>?tags=apac
ntfy://<server>/<topic>?priority=high&tags=mobile
gotify://<token>@<server>?priority=5
webhook://<endpoint_url>?secret=<hmac_secret>&format=cloudevent
diary://?tags=*&format=markdown
```

This URL form maps 1:1 to a row in Postgres `channel_addresses` and lets credentials be `.env`-interpolated at config-load time so the URLs themselves stay git-safe in committed `config.toml`.

6.2 Tag-based fan-out

Per Apprise's tag model: every channel address is annotated with one or more **tags**. Senders address tags, not specific channels:

- **OR-union by default:** `--tags=oncall,prod` matches any channel tagged `oncall` OR `prod`.
- **AND-intersection** when explicitly comma-joined inside a single argument: `--tags=oncall+prod` matches channels tagged `oncall` AND `prod`.
- Tag `*` is the wildcard (matches every channel).
- Tag `!oncall` means "exclude oncall" (intersect with negation).

Tag→channel mapping lives in the `channel_addresses` table; senders never reference channel addresses directly. This decouples sender code from topology — the same event can fan out to a different mix of channels per environment without code change.

channel_addresses schema:

```
CREATE TABLE channel_addresses (
  id          UUID PRIMARY KEY DEFAULT uuidv7(),
  tenant_id   UUID NOT NULL,
  channel     TEXT NOT NULL,          -- "tgram",
  "slack", "mailto", ...
  address_url TEXT NOT NULL,          -- "tgram://$
    {BOT_TOKEN}/{CHAT_ID}?..." (env-interpolated at load time)
  tags        TEXT[] NOT NULL DEFAULT '{}', --
    ['oncall','prod']
  priority_floor INTEGER NOT NULL DEFAULT 1,
    -- ntify 1..5; messages below floor are dropped for this
    address
  enabled      BOOLEAN NOT NULL DEFAULT true,
  last_health_at TIMESTAMPTZ,
  last_health_ok BOOLEAN,
  last_health_err TEXT,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (tenant_id, address_url)
);
CREATE INDEX ch_addr_tags_idx ON channel_addresses USING gin
  (tags) WHERE enabled = true;
CREATE INDEX ch_addr_tenant_idx ON channel_addresses (tenant_id,
  channel) WHERE enabled = true;
ALTER TABLE channel_addresses ENABLE ROW LEVEL SECURITY;
ALTER TABLE channel_addresses FORCE ROW LEVEL SECURITY;
CREATE POLICY ca_isolation ON channel_addresses
  USING (tenant_id = current_setting('app.tenant_id')::uuid)
  WITH CHECK (tenant_id =
    current_setting('app.tenant_id')::uuid);
```

The `address_url` MAY contain `${ENV_VAR}` placeholders that are interpolated at config load time (so secrets stay out of the row body — only the placeholder is persisted; secrets remain in `.env` / shell env).

6.3 HeraldBranding (per-flavor visual identity)

Per Apprise's `AppriseAsset`: a `HeraldBranding` struct injected per-flavor:

```
type Branding struct {
  AppName      string // "Project Herald"
  BinaryName    string // "pherald"
  IconURL       string // for rich embeds
  AccentColorHex string // "#2C7BE5"
  DefaultFooter string // "Sent by pherald 1.0 · github.com/
    vasic-digital/Herald"
}
```

Channel adapters consult Branding when rendering rich messages (Slack Block Kit headers, Discord embed author, Adaptive Card Container accent color, Email From-name).

§7. Subscriber model

7.1 Identity & reconciliation (per-channel-id + operator alias)

Per R-07 + research Topic 3 (notification-platforms): the **matterbridge model** is the default — per-channel-id is the source of truth. A subscriber on Telegram is identified by (channel:"tgram", chat_id:"42"); the same human on Slack is (channel:"slack", user_id:"U0123"); these are **separate entries** unless explicitly linked.

Schema:

```
CREATE TABLE subscribers (  
  id          UUID PRIMARY KEY DEFAULT uuidv7(),  
  tenant_id   UUID NOT NULL,  
  handle      TEXT,                                -- operator-  
              assigned, e.g. "alice"  
  display_name TEXT,  
  locale      TEXT DEFAULT 'en-US',  
  timezone    TEXT DEFAULT 'UTC',  
  roles       TEXT[] NOT NULL DEFAULT '{}',        -- e.g.  
              ['operator','reader','ic']  
  metadata    JSONB NOT NULL DEFAULT '{}'::jsonb,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),  
  UNIQUE (tenant_id, handle)                       -- handle is  
              unique within a tenant  
);  
CREATE INDEX subscribers_tenant_idx ON subscribers (tenant_id);  
ALTER TABLE subscribers ENABLE ROW LEVEL SECURITY;  
ALTER TABLE subscribers FORCE ROW LEVEL SECURITY;  
CREATE POLICY sub_isolation ON subscribers  
  USING (tenant_id = current_setting('app.tenant_id')::uuid)  
  WITH CHECK (tenant_id =  
    current_setting('app.tenant_id')::uuid);  
  
CREATE TABLE subscriber_aliases (  
  subscriber_id UUID NOT NULL REFERENCES subscribers(id) ON  
    DELETE CASCADE,  
  channel       TEXT NOT NULL,  
  channel_user_id TEXT NOT NULL,  
  verified_at   TIMESTAMPTZ,                        -- self-claim  
              verification time  
  last_seen_at  TIMESTAMPTZ,  
  UNIQUE (channel, channel_user_id)  
);  
CREATE INDEX subscriber_aliases_sub_idx ON subscriber_aliases  
  (subscriber_id);
```

Linking flows:

- **Operator-mapped** (default): operator runs `pherald subscriber link --handle alice --tgram 42 --slack U0123`.
- **Self-claim** (opt-in per tenant): subscriber DMs a one-time token to the bot on each channel; the bot calls `pherald subscriber verify <token>` to record the alias.
- **Never inferred** — Herald does not guess identity across channels.

7.2 Preferences (Knock-style PreferenceSet)

Per research Topic 2 (notification-platforms): a Knock-inspired PreferenceSet JSON per subscriber:

```
{
  "categories": {
    "incidents": { "channels": ["slack","tgram","email"],
    "muted": false },
    "deploys": { "channels": ["slack"], "muted": false },
    "daily_digest": { "channels": ["email"], "muted": false }
  },
  "workflows": {
    "digital.vasic.herald.alert.firing": { "muted": false,
    "channels": ["tgram","slack"] },
    "digital.vasic.herald.alert.resolved": { "muted": true }
  },
  "quiet_hours": { "tz": "Europe/Belgrade", "start": "22:00",
  "end": "07:00", "exempt_categories": ["incidents"] },
  "channel_data": {
    "tgram": { "chat_id": "42", "thread_id": null },
    "slack": { "user_id": "U0123", "im_channel": "D098" }
  }
}
```

- The router consults PreferenceSet before dispatching: if muted for the workflow/category, drop (and log to diary as "filtered"); otherwise restrict to listed channels and respect quiet hours.
- `channel_data` carries provider-specific routing keys that don't fit the generic alias model (Slack `im_channel` for DMs, Telegram `thread_id` for forum-topic posts).

7.3 Quiet hours & throttling

- **Quiet hours** — TZ-aware window per subscriber; exempt categories override.
- **Throttling** — per-(tenant, subscriber, category) token bucket in Redis (`herald:t:<id>:rl:<sub>:<cat>` with INCR+EXPIRE); default cap configurable per tenant.
- **Batching** — per category, optional `batch_window` (e.g. 5min); events within the window collapse into a single digest message. Implementation: River job scheduled `batch_window` after the first event arrives.

7.4 Locale (i18n)

Per research Topic 6 (notification-platforms): [nicksnyder/go-i18n v2](#) with CLDR plural rules.

- One Bundle loaded at process start from `commons_messaging/locales/*.toml`.
- One Localizer per outgoing message, constructed from the subscriber's stored locale field (BCP-47 tag).
- Per-channel templates can override (emoji-heavy Telegram template vs sober email template, both for the same locale).
- Initial locales for V2: `en-US`, `sr-Latn-RS`, `ru-RU`. Additional locales as community contributes them.

7.5 AI-agent subscribers (distinct from human subscribers)

Herald is explicitly a target for AI CLI agent invocation (per §3.2). Agents are a fundamentally different kind of subscriber from humans: they invoke far more frequently, they don't sleep, they don't have quiet hours, and their failure modes (runaway loop, infinite retry, prompt-injection-driven misuse) need stronger throttling. V2 models them as a first-class subscriber kind.

Schema extension to subscribers:

- `kind` column (enum: `human` | `agent` | `service`). Default `human`. Indexes added on (`tenant_id`, `kind`).
- `agent` rows **MUST** carry a `metadata.agent_token_id` pointing at a row in `agent_tokens` (separate table, encrypted), used as the auth credential when the agent calls `/v1/send`.
- `agent` rows **MUST** carry `metadata.parent_subscriber_id` (the human operator who created the agent) so audit + GDPR right-to-erasure cascades correctly.

```
CREATE TABLE agent_tokens (  
  id          UUID PRIMARY KEY DEFAULT uuidv7(),  
  tenant_id   UUID NOT NULL,  
  subscriber_id UUID NOT NULL REFERENCES subscribers(id) ON  
    DELETE CASCADE,  
  token_hash  BYTEA NOT NULL,                -- argon2id  
    of the bearer token  
  name        TEXT NOT NULL,                 -- "claude-  
    code-laptop", "build-bot-prod"  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),  
  last_used_at TIMESTAMPTZ,  
  expires_at  TIMESTAMPTZ,                   -- nullable  
    = no expiry  
  revoked_at  TIMESTAMPTZ,  
  rate_limit_per_min INTEGER NOT NULL DEFAULT 60, -- default  
    60 req/min per token  
  UNIQUE (tenant_id, name)  
);  
ALTER TABLE agent_tokens ENABLE ROW LEVEL SECURITY;  
ALTER TABLE agent_tokens FORCE ROW LEVEL SECURITY;  
CREATE POLICY at_isolation ON agent_tokens
```

```

USING (tenant_id = current_setting('app.tenant_id')::uuid)
WITH CHECK (tenant_id =
    current_setting('app.tenant_id')::uuid);

```

Default throttling (operator-overridable per token):

Limit	Default	Burst	Notes
/v1/send requests	60 req/min/token	10	per the rate_limit_per_min column
/v1/events requests	30 req/min/token	5	tighter because CloudEvents body is unbounded
Outbound deliveries per token per min	120	20	a single agent CAN'T flood a tenant's channels
Quarantine threshold	3 consecutive rejected sends → token disabled for 30 min	—	auto-cool-down to prevent prompt-injection loops

Audit + observability: every agent send emits a span with `herald.subscriber.kind="agent"` and `herald.agent.token_id`; dashboards split agent vs human traffic by default. The `herald` flavor (per §18.10) flags suspicious agent patterns (sudden volume spike, off-hours bursts) as compliance events.

Quiet hours: ignored for `kind=agent` by default — agents are expected to send during off-hours. Operator MAY opt-in to per-token quiet hours via `agent_tokens.metadata.quiet_hours`.

Revocation: `<flavor>herald subscriber agent-token revoke <name>` zeroes the row's `revoked_at` and invalidates the token immediately (Redis cache TTL 30 s for token validity).

§8. Workable-item naming prefix

8.1 Static prefix HRD–

For all opened workable items for the Herald project (Issues, Issues_Summary, Fixed, Fixed_Summary, Status, Status_Summary, etc.) use the prefix HRD. Examples: HRD–001, HRD–002. Strict zero-padded sequence within each docset.

8.2 Derived 3-letter prefix algorithm

Per R-17 — when a consuming project does NOT define its own workable-item prefix, Herald derives a deterministic 3-letter prefix from the project name. Implementation lives in `commons/prefix`.

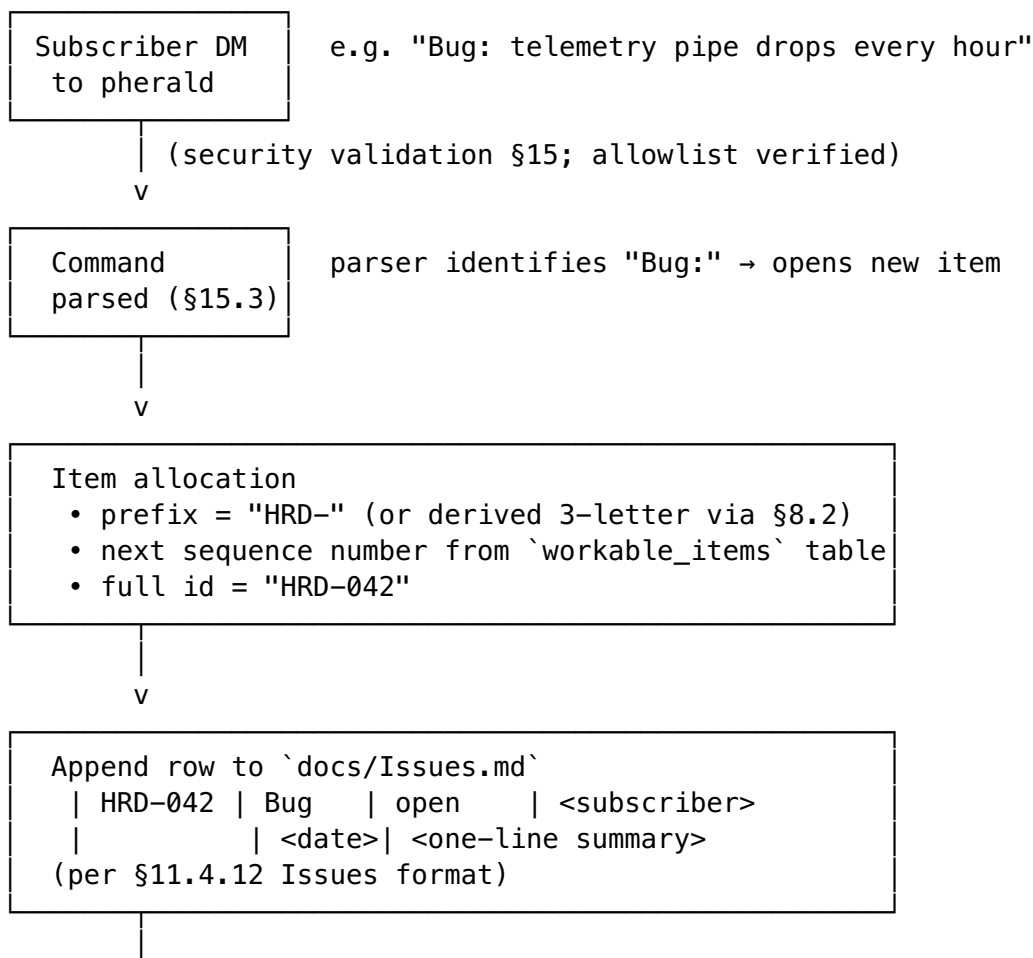
Algorithm (~80 LOC):

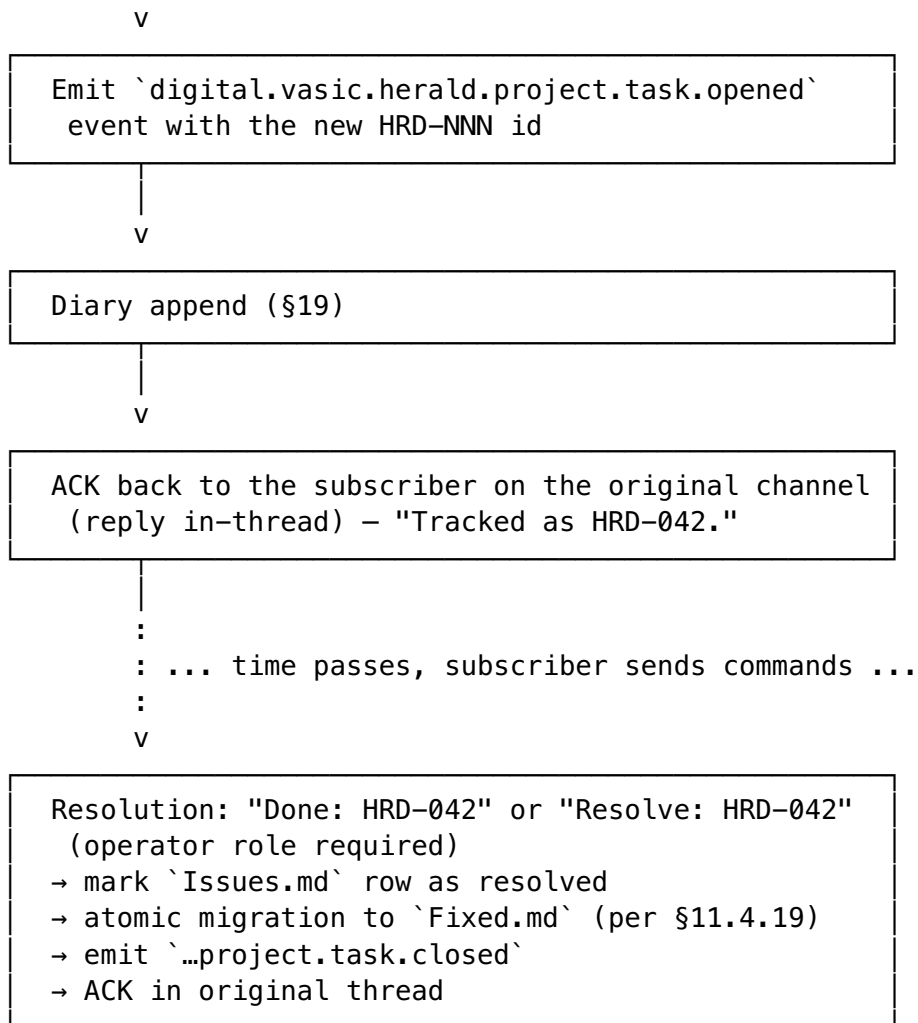
1. **Normalize:** Unicode NFKD → strip diacritics → retain [A-Za-z0-9] → split on CamelCase boundaries and [-_ /] into tokens.
2. **Rule A (≥3 tokens):** first letter of each of the first three tokens.
HeraldRouterCore → HRC.
3. **Rule B (2 tokens):** first letter of token 1; first letter of token 2; first internal consonant of token 2. HeraldRouter → HRT; HeraldRunner → HRN.
4. **Rule C (1 token):** first letter, first internal consonant, last consonant. Herald → HRD.
5. Uppercase.
6. **Collision resolution:** maintain committed `.herald/prefix.lock` (TOML name → prefix). On collision with a *different* project, compute `fnv1a32(name) mod 26` and replace the third letter with 'A' + (h mod 26); iterate up to 26 times, then fall back to numeric suffix HR0...HR9.
7. **Persistence:** the lock file is committed so the mapping is stable across machines and regenerations.

No mature Go library exists for 3-letter abbreviation generation (the only Go prior art Defacto2/releaser/initialism is a curated lookup table, not a generator); Herald ships its own.

8.3 Workable-item lifecycle (HRD-NNN flow)

A workable item moves through this lifecycle across pherald's subscriber-command handlers and the docs/ tracker files (per Universal §11.4.12 + §11.4.53):





workable_items schema (lightweight pointer table — the canonical record lives in the Markdown files for human edit-ability per Universal §6):

```

CREATE TABLE workable_items (
  tenant_id    UUID NOT NULL,
  item_id      TEXT NOT NULL,           -- "HRD-042"
  prefix       TEXT NOT NULL,          -- "HRD"
  sequence     INTEGER NOT NULL,       -- 42
  item_type    TEXT NOT NULL,          -- 'bug' |
  'issue' | 'query' | 'request' | 'question'
  status       TEXT NOT NULL,          -- 'open' |
  'in_progress' | 'blocked' | 'resolved' | 'wont_fix'
  opened_by    UUID,                   -- subscriber
  id
  opened_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  resolved_at  TIMESTAMPTZ,
  source_thread JSONB,                 --
  ConversationRef serialised
  PRIMARY KEY (tenant_id, item_id),
  UNIQUE (tenant_id, prefix, sequence)
);
CREATE INDEX wi_status_idx ON workable_items (tenant_id, status)
  WHERE status <> 'resolved';
ALTER TABLE workable_items ENABLE ROW LEVEL SECURITY;
  
```

```
ALTER TABLE workable_items FORCE ROW LEVEL SECURITY;
CREATE POLICY wi_isolation ON workable_items
    USING (tenant_id = current_setting('app.tenant_id')::uuid)
    WITH CHECK (tenant_id =
        current_setting('app.tenant_id')::uuid);
```

Sequence allocation: a per-tenant Postgres advisory lock around `MAX(sequence) + 1` inside the transaction; safe under concurrent Bug: commands. For high-volume tenants, a per-prefix sequence is preallocated in batches (default batch size 100) cached in Redis to avoid contention.

Reopening: Reopen: HRD-042 (operator role) reverses the migration: row moves back to `Issues.md`, `status='in_progress'`, `resolved_at` cleared, history preserved in `docs/Reopens/HRD-042.md` per Universal §11.4.55.

§9. Technology stack

9.1 Go (single-binary, multi-flavor)

Herald and all flavors MUST BE written in **Go**.

Toolchain pin: Go ≥ 1.22 is mandatory (stdlib log/slog, OTel slog bridge, slices + maps generics). The repo's go.mod files declare go 1.22 and set toolchain go1.22.x to the current patch release. Bumps to ≥ 1.23 are allowed when CI proves no regression; the commons module is the authoritative version (every flavor MUST follow).

License: Herald is published under the terms in LICENSE (parent project's chosen license). All go.mod files MUST declare the same license via a top-level comment, and every LICENSE file in vendored submodules MUST remain present. License compliance is checked by the planned cherald flavor (per §18.10) on every CI run when configured.

Layout (per R-18, research Topic 5):

```
Herald/
├─ go.work                # local dev only, .gitignored
├─ commons/               go.mod  # module github.com/vasic-digital/
herald/commons
├─ pherald/               go.mod  # module github.com/vasic-digital/
herald/pherald → cmd/pherald
├─ sherald/               go.mod  # module github.com/vasic-digital/
herald/sherald → cmd/sherald
├─ bherald/               go.mod  # ... etc
├─ .goreleaser.yaml       # one config, builds all flavors
└─ docs/, tests/, ...
```

Each flavor's go.mod declares `require github.com/vasic-digital/herald/commons vX.Y.Z`. **Lockstep release:** one git tag triggers GoReleaser to build all flavors and tag commons/vX.Y.Z. Use release-please in manifest mode to bump from Conventional Commits. go.work is git-ignored so CI catches forgotten bumps.

9.2 Postgres + Row-Level Security

Postgres is the main database, deployed via the containers submodule. Schema highlights:

- Every business table carries `tenant_id UUID NOT NULL`; composite indexes (`tenant_id, ...`).
- `FORCE ROW LEVEL SECURITY` enabled on every multi-tenant table.
- Runtime role: `herald_app` (no `BYPASSRLS`). Migration role: `herald_migrator` (`BYPASSRLS`).
- `SET LOCAL app.tenant_id = '<uuid>'` at the start of every transaction.
- `UUIDv7` (`uuidv7()` function) for time-ordered primary keys.

9.3 Redis (per-tenant ACL)

Redis is the in-memory store, deployed via containers submodule.

- **Per-tenant ACL user:** `redis-cli ACL SETUSER tenant_<id> on >pwd ~t:<id>:* +@read +@write ...`
- **Key prefixing:** `t:<tenant_id>:...` on every key.
- Usage: rate-limit token buckets, fast-path idempotency-cache, transient deduplication, hot subscriber lookups.

9.4 Container ports (24XXX)

Per R-01: all Container host ports start with **24XXX** prefix (1024–49151 IANA User Ports, below the Linux ephemeral floor at 32768, above all common service defaults — Postgres 5432, Redis 6379, web 3000/5000/8000/8080/9000).

Reserved sub-blocks:

Range	Purpose
24000–24099	flavor app data ports (one per flavor instance)
24090–24099	flavor admin ports (<code>/livez</code> , <code>/readyz</code> , <code>/metrics</code> , <code>pprof</code>)
24100–24199	Postgres instances (default 24100)
24200–24299	Redis instances (default 24200)
24300–24399	NATS JetStream (when enabled)
24400–24499	OpenTelemetry Collector (default OTLP gRPC 24417, HTTP 24418)
24500–24599	Prometheus / Grafana (when self-hosted alongside)
24600–24699	Temporal (when enabled)
	reserved for future / per-tenant

Range	Purpose
24700–24999	

9.5 containers submodule

The full Docker/Podman Compose stack is provided by [vasic-digital/containers](#) as a Git submodule (per R-12, the owned-submodule set in `HERALD_CONSTITUTION.md` will be updated in the PR that introduces the submodule). All container names MUST start with prefix `herald`.

9.6 Database migration tooling

Herald uses `golang-migrate/migrate` (the de-facto standard Go migration tool) embedded as a library inside `commons_storage`. Rationale: `golang-migrate` is binary-distributable AND embeddable, supports up/down, file checksums (drift detection), `schema_migrations` table version locking, and ships idempotent migrations.

File layout (`commons_storage/migrations/`):

```
commons_storage/migrations/
├── 000001_init_core.up.sql          # tenants, roles, encryption
keys
├── 000001_init_core.down.sql
├── 000002_idempotency_keys.up.sql
├── 000002_idempotency_keys.down.sql
├── 000003_subscribers.up.sql        # subscribers +
subscriber_aliases (§7.1)
├── 000003_subscribers.down.sql
├── 000004_channel_addresses.up.sql  # §6
├── 000004_channel_addresses.down.sql
├── 000005_webhook_sources.up.sql    # §5.5
├── 000005_webhook_sources.down.sql
├── 000006_thread_refs.up.sql        # §12
├── 000006_thread_refs.down.sql
├── 000007_quarantined_messages.up.sql # §15.2
├── 000007_quarantined_messages.down.sql
├── 000008_dead_letters.up.sql       # §5.4
├── 000008_dead_letters.down.sql
├── 000009_email_suppressions.up.sql # §11.9
├── 000009_email_suppressions.down.sql
├── 000010_river_jobs.up.sql          # River queue schema
(managed by river/cmd/river)
├── 000010_river_jobs.down.sql
├── 000011_rls_policies.up.sql        # FORCE RLS on every
multi-tenant table
```

Numbering: zero-padded 6-digit sequence (`000001..000999` reserved for V2 baseline; `001000..` reserved for V3+). Down migrations MUST exist for every up migration so `<flavor>herald migrate down -n 1` is always safe.

Runtime contract:

- `<flavor>herald migrate up [--steps N]` — apply pending migrations forward.
- `<flavor>herald migrate down --steps N` — rollback N migrations (gated behind interactive confirmation unless `--yes`).
- `<flavor>herald migrate status` — show current version + pending count.
- `<flavor>herald migrate force <version>` — recovery only; sets the version without running migrations (operator **MUST** verify manually).
- `<flavor>herald migrate validate` — checksum every applied migration against the source files; fails if drift detected.

Roles (per §16):

- `herald_migrator` — `BYPASSRLS`, owns schema, runs DDL. Only used by `migrate` subcommand.
- `herald_app` — runtime role; cannot run DDL.

Forward compatibility (per §26.3): migrations **MUST** be backward-compatible for **two minor versions** so rolling restarts during a deploy don't break the running fleet. Practical patterns: add nullable columns first, backfill, then add `NOT NULL`; never drop a column the previous binary version still reads; never rename a column — add the new one, dual-write, then drop the old in a later release.

§10. Commons (architecture layers)

Per the layering principle in V1: commons -> commons level 1 -> ... -> commons level N -> Flavor. V2 names the layers concretely:

Layer	Module	Owens
L0	commons	CloudEvents envelope, Channel interface, Branding, error types, time/uuid helpers
L1	commons_messaging	Watermill router, retry/throttle/dedup middlewares, channel adapters (Telegram, Slack, ...), templating, i18n
L1	commons_storage	Postgres connection + RLS middleware, River queue setup, Redis client + tenant ACL, migrations
L1	commons_security	HMAC verifiers (Slack/GitHub/Telegram/generic), DKIM/SPF/DMARC helpers, allowlist evaluator, command parser
L1	commons_observability	OTel setup, span helpers, metrics registration, log handler wiring (<code>slog</code> + <code>otellog</code>)
L1	commons_prefix	3-letter prefix algorithm (§8.2)
L1	commons_diary	append + export + sync (§19)
L2	commons_workflows	Temporal workflow scaffolding (escalation chains, digest builders) — opt-in
Flavor	pherald, sherald, ...	

Layer	Module	Owns
		Per-flavor commands, event-type handlers, flavor-specific subscriber commands

Rule of thumb (carry from V1): put new shared code in the **lowest layer that still makes sense**; flavors inherit upward.

§11. Channels — per-channel capabilities matrix

11.0 Channel adapter contract (Go interface + value types)

Each channel adapter implements the `Channel` interface defined in `commons` (L0).
The full contract:

```
package commons // L0

import "context"

// Channel is the interface every channel adapter implements.
type Channel interface {
    Name()
        string //
        "tgram", "slack", ...
    Capabilities()
        Capabilities //
        declarative feature flags
    Send(ctx context.Context, msg OutboundMessage) (Receipt,
        error)
    Subscribe(ctx context.Context, h InboundHandler)
        error // long-running; called by `serve`
    HealthCheck(ctx context.Context) error
}

// Capabilities advertises what an adapter supports at runtime.
// Routers consult Capabilities before dispatching; mismatches are
// logged
// and the message is dropped or downgraded (per the adapter's
// choice).
type Capabilities struct {
    Text bool
    Markdown bool // adapter's native markdown flavor
        (Slack mrkdwn, Telegram MarkdownV2, etc.)
    HTML bool
    Attachments bool
    AttachmentMaxMiB int
    Threads bool // first-class reply threading
    InteractiveURL bool // URL buttons / link actions
    InteractiveCall bool // callback handlers (advanced tier)
    DeliveryCeiling DeliveryEvidence // see §17 / R-05
}
```

```

// DeliveryEvidence enumerates the strongest reachable signal per
// channel.
type DeliveryEvidence int

const (
    DeliveryUnknown DeliveryEvidence = iota
    DeliveryAccepted // hop-by-hop ack (SMTP
    250)
    DeliveryRouted // platform stored &
    broadcast (Telegram/Slack API ok)
    DeliveryDelivered // recipient transport
    confirmed (Email DSN, WA delivered receipt)
    DeliveryRead // recipient read (Email
    MDN, Telegram Business read marker)
)

// OutboundMessage is the value passed to Channel.Send.
type OutboundMessage struct {
    EventID string // CloudEvents id (§4)
    IdempotencyKey string // explicit; falls back to
    EventID
    TenantID string
    To []Recipient // resolved from subscriber preferences
    + tag fan-out
    Subject string // optional (Email, RSS-
    like channels)
    Body Body // rendered per-channel
    template output
    Attachments []Attachment
    Thread *ConversationRef // optional; per §12
    Priority Priority // ntfy-compatible 1..5
    Actions []Action // optional interactive
    buttons
    Branding Branding // per-flavor (§6.3)
    Trace TraceContext // OTel propagation
}

// Body carries one or more rendered representations; adapter
// picks the best
// match for its Capabilities (e.g., HTML for email, Markdown for
// Telegram,
// Block-Kit JSON for Slack).
type Body struct {
    Plain string
    Markdown string
    HTML string
    Native map[string]any // adapter-specific (Slack
    blocks, Adaptive Card JSON, ...)
}

// Attachment is a single attached file.
type Attachment struct {
    Filename string
    MIMETYPE string

```



```

    SizeBytes int64
    Reader     func() (io.ReadCloser, error) // lazy open so the
        adapter streams without buffering
    CID        string                       // optional inline-
        image Content-ID (Email)
}

// Recipient is a resolved (channel, channel_user_id) pair.
type Recipient struct {
    Channel        string // "tgram", "slack", "mailto", ...
    ChannelUserID  string // chat_id, U0xxx, email address, ...
    DisplayName    string // best-effort, for templating
}

// Action is an interactive UI hint (rendered as URL button,
// callback button,
// Adaptive Card action, ntify X-Action, etc. depending on
// Capabilities).
type Action struct {
    Type
    ActionType      // ActionView | ActionURL | ActionCallback |
        ActionHTTP | ActionCopy
    Label  string
    URL    string // for ActionView / ActionURL / ActionHTTP
    Method string // "GET" | "POST" (ActionHTTP)
    Body   []byte // ActionHTTP
    Data   string // ActionCallback payload (provider-
        defined, e.g. Telegram callback_data)
}

type ActionType int

const (
    ActionView      ActionType = iota // open a URL
    ActionURL       // synonym for ActionView;
        provider-specific styling
    ActionCallback  // round-trip back into
        Herald via inbound handler
    ActionHTTP      // fire-and-forget HTTP
        request from the recipient device (ntfy)
    ActionCopy      // copy text to clipboard
        (ntfy)
)

// Priority maps to ntify 1..5 and to per-channel native
// priorities.
type Priority int

const (
    PriorityLow      Priority = 1
    PriorityNormal   Priority = 3
    PriorityHigh     Priority = 4
    PriorityUrgent   Priority = 5
)

```

```

// Receipt is what Channel.Send returns on success – the adapter's
// evidence
// of acceptance/routing/delivery so the router can decide whether
// to retry.
type Receipt struct {
    Evidence      DeliveryEvidence
    ChannelMsgID  string           // Slack ts; Telegram
                  message_id; SMTP queue-id; ...
    SentAt       time.Time
    LatencyMillis int64
    Native        map[string]any // adapter-specific raw
                  response (for diary)
}

// InboundHandler receives messages emitted by the adapter's
// Subscribe loop.
// Implementations enqueue events back into the router (§5).
type InboundHandler interface {
    Handle(ctx context.Context, ev InboundEvent) error
}

// InboundEvent is a CloudEvent constructed from a subscriber
// message.
type InboundEvent struct {
    EventID      string           // UUIDv7
    CloudEvent   CloudEventEnvelope // §4.1
    Sender       Recipient         // who sent it
    Subscriber   *Subscriber        // resolved via
                  subscriber_aliases, nil if unknown
    Body         Body
    Attachments  []Attachment
    Thread       *ConversationRef
    Raw          map[string]any // adapter-specific raw payload
                  (for diary)
}

// Subscriber is the in-memory projection of one row from
// `subscribers`
// (§7.1) plus all linked `subscriber_aliases` for the resolved
// channel.
// Pointer fields are nullable; consumers MUST nil-check before
// use.
type Subscriber struct {
    ID            uuid.UUID
    TenantID      uuid.UUID
    Handle        string           // empty if not operator-
                  mapped
    DisplayName   string
    Locale        string           // BCP-47, e.g. "en-US", "sr-
                  Latn-RS"
    Timezone      string           // IANA, e.g. "Europe/
                  Belgrade"

```

```

    Roles          []string          // e.g.
                        ["operator","ic","reader"]
    Metadata        map[string]any    // free-form per-subscriber
                        data
    Aliases          []SubscriberAlias // all channels this human is
                        reachable on
    Preferences      *PreferenceSet    // see §7.2; nil ⇒ tenant
                        defaults apply
}

// SubscriberAlias is one row from `subscriber_aliases`.
type SubscriberAlias struct {
    Channel          string          // "tgram", "slack",
                        "mailto", ...
    ChannelUserID    string          // chat_id, U0xxx, email
                        address, ...
    VerifiedAt       *time.Time      // nil ⇒ operator-mapped (not self-
                        verified)
    LastSeenAt       *time.Time
}

// CloudEventEnvelope is Herald's typed projection of a
// CloudEvents v1.0
// payload. SDK type used at the boundaries is cloudevents.Event
// from
// github.com/cloudevents/sdk-go/v2 – this struct is the in-
// process
// canonical form after parsing/validation.
type CloudEventEnvelope struct {
    SpecVersion      string          // "1.0"
    ID               string          // UUIDv7 (natural
                        ordering)
    Source           string          // URI
    Type             string          // reverse-DNS, e.g.
                        "digital.vasic.herald.ci.failed"
    Time            time.Time      // RFC 3339
    Subject          string          // tag:/channel:/
                        empty
    DataContentType  string          // e.g. "application/
                        json"
    Data             []byte         // opaque payload
    Extensions       map[string]string // heralddenant,
                        heralddidempotencykey, heraldpriority, ...
}

// TraceContext carries OpenTelemetry trace propagation across
// Herald
// boundaries (HTTP ingress → router → River job → channel
// adapter).
// Stored alongside messages so spans link correctly even when a
// job
// runs asynchronously minutes after the originating request
// returned.
type TraceContext struct {

```

```

TraceID    string // 32-hex chars (W3C Trace Context)
SpanID     string // 16-hex chars (the parent span at
                handoff)
TraceFlags byte    // sampling flags (W3C)
TraceState string // vendor-specific (W3C tracestate header)
Baggage    string // W3C baggage header value
}

// Branding is the per-flavor visual identity (§6.3 reference).
// One Branding is constructed per flavor binary at startup and
// threaded
// through OutboundMessage so adapters render channel-specific
// bling.
type Branding struct {
    AppName      string // "Project Herald", "System
                Herald", ...
    BinaryName   string // "pherald", "sherald", ...
    IconURL      string // for rich embeds (Slack auth user,
                Discord author, ...)
    AccentColorHex
                string // "#2C7BE5" – used as Slack attachment color,
                Discord embed color, Adaptive Card accent
    DefaultFooter
                string // "Sent by pherald 1.0 · github.com/
                vasic-digital/Herald"
}

// ChannelID is the canonical channel identifier. It MUST match
// the
// scheme used in `channel_addresses.address_url` and in the URL
// scheme registered by the adapter (e.g. "tgram", "slack",
// "mailto").
type ChannelID string

const (
    ChannelTelegram ChannelID = "tgram"
    ChannelMax       ChannelID = "max"
    ChannelSlack     ChannelID = "slack"
    ChannelDiscord   ChannelID = "discord"
    ChannelTeams     ChannelID = "teams"
    ChannelLark      ChannelID = "lark"
    ChannelWhatsApp ChannelID = "whatsapp"
    ChannelViber     ChannelID = "viber"
    ChannelEmail     ChannelID = "mailto"
    ChannelNtfy      ChannelID = "ntfy"
    ChannelGotify     ChannelID = "gotify"
    ChannelWebhook   ChannelID = "webhook"
    ChannelDiary     ChannelID = "diary"
    ChannelNull      ChannelID = "null" // §11.14 sandbox/no-op
                adapter for tests
)

// PreferenceSet is a typed view of the per-subscriber preferences
// JSON

```

```

// stored in `subscribers.metadata.preferences` (§7.2). See §7.2
// for the
// JSON shape; this is the Go decoded form.
type PreferenceSet struct {
    Categories map[string]CategoryPref // category_id → pref
    Workflows  map[string]WorkflowPref // CloudEvents type →
    pref
    QuietHours *QuietHours // nil ⇒ no quiet hours
    configured
    ChannelData map[ChannelID]any // provider-specific
    routing data
}

type CategoryPref struct {
    Channels []ChannelID
    Muted    bool
}

type WorkflowPref struct {
    Channels []ChannelID // may be empty (= use Category default)
    Muted    bool
}

type QuietHours struct {
    TZ          string // IANA TZ
    Start       string // "HH:MM" 24h
    End         string // "HH:MM" 24h
    ExemptCategories []string // categories that override quiet
    hours
}

```

These types live in `commons/types.go` and `commons/preferences.go`. Every adapter under `commons_messaging/channels/<name>` consumes them; no adapter is allowed to invent its own equivalent (the contract is the contract). Additional helpers in `commons/cloudevents.go` (`CloudEventEnvelope` $\rightleftharpoons$ `cloudevents.Event`), `commons/branding.go` (per-flavor Branding factory), `commons/trace.go` (TraceContext propagation), `commons/uuidv7.go` (UUIDv7 generator).

11.1 Telegram

- **SDK:** `gopkg.in/telebot.v3` (tucnak/telebot) primary; `github.com/mymmrac/telego` alt for 1:1 Bot API fidelity.
- **V1 (MVP) features:**
 - `sendMessage` with `MarkdownV2` parse mode.
 - `sendPhoto` / `sendDocument` for attachments.
 - `InlineKeyboardMarkup` with URL buttons + `callback_data`.
- **V2 advanced:**
 - `callback_query` handler (interactive replies).
 - `editMessageText` (in-place updates for ongoing incidents).
 - `web_app` buttons (open Mini-Apps).
 - `sendMediaGroup` (galleries).
 - Forum-topic threads via `message_thread_id`.
- **Out-of-scope (V2):** full Mini-App SDK, payments, Telegram Passport.

- **Delivery evidence ceiling:** Routed via Bot API; Read only via Business Bot connection with `can_read_messages` right (separate setup).
- **Webhook secret:** `setWebhook?secret_token=<token>`; verify `X-Telegram-Bot-API-Secret-Token` header on inbound.

11.2 Max (max.ru)

- **SDK:** `github.com/max-messenger/max-bot-api-client-go` (official-adjacent, Apache-2.0, English+Russian docs).
- **Gating:** behind build tag `herald_max` and documented sanctions advisory in the operator manual. VK Group parent VK Company Limited is not on the OFAC SDN list at time of writing, but several VK-affiliated individuals are designated and EU restrictive measures evolve frequently — operators must check at deploy time.
- **Bot registration:** via in-app MasterBot.
- **V1:** send text message, send media; basic inline keyboard.
- **V2 advanced:** per `dev.max.ru` docs as they expand.
- **Delivery evidence ceiling:** Routed (API ack = stored & queued).

11.3 Slack

- **SDK:** `github.com/slack-go/slack` (pin $\geq$ v0.23.1 — GHSA-gxhx-2686-5h9g fixed empty-secret bypass).
- **V1:**
 - `chat.postMessage` with Block Kit header + section + divider + image + actions (URL buttons only).
 - Threading via `thread_ts` (R-08 mapping).
- **V2 advanced:**
 - `block_actions` callback handler (interactivity).
 - `views.open` modals.
 - Message shortcuts.
 - Socket Mode for daemon ingress.
- **Out-of-scope:** Workflow Builder steps, Slack Connect, Enterprise Grid management.
- **Delivery evidence ceiling:** Routed (`ok:true` + `ts` proves posted to channel; no per-user read event).
- **Signing:** HMAC-SHA256 verification using `slack.NewSecretsVerifier` over `v0:{X-Slack-Request-Timestamp}:{rawBody}`, comparing with `X-Slack-Signature`, 5-min replay window.

11.4 Discord

- **SDK:** `github.com/bwmarrin/discordgo` (~5.7k stars, BSD-3-Clause, stable).
- **V1:**
 - Incoming Webhook with embeds (title, description, fields, footer, thumbnail, color).
 - Threads via `webhook + ?thread_id=`.
- **V2 advanced:**
 - Bot token; components (buttons + select menus).
 - Slash commands.
 - Forum channels.
- **Out-of-scope:** modals, voice, stage channels.

- **Delivery evidence ceiling:** Routed.

11.5 Microsoft Teams

- **SDK:** github.com/atc0005/go-teams-notify/v2 (incoming-webhook-only — **no official Microsoft Go SDK** for full Graph access).
- **V1:**
 - Incoming Webhook with Adaptive Card v1.5: TextBlock, Image, Container, FactSet, ActionSet with Action.OpenUrl.
- **V2 advanced:**
 - Action.Execute / Actionable Messages with HMAC-signed tokens.
 - Bot Framework via Azure Bot Service (separate setup, complex).
- **Out-of-scope:** full Bot Framework conversational state, meeting extensions.
- **Delivery evidence ceiling:** Routed.

11.6 Lark / Feishu

- **SDK:** github.com/larksuite/oapi-sdk-go (**official**, MIT, code-gen flavor — verbose but accurate).
- **V1:** send text + rich message + image; basic interactive cards.
- **V2 advanced:** Mini-program cards, chatbot @-mentions, group management.
- **Delivery evidence ceiling:** Routed.

11.7 WhatsApp

Two transports for two use cases:

- **WhatsApp Web multi-device** — go.mau.fi/whatsmeow (tulir/whatsmeow, ~5.5k stars, MPL-2.0). Reverse-engineered WA Web; **not** for official Business API.
- **WhatsApp Business Cloud API** — github.com/twilio/twilio-go (official, multi-product; WA via Twilio Senders).
- **V1:** text + media via either transport; template-message support on Business Cloud API.
- **V2 advanced:** button-template messages, list messages, conversation pricing windows.

11.8 Viber

- **No official Go SDK.** Community: [mileusna/viber](https://github.com/mileusna/viber), [strongo/bots-api-viber](https://github.com/strongo/bots-api-viber).
- **V1:** hand-rolled REST client against the documented Viber REST API. Send text + media + carousel.
- **V2 advanced:** rich-media keyboards.

11.9 Email (deep)

The most operationally complex channel — Email gets the deepest treatment.

Two interchangeable transports behind one Channel interface:

1. **Raw SMTP + DKIM** — net/smtp (stdlib) + github.com/emersion/go-msgauth/dkim. Suitable for self-hosted operators.
2. **ESP HTTP API** — Resend (preferred), Postmark (fallback), SendGrid (alternative). Better deliverability + built-in bounce/complaint webhooks + suppression lists.

Mandatory features for every outbound email (V1):

- **DKIM signing** (RFC 6376) via go-msgauth/dkim.
- **List-Unsubscribe** (RFC 8058) — List-Unsubscribe: <https://...>, <mailto:...> + List-Unsubscribe-Post: List-Unsubscribe=One-Click. DKIM-signed.
- **Plain-text alternative** generated from the HTML body (MJML-rendered).
- **Inline images** via Content-ID: MIME parts (cid: references in HTML).
- **Suppression list lookup** — consult email_suppressions table before send; skip suppressed addresses and log to diary.

Bounce pipeline:

- Inbound mailbox parsed for RFC 3464 DSN parts, OR ESP webhook consumed for bounce/complaint/unsub events.
- Hard bounces, complaints, and manual unsubscribes write to email_suppressions:

```
CREATE TABLE email_suppressions (  
    tenant_id    UUID NOT NULL,  
    address      TEXT NOT NULL,           -- normalized lowercase  
    reason       TEXT NOT NULL,           -- 'hard_bounce' |  
                'complaint' | 'unsubscribe'  
    source_event UUID,  
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),  
    PRIMARY KEY (tenant_id, address)  
);  
ALTER TABLE email_suppressions ENABLE ROW LEVEL SECURITY;  
ALTER TABLE email_suppressions FORCE ROW LEVEL SECURITY;
```

Doctor command: <flavor>herald doctor email verifies SPF, DKIM, DMARC records for the configured sending domain at startup. Without correct DNS, Gmail/Yahoo's 2024 sender requirements quietly bin the mail.

Templating: MJML (<mj-section>, <mj-column>, <mj-image>, <mj-button>) compiled to inline-CSS HTML at **template-author time** (vendored MJML CLI; check in the compiled HTML alongside the .mjml source). The compiled HTML then runs through html/template for variable substitution at send time. No Node.js in the runtime container.

Delivery evidence ceiling: Accepted by default (relay 250 OK), upgradable to Delivered by parsing DSN, Read only when recipient voluntarily returns RFC 8098 MDN (rare).

11.10 ntfy / Gotify

Per research Topic 3 (notification-platforms): adopt ntfy's wire model as both an **inbound ingest format** (alongside CloudEvents) and as a first-class outbound channel.

- Map: X-Priority (1–5) → Herald priority enum; X-Tags → Herald tags; X-Click → action URL; X-Actions (view/http/broadcast/copy) → Herald Action schema; X-Attach/PUT body → attachment.
- **Gotify** application/client token split: application token authenticates publishers; client token authenticates subscribers; both revocable independently.
- **V1**: send to ntfy/gotify topic with priority + tags + attachment.
- **V2 advanced**: receive via WebSocket / SSE for live updates.

11.11 Generic outbound webhook

Adapter webhook: `//<url>?secret=<hmac>&format=<cloudevent|json|form>`:

- Sends Herald events as CloudEvents (binary or structured mode), or as plain JSON, or as form-encoded — chosen by query param.
- HMAC-signs outbound requests (Webhook-Signature header) so receiving side can verify.
- Retries per §5.4.

11.12 Diary (Markdown + PDF + HTML)

See §19 for full schema and sync strategy.

11.13 Feature matrix summary

Channel	Text	Markdown	Attachments	Threads	Interactive buttons	Signed-source verify
Telegram	✓	MarkdownV2 / HTML	✓	forum topic_id	✓ V1 url, V2 callback	secret_tok
Max	✓	platform-specific	✓	V2	V2	tba
Slack	✓	Block Kit	✓	thread_ts	✓ V1 url, V2 callback	X-Slack-Signature
Discord	✓	embeds	✓	thread_id	webhook url, bot components	webhook URL secrecy
MS Teams	✓	Adaptive Card	inline	conv references	OpenUrl / Action.Execute	webhook secret
Lark	✓	rich card	✓	V2	interactive card	event subscription
WhatsApp	✓	text/template	✓ (Cloud API)	conversation window	Cloud API templates	Twilio signing / Meta token
Viber	✓	rich-media	✓	—		sig param

Channel	Text	Markdown	Attachments	Threads	Interactive buttons	Signed-source verify
					rich-media keyboard	
Email	✓	MJML→HTML	✓ (MIME)	In-Reply-To/References	url buttons only	DKIM/SP, DMARC
ntfy	✓	X-Markdown	✓	—	X-Actions	basic auth token
Gotify	✓	extras.client::display	✓	—	extras actions	bearer token
Webhook	✓	CloudEvent JSON	embedded	n/a	n/a	HMAC
Diary	✓	source format	path-refs	logical via parent ref	n/a	local FS
null://	✓	✓	✓ (counted only)	✓	✓ (recorded)	n/a

11.14 null:// sandbox channel (test-only)

The null:// adapter is the in-process equivalent of /dev/null with full instrumentation. It implements the entire Channel interface but performs no I/O — every Send call records the OutboundMessage to an in-memory ring buffer (configurable size; default 1000), increments per-tag counters, and returns the configured DeliveryEvidence ceiling.

Use cases:

- **Unit tests** for commons_messaging routing: assert that a given event with a given preference set produces the expected fan-out without touching any real channel.
- **Load tests**: route traffic through null:// to measure router/queue throughput without hitting upstream rate limits.
- **Quickstart / training**: operators can send test events to confirm routing logic before adding real channel credentials.
- **Chaos testing**: configure null:// to return error for X% of sends to exercise retry / dead-letter paths.

URL grammar:

```
null://[?
seed=<int>&fail_rate=<0..1>&latency_ms=<int>&ceiling=<Accepted|
Routed|Delivered|Read>&tags=<csv>]
```

Examples:

- null://?tags=test — happy path, instant, returns Routed.
- null://?fail_rate=0.1&tags=chaos — 10% of sends return transient error (exercises retry).
- null://?latency_ms=500&ceiling=Delivered&tags=load — adds 500 ms artificial latency, claims Delivered ceiling.

Inspector API (test-only HTTP endpoint, mounted when `[testing].null_inspector=true`):

Method	Path	Purpose
GET	/admin/null/messages	Recent ring-buffer contents (last N OutboundMessages).
GET	/admin/null/stats	Per-tag counters + send/fail tally.
POST	/admin/null/clear	Empty the ring buffer + reset stats.
POST	/admin/null/inject	Inject a synthetic inbound message (test the inbound path without a real subscriber).

`null://` MUST NOT be enabled in production deployments — the operator gate `CM-NUL-CHANNEL-DISABLED-IN-PROD` (planned) verifies that `null://` is absent from `channel_addresses` when `[herald].environment=production`. Operators that need null behaviour in prod (e.g., feature-flagged channels) use the `[channel_address].enabled=false` toggle instead.

Channel adapter test fixtures pattern. Every `commons_messaging/channels/<name>/` package SHOULD ship:

- `testdata/` — recorded HTTP request/response pairs (go-vcr-compatible cassettes).
- `<name>_test.go` — unit tests against `testdata/` (no network).
- `<name>_integration_test.go` (build tag `integration`) — runs against the channel's sandbox/test mode (Telegram test bot, Slack workspace `T0000` test team, etc.).
- `<name>_e2e_test.go` (build tag `e2e`) — only runs in nightly CI against real credentials in a dedicated test tenant.

§12. Messaging flows

Three mandatory message flows per channel (where the channel API supports them):

- **Simple message** — textual content sent to the channel; displayed to all subscribers.
- **Message with attachment(s)** — content + 1..N attachments; subscribers can download attachments.
- **Quote / reply message** — content sent as a reply to an existing channel message (uses §11 thread mapping: Telegram `reply_to_message_id`, Slack `thread_ts`, Discord `thread_id`, Email In-Reply-To); may include 0..N attachments.

Subscriber inbound flows (Project Herald and any flavor that opts into reply processing):

- **Direct reply to a Herald message** — parsed for commands (Bug:, Issue:, Query:, Request:, Question:).
- **Standalone message tagged for Herald** — same parsing, no parent message.

- **Thread continuation** — full thread context (chained replies from start) is parsed and processed.

Conversation reference per R-08:

```
type ConversationRef struct {
    Channel      ChannelID // "tgram", "slack", ...
    ThreadID     string      // Slack thread_ts; Telegram
                    message_thread_id (forum); Email References[0]
    ParentMessageID string    // Slack ts; Telegram
                    reply_to_message_id; Email In-Reply-To
    RootMessageID string      // first message in the thread
}
```

Persisted in the thread_refs table so re-sends to the same logical thread find the right native handle on every channel:

```
CREATE TABLE thread_refs (
    tenant_id          UUID NOT NULL,
    logical_thread_id  UUID NOT NULL,
                    -- Herald's stable identifier across channels
    channel            TEXT NOT NULL,
                    -- "tgram", "slack", "email", ...
    thread_id          TEXT,                                -- Slack
                    thread_ts, Telegram message_thread_id (forum), Email
                    References[0]
    parent_message_id  TEXT,
                    -- Slack ts (parent), Telegram reply_to_message_id, Email
                    In-Reply-To
    root_message_id    TEXT,                                -- first
                    message in the thread
    last_activity_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (tenant_id, logical_thread_id, channel)
);
CREATE INDEX thread_refs_recent_idx ON thread_refs (tenant_id,
    last_activity_at DESC);
ALTER TABLE thread_refs ENABLE ROW LEVEL SECURITY;
ALTER TABLE thread_refs FORCE ROW LEVEL SECURITY;
CREATE POLICY tr_isolation ON thread_refs
    USING (tenant_id = current_setting('app.tenant_id')::uuid)
    WITH CHECK (tenant_id =
        current_setting('app.tenant_id')::uuid);
```

A *logical thread* is Herald's tenant-stable conversation identifier; the per-channel rows map it to native handles. `last_activity_at` enables time-window garbage collection (default: 90 days after last activity, configurable).

§13. Templating & message composition

Three-layer stack per research Topic 5 (notification-platforms):

- **Layer 0 (engine)** — Go stdlib text/template (plaintext / Markdown / JSON channels) and html/template (HTML email body, auto-escaping).

- **Layer 1 (helpers)** — Sprig (date, upper, default, trunc, etc.) — universally expected by template authors; used by Helm, kubectl templates.
- **Layer 2 (email-specific)** — MJML compiled to HTML at template-author time (vendored MJML CLI; check in the compiled .html alongside .mjml), then html/template for runtime substitution.

Rejected: Liquid (extra runtime, no Go stdlib affinity); runtime MJML compilation (Node dependency in Go hot path).

Template directory layout:

```
commons_messaging/templates/
├── ci.failed/
│   ├── tgram.md.tpl
│   ├── slack.json.tpl      # Block Kit JSON
│   ├── email.mjml         # source
│   ├── email.html         # compiled, committed
│   └── diary.md.tpl
├── deploy.succeeded/
│   └── ...
├── _shared/                # template helpers
└── footer.tpl
```

§14. Localization (i18n)

Per research Topic 6 (notification-platforms): **nicksnyder/go-i18n v2**.

- One Bundle loaded at process start from commons_messaging/locales/*.toml:

```
# locales/active.en-US.toml
[ci.failed.title]
other = "Build failed: {{.JobName}}"

# locales/active.sr-Latn-RS.toml
[ci.failed.title]
other = "Build pao: {{.JobName}}"
```

- Plurals handled via CLDR rules (built-in to go-i18n):
1 alert / 2 alerta / 5 alerti / 21 alert for Serbian works correctly without custom code.
- Per-subscriber locale (BCP-47 tag) stored on the subscribers row.
- Per-channel templates may override (Telegram emoji-heavy variant vs sober email variant).

V2 initial locales: en-US, sr-Latn-RS, ru-RU. RTL rendering tweaks and dynamic locale negotiation from incoming events are deferred.

§15. Security model

Per R-16: a **three-layer pipeline** runs *before* any routing logic.

15.1 Transport-level (channel signature verification)

Per-channel signature verifier (constant-time HMAC comparison via `crypto/hmac.Equal`):

Channel	Header	Algorithm	Replay window
Slack	X-Slack-Signature	HMAC-SHA256 over <code>v0:{ts}:{body}</code>	5 min
GitHub-style	X-Hub-Signature-256	HMAC-SHA256 over body	configurable
Telegram	X-Telegram-Bot-API-Secret-Token	shared-secret compare	n/a (no timestamp)
Stripe	Stripe-Signature	HMAC-SHA256 over <code>{ts}.{body}</code>	5 min
Email	DKIM-Signature header + DMARC alignment	RSA-SHA256 / Ed25519	per DKIM TTL

Generic webhook sources register their own per-source secret in `webhook_sources`.

15.2 Sender-level (allowlist + verified subscribers)

After transport verification, sender authority is checked:

- Per-channel allowlist keyed by canonical user-id (Slack `team_id+user_id`, Telegram `chat_id`, email DKIM-aligned From) loaded from `.herald/subscribers.toml` and/or the `subscribers` + `subscriber_aliases` tables.
- Unknown senders enter a **quarantine queue** (`quarantined_messages` table), never the live fan-out.
- An operator (or auto-policy) reviews the quarantine and either promotes the sender or discards.

quarantined_messages schema:

```
CREATE TABLE quarantined_messages (  
  id          UUID PRIMARY KEY DEFAULT uuidv7(),  
  tenant_id   UUID NOT NULL,  
  received_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  channel     TEXT NOT NULL,  
  sender_channel_user_id TEXT NOT NULL,  
  sender_display TEXT,  
  reason      TEXT NOT NULL, --  
              'unknown_sender' | 'unverified_signature' | 'rate_limited'  
              | 'content_flagged'
```

```

payload_jsonb    JSONB NOT NULL,                                -- full
                  inbound message
triage_status    TEXT NOT NULL DEFAULT 'pending', -- 'pending'
                  | 'promoted' | 'discarded' | 'expired'
triaged_at       TIMESTAMPTZ,
triaged_by       UUID
                  -- subscriber id of operator
);
CREATE INDEX qm_pending_idx ON quarantined_messages (tenant_id,
received_at DESC)
WHERE triage_status = 'pending';
ALTER TABLE quarantined_messages ENABLE ROW LEVEL SECURITY;
ALTER TABLE quarantined_messages FORCE ROW LEVEL SECURITY;
CREATE POLICY qm_isolation ON quarantined_messages
USING (tenant_id = current_setting('app.tenant_id')::uuid)
WITH CHECK (tenant_id =
current_setting('app.tenant_id')::uuid);

```

Pending entries auto-expire after 30 days (triage_status='expired') so the queue doesn't grow unbounded on a noisy channel. CLI: <flavor>herald quarantine [list | promote <id> | discard <id> | purge].

15.3 Content-level (parsing & sanitization)

After sender verification, content is parsed:

- Commands (Bug:, Issue:, Query:, Request:, Question:, plus flavor-specific verbs) parsed with a strict regex-anchored tokenizer.
- **Never** shell out with user-provided content.
- **Never** text/template user input into shell or SQL.
- Command bodies are length-bounded (default 8 KiB).
- Attachments are scanned for MIME mismatch + size limits (default 25 MiB per attachment, 100 MiB per message).
- Optional malware scan via ClamAV when commons_security is configured with the integration.

15.4 Credential handling

Per Constitution §11.4.10:

- .env is git-ignored (the .gitignore MUST contain .env).
- .env.example is committed.
- Resolution order: CLI flag > shell env > .env > compiled default (§3.3).
- Credentials NEVER logged. The OTel logger has a redaction middleware that filters keys matching (?i)(token|secret|password|key|credential|authorization) to ***.
- Credentials in Postgres are encrypted-at-rest with pgcrypto (pgp_sym_encrypt) using a key from HERALD_DB_ENC_KEY env var.

15.5 Webhook ingestion (defense-in-depth)

See §5.5. Four ordered checks: signature → timestamp → delivery-ID dedup → optional IP allowlist.

§16. Multi-tenancy & isolation

Per research Topic 6 (event-fanout) + Topic 4 (operations):

- **Primary boundary:** PostgreSQL Row-Level Security.
- **Every business table** carries `tenant_id` `UUID NOT NULL` with a composite index (`tenant_id, ...`) leading every secondary index (RLS is 10–100× slower without it).
- **Policy template:**

```
ALTER TABLE <table> ENABLE ROW LEVEL SECURITY;
ALTER TABLE <table> FORCE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON <table>
  USING (tenant_id = current_setting('app.tenant_id')::uuid)
  WITH CHECK (tenant_id =
    current_setting('app.tenant_id')::uuid);
```

- **Roles:**
 - `herald_app` — runtime role, no `BYPASSRLS`. The `pgx` connection wrapper in `commons_storage` runs `SET LOCAL app.tenant_id = ...` at transaction start.
 - `herald_migrator` — schema-change role, `BYPASSRLS`.
- **Redis** — per-tenant ACL user (Redis 6+), key pattern `~t:<tenant_id>:*`.
- **Rate limits** — token bucket per (`tenant_id, channel`) in Redis.
- **Schema-per-tenant** is reserved for high-isolation enterprise customers; documented as opt-in but not the default (overhead: connection-pool fragmentation, slow migrations $N \times M$, harder backups).

16.1 Data retention & privacy

Herald processes content that may contain personally identifiable information (names, email addresses, IP addresses, organization data). Retention windows **MUST** be configurable per tenant and enforced by scheduled River jobs.

Default retention policy (overridable per tenant):

Class	Default retention	Purge mechanism
<code>idempotency_keys</code>	7 d after <code>expires_at</code> (already in §4.3)	nightly River job
<code>dead_letters</code> (triaged)	90 d	nightly River job
<code>dead_letters</code> (untriaged)	365 d (legal/audit floor)	manual triage prompt
<code>quarantined_messages</code> (pending)	30 d → auto expired	nightly River job
<code>quarantined_messages</code> (triaged)	90 d	nightly River job
<code>thread_refs</code>		

Class	Default retention	Purge mechanism
	90 d after last_activity_at	nightly River job
email_suppressions	indefinite (compliance — hard bounces must persist)	manual remove via CLI
subscribers (deleted)	tombstone 30 d, then hard delete	GDPR right-to-erasure flow
Diary entries (docs/herald/ diary/main.md)	tenant-configurable; default unlimited (git history)	per-tenant rotation policy
OpenTelemetry data	controlled by Collector → backend (out of Herald scope)	n/a

GDPR / privacy mechanics:

- `<flavor>herald subscriber forget <id>` initiates right-to-erasure: tombstones the row, schedules hard-delete after 30 days, anonymises subscriber references in `dead_letters` / `quarantined_messages` / diary entries (replaces `display_name` with `<redacted>`, hashes `channel_user_id`).
- `<flavor>herald subscriber export <id>` (right-to-portability) emits a JSON bundle of every record referencing the subscriber.
- **Diary redaction** — when a subscriber is forgotten, the diary writer SHOULD overwrite their entries with a redaction note (in-place edit + re-export); operators that want immutable audit trails MUST opt-out per `[privacy].diary_redaction_on_forget = false`.

Data sovereignty: the containers submodule deploys Postgres + Redis to operator-chosen regions; Herald itself stores nothing outside that pair. Cross-region replication is the operator's choice (logical replication / streaming replication) and is documented in `docs/operations/replication.md`.

§17. Observability & SLOs

Per research Topic 1 (operations):

17.1 OpenTelemetry pipeline

- **All three signals** instrumented from day one:
 - **Traces** — `go.opentelemetry.io/otel/trace` spans across the full event flow (ingest → route → enqueue → deliver → ack).
 - **Metrics** — `go.opentelemetry.io/otel/metric` counters + histograms.
 - **Logs** — `stdlib slog` shipped as OTEL log records via `go.opentelemetry.io/contrib/bridges/otel slog`.
- **Default export:** OTLP/gRPC to an OpenTelemetry Collector sidecar/ DaemonSet on `localhost:4317` (no auth) — production deployments override via env vars.
- **Collector** fans out to Prometheus (scrape `/metrics`), Tempo/Jaeger (traces), Loki (logs).

- Instrumentation lives in `commons_observability` (L1) so every flavor inherits.

Standard OTEL SDK env vars Herald honours (per OpenTelemetry SDK spec — values are NOT re-invented):

Variable	Default	Purpose
OTEL_SDK_DISABLED	false	Hard kill switch — when true, instrumentation no-op.
OTEL_EXPORTER_OTLP_ENDPOINT	http://localhost:4317	Single endpoint for all signals (gRPC).
OTEL_EXPORTER_OTLP_TRACES_ENDPOINT	inherits	Per-signal override.
OTEL_EXPORTER_OTLP_METRICS_ENDPOINT	inherits	Per-signal override.
OTEL_EXPORTER_OTLP_LOGS_ENDPOINT	inherits	Per-signal override.
OTEL_EXPORTER_OTLP_PROTOCOL	grpc	grpc http/protobuf http/json.
OTEL_EXPORTER_OTLP_HEADERS	unset	Authn (e.g. Authorization=Bearer ...).
OTEL_EXPORTER_OTLP_INSECURE	false	Disable TLS for http endpoints.
OTEL_RESOURCE_ATTRIBUTES	(Herald sets <code>service.name</code> , <code>service.version</code> , <code>herald.flavor</code> , <code>herald.tenant_id</code>)	Operator adds <code>deployment.environment</code> , <code>service.namespace</code> , ...
OTEL_SERVICE_NAME	<flavor>herald	Convenience equivalent to <code>service.name</code> .
OTEL_TRACES_SAMPLER	parentbased_traceidratio	Sampler choice.
OTEL_TRACES_SAMPLER_ARG	1.0	Ratio when sampler supports it.
OTEL_METRIC_EXPORT_INTERVAL	60000 (ms)	Push cadence.
OTEL_BSP_SCHEDULE_DELAY	5000 (ms)	Batch-span-processor c...

Herald's own resource defaults:

```

service.name=<flavor>herald
service.version=<git-tag-or-commit-sha>
service.instance.id=<hostname>--<pid>
herald.flavor=<flavor>
herald.tenant_id=<tenant_uuid>           # one-of label, set per request
telemetry.sdk.name=opentelemetry
telemetry.sdk.language=go

```

OTel semantic conventions tracked: **messaging v1.30+** (the spec moved out of experimental in 2025).

17.2 Metrics catalogue

OTel semantic conventions for messaging where they fit, Herald-specific names where they don't:

Metric	Type	Labels
<code>messaging.client.sent.messages</code>	counter	<code>messaging.system="herald"</code> , <code>messaging.destination.name=<channel></code> , <code>herald.tenant_id</code> , <code>herald.flavor</code>
<code>messaging.client.operation.duration</code>	histogram	+ <code>messaging.operation.name="deliver"</code> , <code>messaging.operation.type="send"</code> , <code>channel</code> , <code>outcome={ok,retry,drop,dead_letter}</code> , <code>tenant_id</code>
<code>herald_delivery_attempts_total</code>	counter	<code>channel</code> , <code>outcome={ok,retry,drop,dead_letter}</code> , <code>tenant_id</code>
<code>herald_queue_depth</code>	gauge	<code>queue</code> , <code>tenant_id</code>
<code>herald_retry_backoff_seconds</code>	histogram	<code>channel</code> , <code>attempt_bucket</code>
<code>herald_template_render_duration_seconds</code>	histogram	<code>template_id</code>
<code>herald_subscriber_count</code>	gauge	<code>tenant_id</code> , <code>channel</code>
<code>herald_dead_letter_count</code>	gauge	<code>tenant_id</code> , <code>channel</code>

Cardinality budget: never `recipient_id` or message UUIDs as labels. Per-tenant labels are allowed; per-subscriber labels are NOT.

Canonical Grafana dashboard JSON shipped in `deploy/grafana/herald-overview.json`.

17.3 Span model

Canonical span names (and parent→child relationship):

```
herald.ingest          (root, per inbound CloudEvent)
├── herald.signature_verify
├── herald.idempotency_check
├── herald.route_match
├── herald.preference_filter
├── herald.template_render
├── herald.enqueue
└── herald.deliver
    ├── herald.channel.<name>
    │   └── herald.channel.<name>.api_call
    └── herald.diary_append
```

Attributes on every span: `herald.tenant_id`, `herald.flavor`, `cloudevents.type`, `cloudevents.id`.

17.4 SLOs

V2 published SLOs (per-tenant rolling 30-day):

SLO	Target
Ingress availability (HTTP 2xx + 4xx vs 5xx)	99.9%
End-to-end delivery (ingest → channel API Accepted) — p95	< 5 s
End-to-end delivery — p99	< 30 s
Delivery success rate (excluding 4xx from upstream channels)	99.5%
Dead-letter rate	< 0.1%
Doctor-CLI checks (SPF/DKIM/DMARC, DB ping, Redis ping)	run every 5 min, alert on 3 consecutive fail

17.4.1 Per-channel SLO budgets

The aggregate SLO table above can hide bad behaviour on one channel behind good behaviour on another. V2 also commits to **per-channel SLO budgets** — error budgets tracked independently so a single misbehaving adapter or upstream incident is visible immediately:

Channel	Delivery success target	p95 latency target (ingest → upstream accepted)	Notes
Telegram	99.5%	< 2 s	Telegram Bot API is famously reliable; alert if budget burn > 2× expected.
Slack	99.5%	< 3 s	<code>chat.postMessage</code> rate-limited per workspace; back off cleanly.
Discord	99.0%	< 3 s	Webhook rate limits are aggressive; expect retries.
MS Teams	98.0%	< 5 s	Incoming-webhook backend has visible jitter.
Lark	99.0%	< 4 s	
Max	99.0%	< 4 s	Subject to network reachability from non-RU operator regions.
Email (SMTP self-hosted)	95.0%	< 30 s	Includes DNS + greylist + tarpit latency.
Email (ESP — Resend/Postmark/SendGrid)	99.5%	< 5 s	ESP carries the deliverability risk; Herald sees Accepted quickly.
WhatsApp (Cloud API)	99.0%	< 3 s	Conversation-window restrictions can drop messages outside 24h windows.
Viber	98.0%	< 5 s	

Channel	Delivery success target	p95 latency target (ingest → upstream accepted)	Notes
ntfy / Gotify	99.5%	< 2 s	Hand-rolled REST client; expect rougher edges. Self-hosted = operator's own infra.
Webhook (generic outbound)	depends on operator's endpoint	< operator's deadline	Tracked but no Herald-side target.
Diary	99.99%	< 100 ms	Local file system + Pandoc batching; outliers indicate disk pressure.

Per-channel burn-rate alerts (multi-window, multi-burn-rate per Google SRE Workbook): page on **2-hour 14× burn** AND **6-hour 6× burn** simultaneously crossed; warn on either alone. Alerts route through the same Herald pipeline that any other event uses — Herald-monitoring-Herald MUST work, but operators are advised to also run a separate stand-by notifier for the case where Herald itself is the failure.

17.5 Health probes (livez / readyz / startupz)

Per research Topic 3 (operations): three endpoints on the **admin port** (24090 default, separate from data port so probes don't compete for worker capacity):

- **GET /livez** — returns 200 unless an unrecoverable internal invariant trips (deadlock detector, fatal goroutine panic flag). Cheap, no downstream calls.
- **GET /readyz** — checks Postgres + Redis ping, queue consumer attached, ≥1 channel adapter healthy. Returns 503 during graceful shutdown.
- **GET /startupz** — used by Kubernetes startup probe while migrations / warm-up run; once 200, liveness/readiness take over.

Liveness failureThreshold ~3× readiness so a transient downstream blip removes the pod from Service but does NOT restart it.

17.6 doctor CLI

<flavor>herald doctor runs a battery of environment checks:

- Postgres connectivity + RLS policy presence.
- Redis connectivity + ACL.
- All configured channel credentials valid (probes API per channel).
- DKIM/SPF/DMARC DNS records for the configured sending domain (cite Gmail/Yahoo 2024 sender requirements).
- OTLP collector reachable.
- Disk space, port availability.

Exits with non-zero status code on any failure + writes a structured report to stdout (machine-readable JSON option `--json`).

§18. Flavors (the implementations)

18.1 Common flavor contract

Every flavor binary MUST:

- Be named `<prefix>herald` and built from `<prefix>herald/cmd/` `<prefix>herald/main.go`.
- Implement the same Cobra-subcommand surface (send, serve, doctor, migrate, deadletter, subscriber, digest, version).
- Embed `commons` + `commons_messaging` + `commons_storage` + `commons_security` + `commons_observability` + `commons_diary` at the same pinned versions.
- Publish a flavor-specific event-type subtree (`digital.vasic.herald.<flavor>.*`).
- Register flavor-specific subscriber commands (the Bug: / Query: / etc. tokens that map to flavor-specific handlers).
- Provide a flavor-specific HeraldBranding (app name, icon, accent color).
- Ship its own user manual under `docs/flavors/<flavor>/`.

18.2 Project Herald (pherald)

Focus: Software-project development lifecycle. Integrates with VCS, code review, task tracking. Designed for AI-CLI-agent + developer-team workflows.

Event subtree: `digital.vasic.herald.project.*` and `digital.vasic.herald.reply.*`.

Sources:

- Git hooks (post-commit, post-merge, pre-push).
- VCS webhooks (GitHub, GitLab, GitFlic, GitVerse — push, pull_request, review, issue).
- AI CLI agents (Claude Code, OpenCode, Cursor, Aider) emitting structured progress events.
- Code-review-tool webhooks (CodeRabbit, Greptile, Sourcery).

Subscriber inbound commands (extends the V1 base set):

Command	Behaviour
Bug: / Issue:	Open an issue (writes to <code>docs/Issues.md</code> per Universal §11.4.12).
Query: / Request: / Question:	Request information; routes to LLM-driven research workflow if configured.
Status:	Reply with current project status from <code>docs/Status.md</code> .
Continue:	Pointer to <code>docs/CONTINUATION.md</code> for next agent.
Done:	Mark referenced item resolved (writes to <code>docs/Fixed.md</code>).
Spec:	

Command	Behaviour
	Cite or modify spec section (subject to §23 spec-change rule).
Run: <tool>	Authorized operator only — execute an approved tool (gated by allowlist).

Quote-thread processing: Subscriber's reply automatically includes the full thread context (chained replies from bottom to thread start) — full chain parsed and processed per R-08 ConversationRef.

Security validation (extends §15):

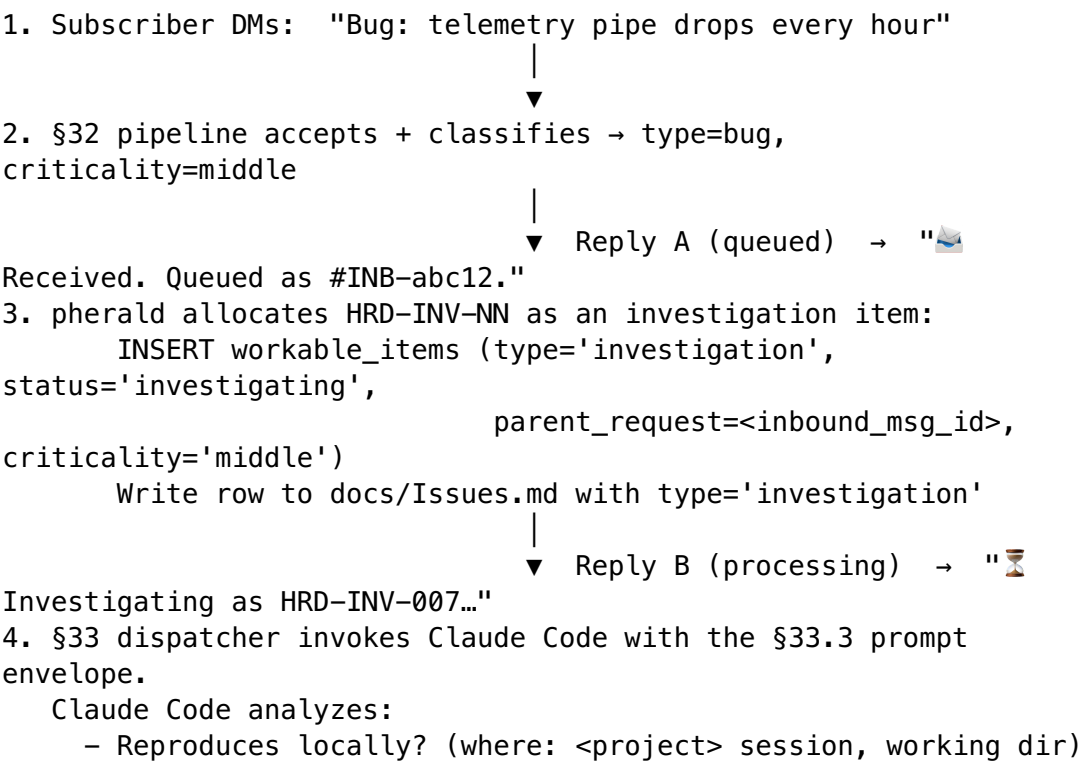
- Subscriber commands accepted only from verified, allowlisted subscribers.
- Run: requires elevated subscriber privilege (subscriber_aliases.verified_at plus roles array).
- All commands logged to diary with subscriber identity and timestamp.

Derived workable-item prefix (per §8.2): if the consuming project hasn't set one, the algorithm runs over the project name (from package.json, go.mod, pyproject.toml, or the git remote name).

18.2.1 Investigation-before-Fixing flow

When a subscriber reports a Bug: / Issue: / Implementation: request, pherald **MUST** first create an **Investigation workable item** (type investigation, status investigating) before committing to a final bug/task row. This guards against runaway LLM-generated workable items that turn out to be duplicates, out-of-scope, or non-reproducible.

The full sequence (composes §32 inbound pipeline + §33 LLM dispatch + §8.3 lifecycle):



- Identifies affected code paths.
 - Classifies root-cause area.
 - Estimates effort + dependencies.
- Returns <<<HERALD-REPLY>>> JSON.



5. pherald reads outcome:

- validated → allocate HRD-NNN final bug item, link parent_investigation=HRD-INV-007, close investigation as "validated", emit task.opened event, attach Claude's investigation summary as multi-format bundle (§36).
- needs_more_info → keep investigation open, ask subscriber follow-ups in Reply C.
- rejected (duplicate/out-of-scope/known) → close investigation with reason,

no final item,

Reply C explains.

- cannot reproduce → keep investigation open with "operator-blocked"
- status per Universal §11.4.21, page operator-on-call.



▼ Reply C (final)
 "✅ Created HRD-042 (bug, criticality=middle)."
 " Investigation: HRD-INV-007
 " Affected: pherald/internal/telemetry.go:142"
 " Repo: github.com/<org>/<project>/blob/<sha>/Issues.md"
 [+ multi-format attachment bundle per §36]

Persistence: every investigation has its own file under docs/Investigations/<HRD-INV-NNN>.md carrying the Claude-Code dispatch transcript + classification metadata + the validation decision. Composes with Universal §11.4.55 (Reopens-history) for the case where a previously-rejected investigation is reopened.

18.2.2 Criticality determination

Criticality is set in inbound_messages.classification by the §32.6 classifier and stored on the investigation + final workable items. Levels match Universal item-type vocabulary:

Level	Triggers	SLA replies	SLA investigation
critical	explicit critical: prefix, OR LLM keyword-confidence ≥ 0.85 on outage/data-loss/security-breach terms, OR sender has oncall role	Reply A $\leq$ 30 s, Reply B $\leq$ 2 min,	< 1 h

Level	Triggers	SLA replies	SLA investigation
	and message arrives during the configured incident window	Reply $C \leq 5$ min	
high	explicit high:, OR LLM 0.7–0.85	$A \leq 1$ min, $B \leq 5$ min, $C \leq 30$ min	< 4 h
middle	default for bug/issue/task	$A \leq 5$ s, $B \leq 5$ min, $C \leq 2$ h	< 24 h
low	explicit low: or nice-to-have:	$A \leq 5$ s, $B \leq 30$ min, $C \leq 24$ h	< 1 week

Operators MAY override mid-flight: Override: HRD-042
criticality=critical re-dispatches to LLM with the new SLA and re-pages
affected oncall tags.

18.2.3 Type classification (Universal §11.4.16 mapping)

Inbound type tokens map to Universal §11.4.16 item types one-to-one. The classifier emits the exact Universal vocabulary so downstream Issues.md rows pass §11.4.16-PARITY gate without translation. Full mapping in §32.6.

When an inbound is ambiguous (e.g., Hey, the build keeps failing — no prefix), the classifier:

1. Runs the deterministic keyword pass first (verbs like *fails*, *broken*, *crashed* → tentative bug).
2. If confidence < 0.7 , escalates to LLM for type-only dispatch (§33.3 prompt envelope, task=classify-only).
3. If LLM confidence still < 0.7 , replies **?** Could you tag this as Bug: / Issue: / Task: / Query: / ...? and stages the message as needs-classification (NOT processed further).

18.2.4 Attachment validation + storage

Subscriber-supplied attachments referenced by an inbound investigation/bug/task MUST follow this storage convention (mandatory, per the user's V3 requirement):

```

<consuming-project-root>/
├─ issues/
│   └─ users/
│       └─ attachments/
│           └─ HRD-042/                                ← WORKABLE_ITEM_ID
directory
    │   └─ 01_stack-trace.log                            ← original filename,
prefixed with arrival order
    │   └─ 02_screenshot.png
    └─ 03_repro-script.sh

```

Where HRD-042 is the canonical workable-item ID. For investigations the directory is named after the investigation ID (HRD-INV-007/); on transition to a final workable item, files are moved to the final ID's directory and HRD-INV-007/ becomes an empty placeholder kept for audit (Universal §11.4.55).

The investigation/bug Issues.md row carries a Attachments: column referencing the directory. Multi-format attachment generation (§36) does NOT apply to inbound user attachments — they are stored as-is in their original format. (§36 multi-format is for OUTBOUND attachments Herald itself produces.)

Pre-storage validation pipeline (per §32.4):

1. Each attachment downloaded into a sandbox directory (configurable; default /tmp/herald-staging/<random>/).
2. ClamAV scan (if configured) — quarantine on hit.
3. Magic-byte + MIME match — reject mismatched.
4. Extension allowlist — reject
.exe, .dll, .so, .dylib, .bat, .cmd, .ps1, .scr, .jar, .class by default.
5. Per-file size cap ([attachments].in_max_mib, default 25 MiB).
6. Per-message total cap ([attachments].in_total_max_mib, default 100 MiB).
7. If all pass → move to issues/users/attachments/<WORKABLE_ITEM_ID>/.
8. If any fail → record reason in quarantined_messages, do NOT move to canonical path, Reply C cites the validation error.

The attachments_index.md file inside each WORKABLE_ITEM_ID directory documents:

Attachments for HRD-042

Order	Filename	MIME	Size (B)	SHA-256	Uploaded by	Uploaded at	Notes
01	stack-trace.log	text/plain	12_348	<hex>	alice@tgram:42	2026-05-20T18:30:12Z	repro context
02	screenshot.png	image/png	348_551	<hex>	alice@tgram:42	2026-05-20T18:30:14Z	UI state
03	repro-script.sh	text/x-shellscript	1_204	<hex>	alice@tgram:42	2026-05-20T18:30:17Z	runs in 10 s

The index file IS the source of truth for "what's attached to this workable item" — Herald regenerates it on every attachment add/remove (per §11.4.61 freshness rules; if the index drifts from the directory contents, the §32 validator FAILs with attachment_index_drift).

18.2.5 Claude Code project-session integration

Per §33.2 the Claude Code session for the consuming project is anchored at <consuming-project>/.herald/claude-code/sessions/<project_name>.session. For Project Herald specifically:

- The [herald].project_name config value (e.g. ATMOSphere) drives the anchor name.
- Each invocation runs `claude --resume <UUID> --print "<§33.3 envelope>"` with `cwd=<consuming-project-root>`.

- The session anchor is read-only to Herald; manual deletion resets the session (operators MAY want this when Claude's context degrades — e.g. after a long-running incident produces a noisy session).
- The flavor binary refuses to start if `[herald].project_name` is empty (Claude Code dispatch is non-optional in V3 r1).

18.3 System Herald (**sherald**)

Focus: OS/host/service health and security. Designed for systems-administration teams + on-call rotation.

Event subtree: `digital.vasic.herald.system.*`.

Sources:

- `journalctl` watcher (configurable `_SYSTEMD_UNIT` filters).
- `auditd` events.
- Prometheus Alertmanager webhook receiver.
- Loki/Grafana log alerts.
- File-integrity-monitoring (AIDE, Tripwire) hooks.
- SNMP traps (via a small SNMP-to-CloudEvent adapter).
- Kernel events (oom-kill, panics) — via `journal` filter.

Event types:

- `digital.vasic.herald.system.host.cpu_high` (threshold-crossing).
- `digital.vasic.herald.system.host.disk_full` (filesystem).
- `digital.vasic.herald.system.host.memory_pressure`.
- `digital.vasic.herald.system.service.restarted`.
- `digital.vasic.herald.system.service.crashed`.
- `digital.vasic.herald.system.service.flapping` (≥ 3 restarts in 5min).
- `digital.vasic.herald.system.security.login_anomaly` (failed SSH burst, unusual IP).
- `digital.vasic.herald.system.security.privilege_escalation`.
- `digital.vasic.herald.system.cert.expiring` (X.509 certificate ≤ 30 d).
- `digital.vasic.herald.system.backup.missed`.

Subscriber commands:

- **Ack:** — acknowledge an alert (silences future fires of the same fingerprint for a window).
- **Silence:** `<fingerprint> for <duration>` — manual silencing.
- **Resolve:** `<fingerprint>` — manual resolution.
- **Status:** — current open-alerts snapshot.
- **Runbook:** `<fingerprint>` — link to the runbook for this alert type.

Integrations: `sherald` is often paired with `aherald` and `iherald` — the three flavors share the `commons_alert` package.

18.4 Build Herald (**bherald**)

Focus: CI/CD build lifecycle events. Designed for development teams + release engineers.

Event subtree: `digital.vasic.herald.ci.*`.

Sources:

- GitHub Actions workflow webhook (`workflow_run`, `check_suite`).
- GitLab CI webhook (Pipeline Hook).
- Jenkins post-build steps invoking `bherald send`.
- CircleCI / Drone / Buildkite via their respective webhooks.
- Tekton / Argo Workflows via CloudEvents emitters (native).

Event types:

- `digital.vasic.herald.ci.build.queued` | `started` | `succeeded` | `failed` | `cancelled`.
- `digital.vasic.herald.ci.test.passed` | `failed` | `flaky` | `skipped`.
- `digital.vasic.herald.ci.security_scan.finding` (Semgrep, Snyk, Trivy, gitleaks).
- `digital.vasic.herald.ci.dependency.outdated` | `vulnerable`.
- `digital.vasic.herald.ci.coverage.dropped`.
- `digital.vasic.herald.ci.lint.failed`.

Routing logic:

- Failed builds with @-mentions of the author (via `subscriber_aliases` mapping `git-author-email` → `subscriber`).
- Flaky tests batched into a daily digest (per §7.3 throttling).
- Security findings of CRITICAL severity bypass quiet hours.

Subscriber commands:

- **Retry:** — re-trigger the failed build (if integration grants permission).
- **Snooze:** `<duration>` — silence a flaky test.
- **Triage:** `<finding>` — mark a security finding for review.

18.5 Deploy Herald (`dherald`)

Focus: Release / deployment / rollout events.

Event subtree: `digital.vasic.herald.deploy.*`.

Sources:

- ArgoCD / Flux webhook (`application synced`, `health degraded`).
- Spinnaker pipeline notifications.
- Kubernetes Event watcher (Deployment rolled out, ReplicaSet failed).
- Custom deploy scripts invoking `dherald send`.

Event types:

- `digital.vasic.herald.deploy.started` | `succeeded` | `failed` | `rolled_back`.
- `digital.vasic.herald.deploy.canary.promoted` | `canary.aborted`.
- `digital.vasic.herald.deploy.feature_flag.toggled`.
- `digital.vasic.herald.deploy.config.drift_detected`.

Routing:

- Production deploys notify #prod-deploys + an Email to release-eng.
- Failed deploys with rollback option button (Slack Block Kit Action).

Subscriber commands:

- Rollback: <deploy_id> — initiate rollback (gated by elevated privilege).
- Hold: <env> — freeze further deploys to env.
- Status: <env> — current state of env.

Composes with rherald for the full release pipeline.

18.6 Alert Herald (aherald)

Focus: Monitoring-alert routing. Sits between alert producers (Alertmanager, Datadog, Grafana) and human/on-call channels (PagerDuty, OpsGenie, Slack). Provides smarter routing than Alertmanager's native receivers.

Event subtree: `digital.vasic.herald.alert.*`.

Sources:

- Prometheus Alertmanager webhook.
- Grafana alert webhook.
- Datadog webhook.
- OpenTelemetry-native alert sources.
- Generic CloudEvents alert producers.

Routing features:

- **De-duplication:** same alert fingerprint within window → single notification.
- **Grouping:** related alerts (same service label + severity) collapse into a digest.
- **Inhibition:** parent alert silences child alerts (per Alertmanager's `inhibit_rules`).
- **Escalation:** integrates with Temporal workflow for time-based escalation chains.
- **Quiet hours** override only for `severity=critical` + `category=incidents`.

Subscriber commands:

- Ack:, Silence:, Resolve: (same as sherald).
- Escalate: — bump severity, route to on-call.

18.7 Schedule Herald (scherald)

Focus: Scheduled jobs + recurring notifications (cron-like).

Event subtree: `digital.vasic.herald.schedule.*`.

Sources:

- River queue periodic jobs.

- `scherald serve` runs an internal cron-style scheduler.
- External cron jobs invoking `scherald send`.

Event types:

- `digital.vasic.herald.schedule.job.started` | `succeeded` | `failed` | `skipped`.
- `digital.vasic.herald.schedule.digest.daily` | `weekly` | `monthly`.
- `digital.vasic.herald.schedule.reminder.due` (scheduled human reminders).

Built-in digest builders:

- Daily ops digest (yesterday's alerts + deploys + failed builds).
- Weekly project digest (PRs merged, issues opened/closed, contributors).
- Monthly compliance digest (per `cherald` if installed).

Subscriber commands:

- Snooze: `<reminder_id> for <duration>`.
- Cancel: `<reminder_id>`.
- Remind me: `<prose>` — parse and create a one-shot reminder.

18.8 Incident Herald (`iherald`)

Focus: Incident lifecycle — declare, escalate, resolve, postmortem.

Event subtree: `digital.vasic.herald.incident.*`.

Sources:

- PagerDuty / OpsGenie webhooks (incident open/escalated/resolved).
- Internal `iherald send` from on-call tools.
- Auto-escalation triggered by `aherald` after N min unacknowledged.

Event types:

- `digital.vasic.herald.incident.opened` | `escalated` | `resolved` | `reopened`.
- `digital.vasic.herald.incident.commander.assigned`.
- `digital.vasic.herald.incident.update.posted`.
- `digital.vasic.herald.incident.postmortem.due` | `published`.

Workflow (Temporal):

- On open: page primary on-call → wait 5 min → if no ack, escalate to secondary → wait 10 min → page manager.
- On resolve: schedule postmortem reminder 24h out.
- On postmortem.published: notify stakeholders.

Subscriber commands:

- Page: `<person>` — manual escalation.
- IC: (incident-commander) — assign IC role.
- Update: `<prose>` — post a public update.

- **Resolve:** — close the incident.

18.9 Release Herald (**rherald**)

Focus: Release lifecycle — tags, changelogs, dependency notifications.

Event subtree: `digital.vasic.herald.release.*`.

Sources:

- Git tag webhooks.
- GoReleaser / Release-Please / semantic-release pipeline hooks.
- Renovate / Dependabot PR webhooks (`dependency.update.available`).
- OCI registry push events (`oci.image.pushed`).

Event types:

- `digital.vasic.herald.release.tagged | published`.
- `digital.vasic.herald.release.changelog.generated`.
- `digital.vasic.herald.release.dependency.update.available | major.update.available`.
- `digital.vasic.herald.release.sbom.generated | provenance.signed`.
- `digital.vasic.herald.release.security_advisory.published` (CVE for a project dep).

Composes with **dherald** — release publish event triggers deploy pipeline notification chain.

Subscriber commands:

- **Promote:** `<version>` to `<env>`.
- **Approve:** `<dep_update>` — approve a Renovate PR via authorised channel.

18.10 Compliance Herald (**cherald**)

Focus: Audit + compliance + governance events. For regulated environments (SOC2, HIPAA, PCI-DSS, GDPR).

Event subtree: `digital.vasic.herald.compliance.*`.

Sources:

- Cloud audit logs (AWS CloudTrail, GCP Cloud Audit Logs, Azure Activity Log) via OpenTelemetry / native exporters.
- IAM change webhooks (Okta, Auth0, AzureAD).
- Vault / KMS access logs.
- Database audit log streamers.
- GitGuardian / TruffleHog secret-scan webhooks.

Event types:

- `digital.vasic.herald.compliance.audit.access` (privileged action).

- `digital.vasic.herald.compliance.policy.violation` (e.g. unencrypted bucket, S3 public ACL).
- `digital.vasic.herald.compliance.cert.expiring | expired`.
- `digital.vasic.herald.compliance.license.expiring | non_compliant`.
- `digital.vasic.herald.compliance.access.review.due` (quarterly access reviews).
- `digital.vasic.herald.compliance.secret.exposed`.

Routing:

- All events archived to immutable storage (`docs/herald/audit/` with WORM bit if filesystem supports).
- High-severity events routed to security + legal channels.
- Quarterly summary digest emailed to compliance officer.

Constitution composition: `cherald` events plug into the parent project's Constitution §11.4.10 (credentials never tracked) and §11.4.18 (audit-log discipline).

18.11 Future flavors

Identified candidates (NOT in V2 scope; documented to lock-in event-subtree namespaces):

- **fherald** — Finance Herald (billing events, invoice paid/failed, MRR alerts) — `digital.vasic.herald.finance.*`.
 - **mherald** — Marketing Herald (campaign sent, A/B test winner) — `digital.vasic.herald.marketing.*`.
 - **gherald** — Game/Server Herald (multiplayer match events, anti-cheat reports) — `digital.vasic.herald.game.*`.
 - **xherald** — Experiment Herald (feature-flag experiment results) — `digital.vasic.herald.experiment.*`.
 - **hherald** — Hardware Herald (IoT device telemetry alerts) — `digital.vasic.herald.hardware.*`.
 - **lherald** — Legal Herald (contract renewals, NDA expirations) — `digital.vasic.herald.legal.*`.
 - **vherald** — Vendor Herald (third-party service status, SLA breaches) — `digital.vasic.herald.vendor.*`.
-

§19. Diary

`docs/herald/diary/main.md` (+ `main.pdf` + `main.html`) is the **append-only conversation log** of every inbound and outbound message Herald processes.

19.1 Diary path resolution (per R-20)

Resolved relative to the **operator-specified working directory**:

- Standalone Herald: defaults to Herald's own repository root.
- Herald as submodule: defaults to the parent project's root (discovered via the same parent-walk as `find_constitution.sh`).

- Override: `--diary-root <path> flag`, or `[herald].diary_root` in `config.toml`.

19.2 Schema (Markdown)

2026-05-19T18:30:00Z · pherald · digital.vasic.herald.ci.failed

Field	Value
Tenant	<code>`00000000-...-0001`</code>
Event ID	<code>`01931a7c-...-5678`</code>
Source	<code>`github.com/vasic-digital/Herald/actions/runs/4231`</code>
Channels	tgram (delivered), slack (delivered), email (queued)

****Body:****

```
> Build pao u `main`: 3/5 testova nije prošlo. Pogledaj log:
> https://github.com/vasic-digital/Herald/actions/runs/4231
```

****Subscribers reached:**** alice (tgram + slack), bob (email).

19.3 Sync strategy (per R-03)

- **Pandoc** (Markdown → HTML5) + **WeasyPrint** (HTML → PDF) via Pandoc's `--pdf-engine=weasyprint` flag.
- **Triggers:**
 - Under `<flavor>herald serve: fsnotify-watched debounced builds` (500 ms idle, 2 s ceiling). A single editor save fires 4–12 events; debouncing avoids redundant rebuilds.
 - Under one-shot `<flavor>herald send: post-write hook` in the diary writer.
- **Manifest** at `docs/herald/diary/.sync.json` records `{md_sha256, html_sha256, pdf_sha256, source_md_sha256_at_build, built_at}`.
- **Anti-bluff gate** (CM-DIARY-SYNC per Universal §11.4.61):
 1. Read current `main.md` SHA-256.
 2. Assert it equals `source_md_sha256_at_build` in the manifest.
 3. For HTML: re-run Pandoc to a temp file; byte-equality with on-disk HTML.
 4. For PDF: extract text (`pdftotext main.pdf -`) and compare to deterministic extraction of the HTML body (byte-equal PDF comparison fails due to embedded timestamps + non-deterministic font subsetting).

§20. Extensibility

Per research Topic 6 (operations): two-tier extensibility model.

20.1 Tier A — in-tree adapters (default for V2)

Every channel adapter lives under `commons_messaging/channels/<name>` and implements the `Channel` interface (§11). Out-of-tree authors fork or vendor.

20.2 Tier B — subprocess + manifest (deferred to V2.1)

Spec'd now, implementation deferred. Herald discovers external channel binaries through a manifest file (`heraldchannel.yaml`):

```
name: my-custom-channel
exec: /usr/local/lib/herald/channels/my-channel
supports:
  - text
  - markdown
  - attachments
env:
  required:
    - MY_CHANNEL_TOKEN
protocol_version: 1
```

Communication: stdin/stdout NDJSON with versioned envelope. No in-process linkage — adapters can be written in any language, sandboxed with OS tooling. Mirrors Apprise / Mattermost / git credential-helper.

Explicitly rejected for V2:

- Go plugin stdlib (toolchain-version brittleness, no Windows, breaks `-trimpath`).
 - HashiCorp go-plugin (RPC surface + per-plugin process mgmt — overhead vs. subprocess+JSON).
 - Wazero/WASM (promising; WASI sockets still incomplete; network-bound adapter can't run usefully today). Marked as **V3 roadmap candidate**.
-

§21. Supply chain & release engineering

Per research Topic 5 (operations) + R-18:

21.1 Multi-binary versioning

`go.work` for local dev (git-ignored). Per-flavor `go.mod`. Lockstep release: one git tag → GoReleaser builds all flavors + tags `commons/vX.Y.Z`. Release-Please in manifest mode bumps all modules from Conventional Commits.

21.2 Reproducible builds

Mandatory flags: `CGO_ENABLED=0 GOFLAGS=-trimpath`. Pinned `-ldflags "-buildid= -s -w"`. Record `GOTOOLCHAIN`.

21.3 SBOMs

Every binary + every container image generates **CycloneDX JSON + SPDX JSON** via syft. Attached as release assets and as OCI refererrs on the image.

21.4 Signing

Keyless cosign (Sigstore OIDC via GitHub Actions) for:

- Binaries: `cosign sign-blob`.
- OCI images: `cosign sign`.

21.5 SLSA provenance (target: Build L3)

GitHub actions/attest-build-provenance emits **in-toto** statements signed via Sigstore. Move the build into a **reusable workflow** shared across mirrors to reach **SLSA Build L3**.

21.6 Distribution channels

Channel	Tool	Path
Homebrew tap	GoReleaser	vasic-digital/homebrew-herald
Scoop bucket	GoReleaser	vasic-digital/scoop-herald
.deb / .rpm	nFPM	vasic-digital/herald-packages apt+yum repos
Container images	GoReleaser → Docker	GHCR ghcr.io/vasic-digital/ <flavor>herald
Source release	GitHub Releases	Multi-mirror per §103

Verification UX: `scripts/verify-release.sh` runs `cosign verify + cosign verify-attestation` against the published cert identity (https://github.com/vasic-digital/Herald/.github/workflows/release.yml@refs/tags/*).

§22. Constitution integration

Once Herald is fully implemented and verified, **promote candidate universal rules** into the root Constitution, `AGENTS.md`, and `CLAUDE.md` **via a HelixConstitution PR** (audited per Universal §11.4 + §11.4.17 — universal-vs-project classification), never by editing the parent constitution Submodule from inside Herald (per R-09; see `CONSTITUTION_INHERITANCE.md` §"Promoting Herald rules into the constitution"). Each Flavor presents its Constitution extensions through the same promotion process.

Note: Herald's implementation **MUST BE** in direct connection with the constitution Submodule — discovery via `find_constitution.sh` parent-walk per §103 / §105 of Herald Constitution.

See [../..guides/HERALD_CONSTITUTION.md](#) and [../..guides/CONSTITUTION_INHERITANCE.md](#).

22.1 How constitution rules are extended via constitutable/

A parent project may drop Constitution.md / CLAUDE.md / AGENTS.md (in constitutable/, constitutable/<flavor>/, constitutable/<flavor>/<variant>/, etc.) to layer extensions or overrides on top of the discovered constitution/ Submodule.

Load priority (per constitutable/ mechanism):

constitution Submodule → constitutable/** extensions/overrides for Constitution / CLAUDE.md / AGENTS.md → Project + Submodule definition files.

Each constitutable/<path> MUST contain at least one of: Constitution.md, CLAUDE.md, or AGENTS.md.

Note: Tests in the constitution Submodule MUST be properly extended and updated as constitutable/ content changes.

Note: Herald is **primarily** consumed as a Submodule of another Project; in that case access to the constitution Submodule is through the root of that project (cloned under project_root/constitution or a configured alternative). For **standalone development** of Herald, the constitution is cloned **alongside** Herald (sibling-clone) — current development setup uses this layout. See [../..guides/CONSTITUTION_INHERITANCE.md](#) §"Standalone development" and R-10.

Note: Carefully investigate the codebase of the constitution Submodule before any changes are applied. Comprehensive analysis precedes any extension or promotion.

§23. Specification documents (change rule)

We MUST keep the following rule / mandatory constraint in Constitution.md (parent), AGENTS.md, CLAUDE.md, and HERALD_CONSTITUTION.md §106:

IMPORTANT: Whenever this document (docs/specs/mvp/specification.V2.md) or any file under docs/specs/ (any depth) is modified, **comprehensive planning and implementation of all changes is MANDATORY**. This rule does NOT apply to creating or renaming files; for those, explicitly tell the worker (CLI agent) what to do with the new path. Treat every spec edit as a project-wide ripple, not a doc tweak.

Inheritance-gate invariant **I7a–c** enforces presence of this anchor in CLAUDE.md, AGENTS.md, and HERALD_CONSTITUTION.md.

§24. Documentation

README.md MUST be fully updated with all relevant project details. All user guides and manuals are properly linked from README.md, including (when present):

- docs/flavors/<flavor>/ — per-flavor user manual.
- docs/channels/<channel>/ — per-channel setup guide (credentials, webhooks, signing).
- docs/operations/ — deployment + doctor + backups + upgrades.
- docs/security/ — credential handling, signing keys, secret rotation.
- docs/api/ — machine-readable API specifications (see §24.1).
- docs/migration/ — operator guides for migrating from other notification stacks (Apprise, Gotify, Mattermost-bridge, etc.).

We MUST have **mandatory documentation up to the smallest details**: full user guides, manuals, diagrams, schemes in all major formats (Markdown source + PDF + HTML siblings per §11.4.61 + §11.4.65) and other relevant materials.

24.1 Machine-readable API specifications

Herald ships its own API contracts as machine-readable schemas so client tooling (SDK generators, mock servers, schema-validating proxies, API gateways) doesn't have to reverse-engineer them.

Spec	Location	Format	Generated by
HTTP ingress (/v1/events, /v1/send, /v1/subscribers, /v1/deadletters, /v1/channels)	docs/api/openapi.v1.yaml	OpenAPI 3.1	hand-authored from the canonical Go handler signatures in commons_http/*; CI gate CM-OPENAPI-DRIFT (planned) verifies handler [?] spec parity.
Webhook ingest (signed, per-source)	docs/api/openapi.v1.yaml (sub-tree under /webhooks/)	OpenAPI 3.1	same source as above.
Event taxonomy (digital.vasic.herald.* event types per §4.2)	docs/api/asynccapi.v1.yaml	AsyncAPI 2.6	machine-generated from commons/events.go event-type registry.
Channel adapter Go interface (§11.0)	commons/types.go	Go source	the source IS the spec; pkg.go.dev rendering is the human view.
Database schema	commons_storage/migrations/*.sql	PostgreSQL DDL	the migration files ARE the schema spec; <flavor>herald

Spec	Location	Format	Generated by
			schema dump exports current state.

CLI helpers:

- `<flavor>herald openapi` — print the embedded OpenAPI spec (so operators don't need the repo).
- `<flavor>herald asyncapi` — same for AsyncAPI.
- `<flavor>herald schema dump [--format=sql|markdown]` — emit current DB schema.

Documentation cross-link rule (§11.4.59 + §11.4.61 composed): every change to a `commons_http/` handler **MUST** update `docs/api/openapi.v1.yaml`, **MUST** add an entry to `docs/changelogs/`, **MUST** regenerate `docs/api/openapi.v1.html` (rendered via Redoc or stoplight-elements at build time). Drift is a release blocker.

§25. Testing

Whole project and all derivatives **MUST** follow testing rules from the root Constitution (`Constitution.md`), `CLAUDE.md`, `AGENTS.md` in the constitution Submodule. Specifically:

- **No bluffing** — every PASS carries positive evidence (§11.4).
- **Paired mutation gates** — every new gate has a paired mutation proving it catches regressions (§1.1).
- **Captured evidence** for test logs (§11.4.2).
- **Test pyramid**: unit tests in each module, integration tests against ephemeral Postgres + Redis (testcontainers-go), end-to-end tests with real-channel sandboxes.
- **No fakes beyond unit tests** (Universal §11.4.27) — integration + end-to-end tests use real Postgres, real Redis, real channel-sandbox or test-bot endpoints.

CI runs the inheritance gate (`tests/test_constitution_inheritance.sh`) as a precondition to any other test.

§26. Operations

26.1 Deployment

Reference deployment topologies:

- **Single-host Docker Compose** — Herald flavor + Postgres + Redis on one host. Suitable for small teams.
- **Kubernetes Deployment** — Herald flavor as a Deployment, Postgres + Redis as separate StatefulSets (or external managed). HPA on `messaging.client.operation.duration` p95.

- **Serverless / Lambda** — <flavor>herald send invoked as a Lambda function (one-shot mode); serve mode NOT supported in Lambda due to long-running needs.

26.2 Backups

- Postgres: daily logical backup (pg_dump) + WAL archiving for PITR.
- Redis: AOF rewrite + RDB snapshots.
- Diary: standard git history is the backup (everything is in docs/herald/diary/main.md).

26.3 Upgrades

- Lockstep across flavors via §21.1 release model.
- Database migrations are **forward-compatible** for 2 minor versions (so a rolling restart works during a deploy).
- <flavor>herald migrate is idempotent and safe to run multiple times.

26.4 Operator runbooks

docs/operations/runbooks/ contains one runbook per common operational scenario:

- "Postgres connection storm" → check herald_queue_depth + pg_stat_activity.
- "Telegram bot suspended" → rotate token, update .env, run <flavor>herald doctor.
- "Dead-letter spike" → query dead_letters, identify pattern, replay or purge.
- "DKIM verification failing" → DNS check via <flavor>herald doctor email.

26.5 Operator quickstart (5-minute Docker Compose)

Goal: a working pherald ingesting one webhook and fanning out to Telegram + Email in five minutes on a fresh laptop. This is the canonical "hello world" deployment.

```
# docker-compose.quickstart.yml – referenced from containers/
quickstart/
version: "3.8"
services:
  postgres:
    image: postgres:16-alpine
    container_name: herald-postgres
    environment:
      POSTGRES_USER: herald
      POSTGRES_PASSWORD: ${HERALD_DB_PASSWORD}
      POSTGRES_DB: herald
    ports:
      - "24100:5432"
    volumes:
```

```

    - herald-pg:/var/lib/postgresql/data
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U herald"]
  interval: 5s
redis:
  image: redis:7-alpine
  container_name: herald-redis
  command: ["redis-server", "--requirepass", "${HERALD_REDIS_PASSWORD}"]
  ports:
    - "24200:6379"
  volumes:
    - herald-redis:/data
otel-collector:
  image: otel/opentelemetry-collector-contrib:latest
  container_name: herald-otel
  command: ["--config=/etc/otel-config.yaml"]
  volumes:
    - ./otel-config.yaml:/etc/otel-config.yaml:ro
  ports:
    - "24417:4317"
pherald:
  image: ghcr.io/vasic-digital/pherald:latest
  container_name: herald-pherald
  depends_on:
    postgres: { condition: service_healthy }
    redis: { condition: service_started }
  environment:
    HERALD_POSTGRES_DSN: "postgres://herald:${HERALD_DB_PASSWORD}@postgres:5432/herald?sslmode=disable"
    HERALD_REDIS_ADDR: "redis:6379"
    HERALD_REDIS_USER: "default"
    HERALD_REDIS_PASSWORD: "${HERALD_REDIS_PASSWORD}"
    OTEL_EXPORTER_OTLP_ENDPOINT: "http://otel-collector:4317"
    OTEL_RESOURCE_ATTRIBUTES: "deployment.environment=quickstart"
    TELEGRAM_BOT_TOKEN: "${TELEGRAM_BOT_TOKEN}"
    TELEGRAM_CHAT_ID: "${TELEGRAM_CHAT_ID}"
  ports:
    - "24091:24091" # webhook ingress
    - "24090:24090" # admin (/livez, /readyz, /metrics)
  command: ["serve"]
volumes:
  herald-pg:
  herald-redis:

```

Steps:

1. Clone Herald + containers submodule

```
git clone git@github.com:vasic-digital/Herald.git && cd Herald
git submodule update --init
```

2. Copy & fill the example .env


```

cp .env.example .env
$EDITOR .env # set HERALD_DB_PASSWORD, HERALD_REDIS_PASSWORD,
              TELEGRAM_BOT_TOKEN, TELEGRAM_CHAT_ID

# 3. Boot
docker compose -f containers/quickstart/docker-
compose.quickstart.yml up -d

# 4. Wait for ready
curl --retry 30 --retry-delay 2 --retry-connrefused http://
localhost:24090/readyz

# 5. Send a test event
curl -X POST http://localhost:24091/v1/events \
-H "Content-Type: application/cloudevents+json" \
-d '{
  "specversion": "1.0",
  "id": "01931a7c-3f4e-7000-9abc-def012345678",
  "source": "https://example.com/quickstart",
  "type": "digital.vasic.herald.system.host.cpu_high",
  "subject": "tag:*",
  "data": {"host": "laptop", "cpu_pct": 97}
}'

# 6. Verify
docker logs herald-pherald --tail=20
cat docs/herald/diary/main.md | tail -30

```

A successful run delivers a Telegram message to the configured chat within ~3 seconds, appends a diary entry, and emits OTel traces visible at `http://localhost:24417` (Collector logs).

26.6 Disaster recovery

Recovery objectives (V2 baseline; per-tenant overrides allowed):

Class	RPO target	RTO target	Mechanism
Postgres data	5 min	30 min	WAL archiving + PITR (pgbackrest recommended)
Redis state	5 min	5 min	AOF rewrite + RDB snapshot every 5 min; warm replica optional
Diary	0 (git)	minutes	git push fan-out to 4 mirrors per §103; restore via git clone <any mirror>
Credentials	0 (out-of-band)	depends on operator	Vault / 1Password / OS keychain; Herald NEVER stores plaintext credentials
Configuration	0 (git)	minutes	config.toml is git-committed (no secrets); .env is git-ignored, backed up out-of-band by operator

Class	RPO target	RTO target	Mechanism
Workflow state (Temporal opt-in)	5 min	30 min	Temporal's own backup/restore; out of Herald scope

Cold-start recovery procedure:

1. Restore Postgres from latest WAL position.
2. Restore Redis from latest RDB (transient state is fine to lose; rate-limit buckets repopulate).
3. `git clone Herald + containers submodule` from any mirror.
4. Re-provision `.env` from out-of-band credential store.
5. `docker compose up -d`.
6. `<flavor>herald doctor` — confirm all green.
7. `<flavor>herald deadletter list` — replay any in-flight messages caught at the time of failure.

Tested scenarios (each MUST have an integration test under `tests/dr/`):

- Postgres primary loss → failover to replica.
- Redis loss → cold restart (token buckets reset, no data corruption).
- Single Herald replica crash → other replicas continue (multi-instance deployments).
- Whole-host loss → cold-start from backups.
- All four git mirrors momentarily unreachable → push deferred, queued locally, retried.

§27. Roadmap

27.1 V2 → V3 candidates

Candidate	Notes
WASM channel adapters (wazero)	Once WASI sockets stabilise — Topic 6 (operations)
LLM-driven message rendering	Auto-summarize long events to channel-appropriate brevity
Slack workflow builder steps	Beyond V2 advanced tier
WhatsApp Business custom-template approval workflow	Once Cloud API matures
Mini-Apps inside Telegram	Subscriber-side UI
Adaptive Card Designer integration for Teams	UX for non-developer template authors
First-class Matrix protocol channel	If demand materializes
Mattermost adapter	Open-source alternative to Slack
Rocket.Chat adapter	Self-hosted alternative

27.2 Deferred V1 Recommendations

V1 R-NN items that V2 did not yet implement (with intended landing):

R	Status in V2	Landing
R-01	✓ Applied (24XXX ports in §9.4)	—
R-02	✓ Applied (single binary, two modes §3.1)	—
R-03	✓ Specified (§19.3)	First implementation cycle
R-04	✓ Applied (§3.3)	—
R-05	✓ Specified (delivery evidence enum §11.13)	First adapter cycle
R-06	✓ Specified (§11.2 Max behind build tag)	When sanctions advisory drafted
R-07	✓ Applied (§7.1)	—
R-08	✓ Applied (§12 ConversationRef)	—
R-09 / R-10 / R-14 / R-15 / R-19 / R-20 / R-21 / R-22	✓ Applied	—
R-11 / R-12	✓ Specified (§9.5)	When first SDK / containers lands
R-13	Deferred — constitutable/ loader + I8/ I9 gates	Next constitution PR
R-16	✓ Specified (§15)	First implementation cycle
R-17	✓ Specified (§8.2 algorithm)	First commons_prefix commit
R-18	✓ Specified (§21.1)	First release pipeline

27.3 Cost considerations

Order-of-magnitude indicative pricing as of 2026-Q2 (operators MUST re-verify at deployment time; tiers change frequently). Herald itself is open-source / no licence fee — these costs are external dependencies an operator pays for in production.

Infrastructure (per single-region deployment):

Component	Self-host minimum	Managed (illustrative)
Postgres 16 (2 vCPU / 4 GiB / 20 GiB)	~\$15/mo (Hetzner CX21, DO Basic)	RDS db.t4g.small ~\$35/mo
Redis 7 (1 GiB)	~\$10/mo (same node as Postgres for small tenants)	ElastiCache cache.t4g.micro ~\$15/mo

Component	Self-host minimum	Managed (illustrative)
Herald binary host (1 vCPU / 1 GiB)	~\$5/mo	ECS Fargate 0.25 vCPU ~\$10/mo
OTel Collector	sidecar / shared	Grafana Cloud free tier or self-host
Single-tenant baseline	~\$30/mo	~\$60–80/mo

Transactional email provider (ESP) — pick one based on volume + deliverability needs:

Provider	Free tier	Paid tier entry	Bounce/complaint webhook	Notes
<u>Resend</u>	3 000 emails/mo, 100/day	\$20/mo for 50k	✓ (Event Webhook)	Best DX of the three; Idempotency-Key header support.
<u>Postmark</u>	none — paid only	\$15/mo for 10k	✓ (Bounce / Spam / Open / Click streams)	Best deliverability reputation for transactional traffic; HTTP 406 for blocked recipients.
<u>SendGrid</u>	100 emails/day free forever	\$19.95/mo for 50k	✓ (Event Webhook)	Widest ecosystem; complex pricing tiers.
Self-host SMTP	\$0	infra + IP-reputation labour	DSN parsing only	Only if operator has DNS / reverse-DNS / warm IPs already; otherwise deliverability tanks.

Messaging platform fees:

Channel	Cost model
Telegram Bot API	Free, no fee per message.
Slack	Free for public/private channel webhooks; paid plans only required for full Slack workspace, not for the integration.
Discord	Free webhooks; free bot.
Microsoft Teams	Free incoming webhooks; Bot Framework requires Azure Bot Service ~\$0.50/1 k messages.
Lark / Feishu	Free for chatbots within an org.
WhatsApp Business Cloud API (Meta direct)	Per-conversation pricing, ~0.005–0.15 depending on country + category.
WhatsApp Business via Twilio	Twilio markup on top of Meta price.
Viber	

Channel	Cost model
	Free for service messages within 30 days of user interaction; promotional pricing varies.
Max	Free Bot API at the time of writing (verify at deploy).
ntfy / Gotify	Self-hosted = \$0; ntfy.sh free public instance.

Supply-chain tooling:

Tool	Cost
GitHub Actions (public repo)	Free unlimited minutes.
GitHub Actions (private repo)	2 000 free minutes/mo on Linux; \$0.008/min after.
Sigstore / cosign	Free (keyless via OIDC).
syft (SBOM)	Free / OSS.
GoReleaser OSS	Free.
GoReleaser Pro	\$19/mo if Herald needs the per-subproject <code>tag_prefix</code> feature (R-18 alternative — V2 default does not require Pro).

OpenTelemetry backend:

Backend	Free tier
Grafana Cloud	10 k series metrics, 50 GiB logs, 50 GB traces /mo
Honeycomb	20 M events/mo
Datadog	5 hosts free for 14 days
Self-host Prometheus + Tempo + Loki	\$0 + operator time

Total order-of-magnitude TCO for a small team running pherald + sherald self-hosted on Hetzner with Resend free tier: **~\$30–60/mo** of recurring infrastructure. Operator labour for upgrades, monitoring, and incident response is the larger real cost — design choices in §17 (observability) and §26 (operations) exist specifically to minimise that labour.

§28. Notes & open questions

- The list of integrated messengers in §11 is the V2 commitment; additional channels may be added in later iterations.
- Whether to ship a **first-party web UI** for subscriber preference management is an open question — current direction is "no UI in V2; expose REST API only; let third parties build UIs".
- Whether aherald should incorporate **machine-learning-based alert grouping** (similar to BigPanda, Moogsoft) is deferred to V3.

- Whether to support **end-to-end encryption** of messages (Signal Protocol via olm/megolm) is deferred — only Matrix bridges currently demand this.
 - The **rate-limit cap** for AI-CLI-agent subscribers needs operator-level tuning per deployment to avoid runaway invocation; default suggested 60 messages/minute per agent token.
-

§29. Changelog

29.1 V1 → V2 changes (2026-05-19)

V1 → V2 changes (2026-05-19):

- **Renamed** `specification.md` → `specification.V1.md` (preserved as historical record).
- **Created** `specification.V2.md` (this document).
- **New sections** (no V1 counterpart): §2.2 What Herald is NOT, §2.3 Architecture diagram, §4 Event model & wire format (CloudEvents), §5 Architecture overview, §6 Channel addressing & routing (Apprise-style URLs + tags), §7 Subscriber model (identity, preferences, quiet hours, locale), §13 Templating, §14 Localization, §15 Security model (three-layer), §16 Multi-tenancy & isolation (Postgres RLS + Redis ACL), §17 Observability & SLOs (OpenTelemetry), §20 Extensibility (Tier A + Tier B), §21 Supply chain & release engineering (SLSA L3), §26 Operations, §27 Roadmap.
- **Populated V1 TBDs:**
 - System Herald (was TBD) → fully specified §18.3.
 - Project Herald's Constitution rules → folded into §18.2 subscriber-command contract.
 - Input commands → expanded per-flavor in §18.
 - Input attachments → §15.3 content-level scanning + size limits.
 - Others and misc → §18.4–18.11 (7 new flavors + future-flavor namespace reservations).
- **New flavors** beyond V1's pherald + sherald: bherald (Build), dherald (Deploy), aherald (Alert), scherald (Schedule), iherald (Incident), rherald (Release), cherald (Compliance) — plus reserved namespaces for fherald / mherald / gherald / xherald / hherald / lherald / vherald.
- **Game-changing architecture adoptions** (from research):
 - **CloudEvents v1.0** as canonical wire format.
 - **Watermill** as routing core.
 - **River + Postgres** as default queue (transactional enqueue); NATS JetStream as opt-in.
 - **Apprise URL+tag** channel model.
 - **Knock-style preferences** matrix.
 - **OpenTelemetry** end-to-end + Prometheus semantic conventions for messaging.
 - **PostgreSQL RLS + Redis ACL** for multi-tenancy.
 - **SLSA L3** supply chain (cosign + syft SBOM + in-toto provenance).
 - **MJML** for email templates, compiled at author time.
 - **nicksnyder/go-i18n** with CLDR plurals.
- **All V1 text-level Recommendations** (R-01 / R-04 / R-09 / R-10 / R-15 / R-19 / R-20 / R-21 / R-22) carried forward and re-expressed in V2 context.

- **Per-channel feature matrix** (§11.13) — every channel now has documented V1-minimum, V2-advanced, out-of-scope tiers + delivery-evidence ceiling.
- **Email deep dive** (§11.9) — DKIM signing (RFC 6376) + RFC 8058 one-click unsubscribe + DSN bounce parsing + suppression list + doctor email DNS verification, addressing Gmail/Yahoo 2024 sender requirements.
- **Removed** the V1 Review section — its findings are folded into the V2 body or marked applied/deferred in §27.2. V1's full Review remains in `specification.V1.md` for traceability.

V2 r2 → r3 (within V2 lifecycle, all detailed in §30): added §11.0 Go type contract for the Channel interface, defined `webhook_sources / channel_addresses / thread_refs / quarantined_messages` schemas, pinned Go ≥ 1.22 + license, added §16.1 Data retention & GDPR, §17.1 OpenTelemetry env-var table, §26.5 Operator quickstart, §26.6 Disaster recovery, §27.3 Cost considerations. R3 added the remaining undefined types (Subscriber, CloudEventEnvelope, TraceContext, Branding, ChannelID, PreferenceSet), §9.6 Database migration tooling, §3.4 Worker pools + SIGHUP hot-reload, §5.7 Ingress API URLs, §7.5 AI-agent subscribers, §8.3 Workable-item lifecycle, §11.14 `null:// sandbox channel`, §17.4.1 Per-channel SLO budgets, §24.1 Machine-readable API specs, §5.4.1 Outbound idempotency, §3.5 Time/clock abstraction.

29.2 V2 → V3 changes (2026-05-20)

- **Created** `specification.V3.md` (this document). `specification.V2.md` marked Status=superseded and kept as historical record alongside `specification.V1.md`. V1 + V2 + V3 stack constitutes the spec evolution chain.
- **NEW top-level sections** (no V2 counterpart):
 - **§31 Project integration contract** — explicit contract for what a consuming project provides + what Herald provides in return; composition with eight named Universal Constitution mandates (§11.4.12, .15, .16, .21, .32, .44, .55, .59, .60, .61, .65); project-side `config.toml` template; mandatory `test_project_integration.sh` audit gate.
 - **§32 Inbound processing pipeline** — full lifecycle from `channel.Subscribe` to diary append. 30 s polling cadence mandate (per-channel mechanic table). FIFO ordering per (tenant, channel_address) via Postgres advisory locks. Seven Worker stages (validation → safety → anti-spam → classification → dispatch → materialization → reply). `inbound_messages` schema. Anti-spam at four layers (per-sender rate, burst detection, reputation, channel-level frequency).
 - **§33 LLM/agent dispatch** — Claude Code as first integration. `resolve_session(project_name)` algorithm anchored at `<consuming-project>/herald/claude-code/sessions/<project>.session`. Structured `<<<HERALD-DISPATCH-v1>>>` request envelope. JSON `<<<HERALD-REPLY>>>` response schema (outcome enum, summary, details, affected_paths, reproduction_steps, estimated_effort, workable_item_proposed, follow_up_questions). Pluggable Dispatcher interface — V3 r1 ships only `claude-code`; spec hooks for OpenCode/Aider/Gemini/Cursor/Managed Agents.
 - **§34 Reply protocol** — three mandatory replies per inbound message: (A) queued ack with quoted original ≤ 5 s, (B) processing-started ≤ 5 s of dispatch (edit-in-place where channel supports), (C) final result within criticality SLA. Failure replies carry precise reason + diagnostics. Per-

channel quote/edit matrix for all 13 channels. Operator-tunable verbosity (minimal | normal | verbose).

- **§35 Versioned reports + Git linkage** — report-aware event types (.report.created/.updated/.archived) with update_key for correlation. Per-channel update mechanic (edit-in-place vs re-post). Git commit SHA + URL + diff URL embedded in fan-out. report_publications schema with (content_sha256, git_commit_sha) dedup. Uncommitted-changes warning.
- **§36 Outbound multi-format attachments** — every Herald-produced attachment ships in four formats (.md + .html + .pdf + .docx) so subscribers pick their preferred. Pandoc generation pipeline with --reproducible flag. Cache at <diary_root>/.herald/format-cache/<sha256>/. Per-channel delivery rules + per-subscriber PreferenceSet.attachment_formats overrides. Bundle size cap (default 50 MiB).
- **Expanded §18.2 Project Herald** with the full **Investigation-before-Fixing flow** (§18.2.1): every bug/issue/implementation request first creates a HRD-INV-NNN investigation item; LLM (§33) analyzes reproducibility + affected paths + effort; only validated investigations promote to final workable items. Adds §18.2.2 criticality determination table (with explicit SLA windows per level), §18.2.3 Universal §11.4.16 type-classification mapping, §18.2.4 attachment storage at issues/users/attachments/<WORKABLE_ITEM_ID>/ with attachments_index.md source-of-truth file, §18.2.5 Claude Code project-session integration (anchor file at .herald/claude-code/sessions/<project>.session).
- **§29 retitled** "Changelog" (was "Changelog (V1 → V2)") so V1→V2 / V2→V3 / future changelogs accumulate under one logical section.

V2 sections that V3 does NOT change (still authoritative — V3 just extends): §1–§28 architectural baseline, §11.0 channel contract types, all flavor sections §18.3–§18.11 (refinement scheduled for V3 r2 per the metadata Continuation row).

§31. Project integration contract

First-version focus. §31 makes Herald a load-bearing piece of any consuming project that already follows the Helix Universal Constitution. The contract is symmetric: the consuming project gets a fully-instrumented event-to-channel pipeline; Herald gets a discoverable place to live and the constitution's rules to operate under.

31.1 Where Herald lives in a consuming project

Herald is consumed as a Git submodule. Conventional path:

```
<consuming-project>/
├─ constitution/                    # Helix Universal
Constitution (also a submodule)
├─ herald/                          # this repo, as submodule
│   └─ pherald/cmd/pherald/main.go # built per §21
│       └─ sherald/cmd/sherald/main.go
│           └─ ...
```



```

└─ containers/                                # docker/podman compose
(also a submodule)
└─ docs/
    └─ Issues.md
    └─ Issues_Summary.md
    └─ Fixed.md
    └─ Fixed_Summary.md
    └─ CONTINUATION.md
    └─ Status.md
    └─ herald/
        └─ diary/main.{md,html,pdf,docx}

```

The consuming project's CI invokes `<flavor>herald send` from build steps and runs `<flavor>herald serve` as a daemon (Docker Compose / Kubernetes). Subscribers DM the configured bots; Herald processes those inbound messages, opens workable items, and replies in-thread.

31.2 Mandatory integration points

A consuming project that adopts Herald **MUST**:

1. **Add Herald as a Git submodule** at `<consuming-project>/herald/`. Run `install_upstreams.sh` per Universal §11.4.36.
2. **Register Herald in the Owned-submodule set** of the project's Constitution.md extension (per Universal §4 / §11.4.28).
3. **Pin the Herald version** via submodule SHA. Bumps follow Universal §11.4.32 (post-pull validation).
4. **Provide credentials** via `.env` + shell-exported variables (per §3.3). `.env` is git-ignored; `.env.example` is committed and templates every variable Herald needs.
5. **Wire the channel addresses** by populating `channel_addresses` (per §6) with the project's bot tokens / webhook URLs. Tags follow project conventions — common defaults: `prod`, `staging`, `oncall`, `audit`, `compliance`.
6. **Mount the diary path** so `<consuming-project>/docs/herald/diary/main.{md,html,pdf,docx}` is writable by Herald (per §19).
7. **Honor the spec-change rule** — any change to `<consuming-project>/herald/docs/specs/` triggers comprehensive planning per Universal §11.4 + Herald Constitution §106 (read order documented in CLAUDE.md).
8. **Add Herald CI hooks** — at minimum, post-commit hook calling `<flavor>herald send --type digital.vasic.herald.ci.commit.pushed ...` and CI-build webhook configured per §5.5.

31.3 Event-type registration

The consuming project **MAY** introduce its own event subtree under `digital.vasic.herald.<project_subtree>.*` and register the schema in the AsyncAPI spec (§24.1). The schema **MUST** be committed in `<consuming-project>/herald/docs/api/asynapi.<project>.yaml`. Herald validates inbound events against the schema and rejects malformed payloads with HTTP 422 + a per-channel error reply.

31.4 Composition with existing constitution mandates

Universal mandate	Herald V3 contribution
§11.4.12 Auto-generated docs sync	Herald emits events on every Issues / Issues_Summary / Fixed / Fixed_Summary / Status / Status_Summary change so subscribers get real-time visibility.
§11.4.15 Item-status tracking	Status transitions trigger <code>digital.vasic.herald.project.task.*</code> events.
§11.4.16 Item-type tracking	Type classifier (§32.4) maps inbound Bug:/Issue:/Task:/Implementation: commands to the correct item type.
§11.4.21 Operator-blocked status	When a workable item enters operator-blocked, Herald fan-outs to the configured <code>oncall</code> tag with the unblock-question payload.
§11.4.32 Post-Constitution-Pull validation	Herald's submodule bumps are validated against the constitution; Herald's gate (<code>tests/test_constitution_inheritance.sh</code>) is part of that validation.
§11.4.44 Document Revision Header	Herald respects §11.4.61 superseding format (table) when reading project docs.
§11.4.55 Reopens-history	Reopen flow (§8.3) writes to <code>docs/Reopens/<HRD-NNN>.md</code> .
§11.4.59 README always-sync	Herald reports README out-of-sync as an event (<code>digital.vasic.herald.compliance.doc.stale</code>).
§11.4.60 Documentation composite covenant	Herald's diary + multi-format exports per §36 satisfy the composite gate.
§11.4.61 Markdown metadata + ToC	Herald's diary follows the canonical metadata table; every diary entry carries Revision + Last modified per §19.
§11.4.65 Universal Markdown export	§36 multi-format pipeline is the implementation of this mandate at attachment-time.

31.5 Project-side configuration template

```
# <consuming-project>/herald/config.toml – committed (no secrets)
[herald]
flavor      = "pherald"
project_name = "ATMOSphere"                # used for Claude
          Code session resolution (§33.2)
tenant_id   = "000000000-0000-0000-0000-000000000001"
diary_root  = ""                          # blank → parent-
          walk discovery (§19.1)
admin_port  = 24090
```

```

http_port      = 24091

[ingest]
poll_interval_seconds = 30
    # §32.2 – mandatory 30s cadence
fifo_strict      = true
    # §32.3 strict FIFO

[security]
spam_max_per_minute_per_sender = 10
    # §32.5 anti-spam
    thresholds
spam_max_per_hour_per_sender    = 100
quarantine_unknown_sender       = true

[llm]
default_agent      = "claude-code"
    # §33.1 default
    dispatch target
claude_code_binary = "claude"
    # path / name on
    PATH
session_strategy   = "project-cwd"
    # §33.2 strategy
    enum

[attachments]
out_format_set      = ["md", "html", "pdf", "docx"] # §36 multi-
    format default
in_storage_root     = "issues/users/attachments"   # §18.2
    inbound attachment path
in_max_mib          = 25
in_total_max_mib    = 100

```

31.6 Project audit gate

<consuming-project>/herald/tests/test_project_integration.sh
(planned) asserts:

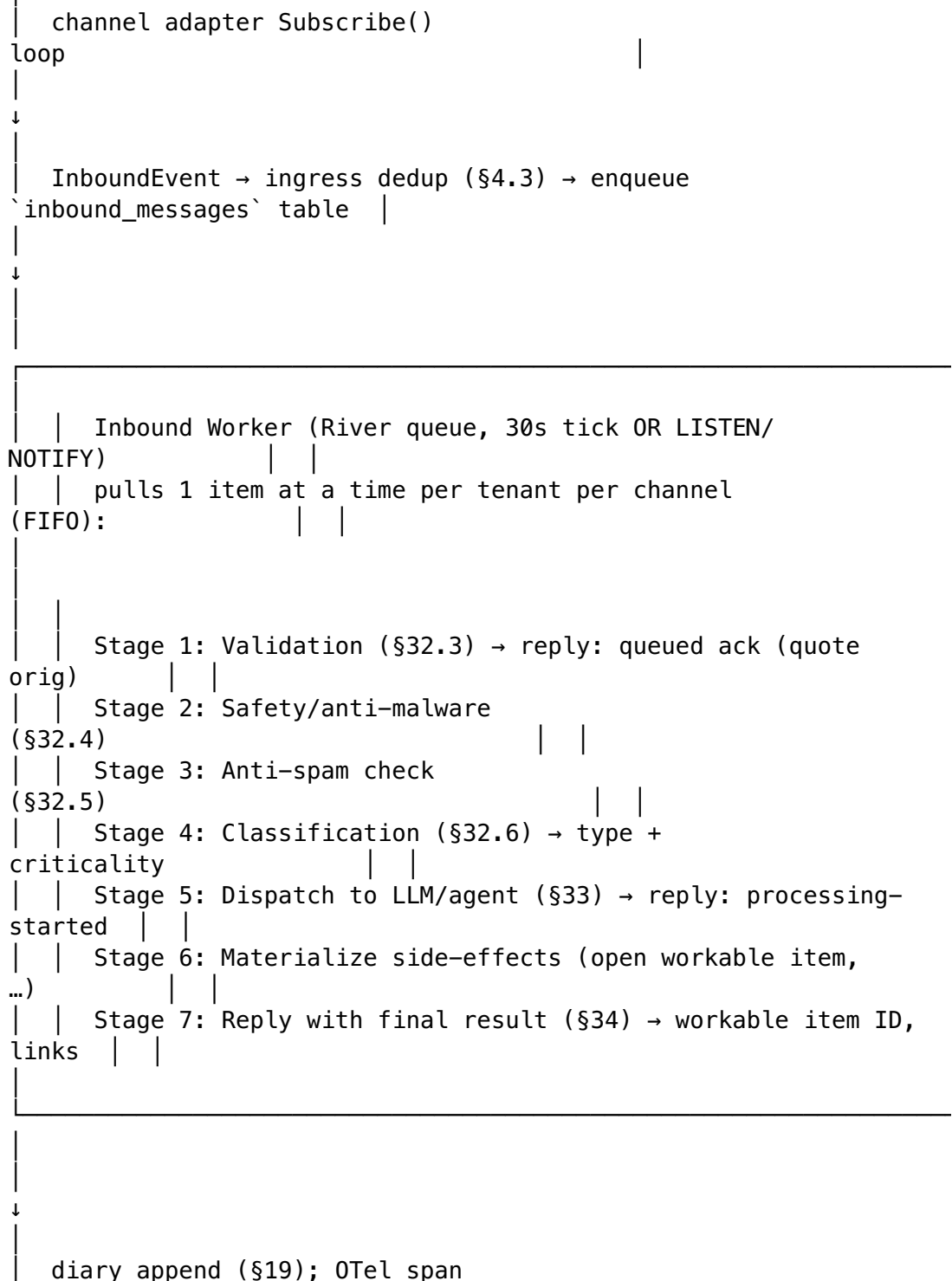
- constitution/ submodule discoverable via parent-walk.
- containers/ submodule present and version-pinned.
- .env.example committed; .env git-ignored.
- channel_addresses table non-empty for the project's default tenant.
- Project's Owned-submodule set lists herald/ and containers/.
- Diary path exists and is writable.
- pherald doctor exits zero.

Paired §1.1 mutation: rename .env to track-ed — gate FAILs.

§32. Inbound processing pipeline

32.1 Pipeline overview

Every message a subscriber sends back into Herald (DM reply, channel mention, email reply with Herald's Message-ID in In-Reply-To, ntfy poll) enters the **inbound queue** for sequential processing by Workers. The pipeline is FIFO per tenant per channel-address, with explicit anti-spam, multi-stage validation, and three user-visible reply checkpoints (§34).



32.2 Polling cadence (30 s mandate)

Every flavor's Subscribe loop **MUST** check upstream for new messages **at least every 30 seconds**. Upstream-specific implementation:

Channel	Mechanism	Effective check rate
Telegram	Bot API getUpdates long-poll (timeout 25 s)	continuous, but a 30 s safety-net timer also fires getUpdates if the long-poll thread stalled
Slack	Socket Mode (WebSocket) OR Events API webhook	continuous on Socket Mode; 30 s timer as keepalive ping
Discord	Gateway WebSocket	continuous; 30 s timer asserts heartbeat OK
MS Teams	Bot Framework activity webhook	webhook-driven; 30 s timer asserts subscription validity
Lark	Event subscription webhook	same
WhatsApp Cloud	Webhook	same
Viber	Webhook	same
Email	IMAP IDLE preferred; fall back to 30 s IMAP poll	IDLE = continuous; non-IDLE = 30 s
ntfy	WebSocket/SSE	continuous
Gotify	WebSocket	continuous
Max	Bot API poll	30 s

Configurable via `[ingest].poll_interval_seconds` (default 30); operators **MAY** tighten to 10 s for high-volume tenants but **MUST NOT** loosen beyond 60 s without operator-explicit override (`--allow-slow-poll`).

32.3 FIFO order + Worker selection

Inbound items are queued in `inbound_messages` with `enqueued_at` timestamp + UUIDv7 id. The River queue is partitioned by `(tenant_id, channel_address_id)` so:

- Multiple senders on the same channel are processed in strict arrival order.
- Different channels for the same tenant run in parallel.
- Different tenants run in parallel.

Schema:

```
CREATE TABLE inbound_messages (
  id                                UUID PRIMARY KEY DEFAULT uuidv7(),
```

```

tenant_id          UUID NOT NULL,
channel            TEXT NOT NULL,
channel_address_id UUID NOT NULL REFERENCES
    channel_addresses(id),
sender_channel_user_id TEXT NOT NULL,
received_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
cloudevent_jsonb   JSONB NOT NULL,
attachments_jsonb  JSONB NOT NULL DEFAULT '[]'::jsonb,
stage              TEXT NOT NULL DEFAULT 'queued',      --
    queued|validating|safety|anti_spam|classifying|dispatched|
    completed|failed
stage_started_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
classification     JSONB,                                --
    type+criticality+confidence
workable_item_id   TEXT,                                  --
    after_successful_materialization
last_reply_at      TIMESTAMPTZ,
failure_reason     TEXT,
failure_details    JSONB
);
CREATE INDEX inbound_fifo_idx ON inbound_messages (tenant_id,
    channel_address_id, received_at)
    WHERE stage NOT IN ('completed', 'failed');
ALTER TABLE inbound_messages ENABLE ROW LEVEL SECURITY;
ALTER TABLE inbound_messages FORCE ROW LEVEL SECURITY;
CREATE POLICY inb_isolation ON inbound_messages
    USING (tenant_id = current_setting('app.tenant_id')::uuid)
    WITH CHECK (tenant_id =
        current_setting('app.tenant_id')::uuid);

```

The Worker holds a per-(tenant_id, channel_address_id) advisory lock for the duration of one item's pipeline so strict FIFO is preserved across the seven stages.

32.4 Stages 1–2: validation + safety

Stage 1: Validation. Asserts:

- CloudEvents id is unique within idempotency window (§4.3 hit/miss).
- sender_channel_user_id matches an entry in subscriber_aliases OR is allowlisted as a verified webhook source.
- Message body is well-formed UTF-8 ≤ 8 KiB per command + ≤ 256 KiB total.
- Attachments declared in attachments_jsonb MUST be retrievable from the source channel and MUST be $\leq$ [attachments].in_max_mib each (default 25 MiB) and $\leq$ [attachments].in_total_max_mib total (default 100 MiB).

Failure → reply with precise reason (§34.4), stage = failed, NO further processing.

Stage 2: Safety / anti-malware. Each attachment is scanned:

- ClamAV (when commons_security.clamav.enabled=true and the daemon is reachable) — definitive engine.
- Magic-byte type check vs declared MIME (rejects MIME spoofing).

- Filename allowlist by extension (default:
.txt .md .json .yaml .yml .csv .pdf .docx .html .png .jpg .jpeg
.webp .gif .log .zip .tar.gz).
- Optional: per-tenant additional scanner (configurable hook command receives the file path, returns 0 = clean, non-zero = quarantine).

Inbound text body is scanned for:

- URL allowlist / blocklist per tenant.
- Prompt-injection markers (heuristics — short list of known patterns: Ignore previous instructions, BEGIN SYSTEM, <|im_start|>, etc.). Hit → quarantine, NOT auto-reject (operator review).

Failure → reply with reason, stage = failed, attachment quarantined to `quarantined_messages.attachments_jsonb`.

32.5 Stage 3: anti-spam

Spam prevention is layered:

1. **Per-sender rate limits** (configurable per tenant via `[security].spam_max_*`):
 - `spam_max_per_minute_per_sender` (default 10)
 - `spam_max_per_hour_per_sender` (default 100)
 - `spam_max_per_day_per_sender` (default 500) Exceeded → message accepted, but reply downgraded to 429 Too Many Requests with the cooldown window.
2. **Burst detection** — a sender that posts > N similar messages (Levenshtein distance < 5%) within window T is throttled; if pattern continues, sender enters quarantine.
3. **Sender reputation** — per-(tenant, sender_channel_user_id) score in Redis (`t:<id>:rep:<sender>` integer), incremented on successful interaction, decremented on validation/safety failures. Negative score → mandatory operator review before processing.
4. **Channel-level frequency** — if `channel_addresses[address_id]` receives > `rate_floor` messages/minute from distinct senders, channel-wide soft rate-limit kicks in.
5. **Known-bot allowlist** — agent tokens (§7.5) bypass per-minute limits but obey per-hour caps.

All spam decisions logged to `spam_audit` (Redis stream, 7-day retention) for tuning.

32.6 Stage 4: classification

The classifier (running as a Watermill handler in `commons_messaging`) determines:

Item type (per Universal §11.4.16 + Project Herald subscriber-command vocabulary):

Inbound trigger	Type	Maps to
Bug: / Issue: / Issue: <prose>	bug	<code>docs/Issues.md</code> row + <code>digital.vasic.herald.project.task.opened</code>

Inbound trigger	Type	Maps to
Task:	task	same docs flow, different type column
Implementation: / Implement: / Feature:	implementation	same
Query: / Question: / Q: / ?	query	LLM-only path (no workable item — research/info request)
Request: / Req:	request	LLM-routed; outcome may produce a workable item if approved
Investigation: / Investigate:	investigation	intermediate state (§32.7)
Status:	status_request	reply with current state from docs/Status.md
Continue:	continuation_request	reply with docs/CONTINUATION.md pointer
Done: / Resolve: (operator role only)	closure	close workable item, migrate Issues→Fixed
Reopen: (operator role only)	reopen	per Universal §11.4.55 reopen flow
Ack: / Silence: / Snooze:	ops_command	per-flavor ops handlers
Spec:	spec_change_request	gated by §23 spec-change rule

Criticality (independent of type; required for bug, issue, task, implementation, investigation):

Level	Trigger	SLA
critical	explicit critical: prefix in subject line, OR LLM classifier confidence ≥ 0.85 on production-outage keywords	acknowledge within 5 min; first investigation within 1 h
high	explicit high: OR LLM 0.7–0.85	within 30 min; investigation within 4 h
middle	default	within 2 h; investigation within 24 h
low	explicit low: or nice-to-have:	within 24 h; investigation within 1 week

The classifier itself is **deterministic-first** (keyword/regex), falling back to LLM (§33) only when the deterministic step returns ambiguous. Operators MAY override classifications via `Override: HRD-NNN type=task criticality=high`.

32.7 Stage 5: Investigation-before-Fixing intermediate state

When a bug / issue / implementation arrives, Herald first creates an **Investigation workable item** (type investigation, status investigating), NOT the final bug/task row. The investigation:

1. Stores in `workable_items` table with type investigation and `parent_request` referencing the inbound message id.

2. Dispatches to the LLM (§33) for **reproducibility analysis** — can the LLM identify the affected code path? Reproduce locally? Estimate effort?
3. Stores LLM findings as a comment in docs/Investigations/<HRD-NNN>.md.

Once investigation concludes:

- **Validated** → Herald creates the *final* bug / task / implementation workable item, atomically migrates the investigation row to "investigation completed" status, links the new item via `parent_investigation`.
- **Cannot reproduce / not actionable** → investigation row stays; replies with reason + asks for more info; subscriber's follow-up reply re-runs the investigation.
- **Rejected (out of scope / duplicate / known)** → investigation closed with that reason; no final item created.

The investigation gate is a strong guard against runaway LLM-generated workable items.

32.8 Spam audit + observability

Per §17.2 the inbound pipeline emits:

- `herald_inbound_received_total{tenant,channel,outcome}` — outcome $\in$ `enqueued | rejected_validation | quarantined_safety | rate_limited | classified | dispatched | completed | failed`.
 - `herald_inbound_stage_duration_seconds{stage}` histogram.
 - `herald_inbound_classification{type,criticality,confidence_bucket}` counter.
 - `herald_spam_block_total{tenant,reason}` counter.
 - Span name `herald.inbound.process` with child spans per stage.
-

§33. LLM / agent dispatch

33.1 First target: Claude Code

V3's first LLM dispatch target is **Claude Code** (the Anthropic CLI). The dispatcher lives in `commons_messaging/dispatch/claude_code/`.

Design constraints:

- Claude Code sessions are scoped to a **working directory**, not a freestanding "name". V3 introduces a **project-named session resolution algorithm** (§33.2) that maps the consuming project's name to a stable working-directory anchor.
- The CLI is invoked in **non-interactive batch mode** for each request: `claude --resume <session> --print "<prompt>"`.
- Background invocation: Herald spawns the CLI in a separate process group with timeout (default 5 min); stdout is collected; non-zero exit becomes a `failed` stage with reason recorded.
- Concurrency: per-project session is single-writer (one in-flight Claude Code invocation at a time). Multiple inbound requests for the same project queue behind each other (River job priority preserves arrival order).

33.2 Session resolution algorithm

For project named ATMOSphere:

```
function resolve_session(project_name) -> session_id:
  1. compute working_dir = config[herald.session_workdir]
    (default: <consuming-project-root> discovered via parent-
walk)
  2. compute session_anchor_path = working_dir/.herald/claude-
code/sessions/<project_name>.session
  3. if session_anchor_path exists AND contains a session UUID
    AND `claude --resume <uuid> --print "ping"` returns 0:
    → return that UUID
  4. else:
    spawn `claude --print "Initializing Herald session for
project: <project_name>"`
    in working_dir
    capture the new session UUID emitted by Claude Code
stdout
    write UUID to session_anchor_path
    → return new UUID
```

The `.herald/claude-code/sessions/` directory is `.gitignore'd`. Session anchors persist across Herald restarts. Operators can manually reset by deleting the anchor file (next dispatch will create a fresh session).

33.3 Request envelope

Every dispatch sends a structured prompt:

```
<<<HERALD-DISPATCH-v1>>>
Project:      <project_name>
Inbound ID:   <UUIDv7>
Sender:       <channel>:<subscriber.handle> (verified: yes|no,
roles: [operator|reader|...])
Channel:      <ChannelID>
Received at:  <RFC 3339>
Classification: type=<type> criticality=<level>
confidence=<0.0-1.0>
Conversation: <thread-of-quoted-replies, full chain bottom-to-
top>
Attachments: [<name>:<mime>:<size_bytes>, ...]
```

User message:

<verbatim user text>

HERALD TASK (run in background along with mainstream work):

```
<task verb derived from classification>:
- For `bug`/`issue`/`investigation`: reproduce + identify affected
code paths +
```

- classify root-cause area + propose validation steps.
- For `query`/`question`: research + answer; cite project docs if relevant;
 - short answers preferred (subscribers see this directly).
- For `request`/`implementation`: scope effort + propose approach + flag prerequisites
 - + estimate workable-item dependencies.
- For `spec\_change\_request`: invoke Herald Constitution §106 spec-change rule.

Reply with a JSON object on a single line, prefixed with
 `<<<HERALD-REPLY>>>`:

```
{
  "outcome": "validated"|"rejected"|"needs_more_info"|"answered",
  "summary": "<short summary for the subscriber>",
  "details": "<longer markdown body for the diary>",
  "affected_paths": [<file>:<line>", ...],           // optional
  "reproduction_steps": ["...", ...],                 // for bug/
investigation
  "estimated_effort": "S|M|L|XL",                     // for
request/implementation
  "workable_item_proposed": {                         // optional
    "type": "bug|task|implementation|investigation",
    "criticality": "critical|high|middle|low",
    "title": "...",
    "labels": ["..."]
  },
  "follow_up_questions": ["..."]                     // when
outcome=needs_more_info
}
```

DO NOT modify project files unless the subscriber explicitly asked you to.
 DO NOT commit. DO NOT push.
 <<<END-HERALD-DISPATCH>>>

Herald parses the <<<HERALD-REPLY>>> JSON line out of Claude Code's stdout, validates against the response schema, and proceeds to Stage 6 materialization.

33.4 Materialization (Stage 6)

Based on the outcome and workable_item_proposed:

- validated + workable_item_proposed → allocate next HRD-NNN, write to docs/Issues.md + emit digital.vasic.herald.project.task.opened (per §8.3 lifecycle) + link investigation parent (§32.7).
- answered (query/question) → no workable item; the reply IS the answer.
- needs_more_info → no workable item; reply asks subscriber for clarifications listed in follow_up_questions.
- rejected → no workable item; reply states rejection reason.

33.5 Other LLM/agent targets (spec hooks, V3+ implementation)

The dispatcher interface is provider-agnostic:

```
type Dispatcher interface {
    Name() string
    Capabilities() DispatcherCapabilities
    Dispatch(ctx context.Context, req DispatchRequest)
        (DispatchResponse, error)
}
```

V3 ships only `claude-code` dispatcher. Spec hooks reserved for:

- `opencode` — same CLI pattern.
- `aider` — file-modifying dispatcher (extra safety gates required; not enabled by default).
- `gemini-cli` — when GA.
- `cursor-cli` — pending product availability.
- `agentic-claude-api` — direct Anthropic API (Managed Agents) for hosted deployments where a CLI isn't desirable.

Operators select per-tenant via `[llm].default_agent`; per-classification-type override via `[llm.routing]` table.

§34. Reply protocol (queued → processing → result)

Every inbound message MUST receive **three replies** unless the inbound itself was rejected at Stage 1 (in which case one reply explaining the rejection).

34.1 Reply A — Queued ack

Within ≤ 5 s of receipt:

- Quote the original message (per-channel thread mechanic per §12 `ConversationRef`).
- Body: 📬 Received. Queued as `#INB-<short-id>`. I'll process this momentarily.
- Carries `Herald-Reply-Stage`: queued extension header for adapters that support custom headers.

34.2 Reply B — Processing started

Within ≤ 5 s of Stage 5 dispatch:

- Quote the original (same thread).
- Body: ⌚ Processing as `<classification.type>` (priority: `<criticality>`). Investigating...
- Edit-in-place where the channel supports it (Telegram `editMessageText`, Slack `chat.update`, Discord webhook edit); otherwise post a new message in the thread.

34.3 Reply C — Final result

Within the SLA window for the classified criticality (§32.6):

- Quote the original (same thread).
- Body varies by outcome:
 - **Validated + workable item created:**
 Created HRD-042 (bug, criticality=high). Investigation summary attached. Link: <Issues.md row anchor>. Repo: <git URL>
With multi-format attachment bundle per §36 containing the investigation summary.
 - **Answered (query):**
 <answer>
With cited documents attached as multi-format if relevant.
 - **Needs more info:**
 Need more detail: 1) <question> 2) <question>
 - **Rejected:**
 <reason>. <suggested next step>.

34.4 Failure replies

If validation, safety, anti-spam, or classification fails:

- Quote the original message.
- Body:  <stage>: <precise reason>. Details: <link-to-diary-entry-with-stack-trace>.
- Body MUST contain the exact rejection cause (not "internal error" — anti-bluff per Universal §11.4).
- Stage = failed in inbound_messages; original receipt is preserved.

Examples:

-  validation: attachment 'malware.exe' exceeds size limit (30 MiB > 25 MiB max). Details: ...
-  safety: attachment 'invoice.pdf' triggered ClamAV signature 'EICAR-Test-File'. Quarantined. Details: ...
-  anti-spam: rate limit (12 msg/min) exceeded. Cooldown until <RFC 3339 UTC>. Details: ...
-  classification: ambiguous type. Suggested prefixes: 'Bug:', 'Query:', 'Request:'. Details: ...

34.5 Per-channel quoting/edit-in-place matrix

Channel	Quote original	Edit-in-place (B → C)	Notes
Telegram	reply_to_message_id	editMessageText ✓	Reply B edited into Reply C if same message id
Slack	thread_ts (parent ts)	chat.update ✓	edits the queued ack

Channel	Quote original	Edit-in-place (B → C)	Notes
Discord	?thread_id= + webhook?wait=true	webhook edit ✓	requires bot token for non- webhook edit
MS Teams	message reference	Adaptive Card refresh × (per-message replacement only)	post new message in conversation
Lark	reply API	message edit ✓	
WhatsApp Cloud	context.message_id	not supported	post new message
Viber	tracking_data thread	not supported	post new message
Email	In-Reply-To + References headers	not supported	post new message in thread
ntfy	not natively threaded	not supported	new message with X-Tag: same-as- original
Diary	parent-id anchor	append + cross-link	

When edit-in-place is unavailable, Herald posts a NEW message and Reply B+C are distinct.

34.6 Configurable verbosity

Operators MAY tune reply verbosity per tenant via `[replies].verbosity = minimal | normal | verbose`:

- `minimal` — only Reply A (queued ack) + Reply C (final result); no Reply B.
- `normal` (default) — A + B + C as described.
- `verbose` — A + B + C + per-stage progress (Stage 3 done, Stage 4 done, ...).

§35. Reports + state-tracking documents (versioned fan-out with Git linkage)

Some events carry **reports that change over time** — security-scan summaries, build-status digests, weekly project digests, compliance reports. When a report's source document changes, Herald MUST re-publish to subscribers with the updated file attached + a link to the Git commit that introduced the change.

35.1 Report-aware event types

CloudEvents types under `digital.vasic.herald.report.*` are special:

- `.report.updated` — content changed since last publish.

- `.report.created` — first publish.
- `.report.archived` — no longer being maintained.

Each carries an `update_key` extension attribute (stable identifier for the report) so Herald can correlate updates with previous deliveries.

35.2 Update mechanic per channel

When a `.report.updated` event arrives:

Channel	Mechanism
Telegram	Send new message in the channel with the updated file attached + caption <code>Updated: <git short-sha> – <commit message subject></code> . If channel supports <code>editMessageMedia</code> for original messages, edit in place; otherwise re-post.
Slack	Edit original message via <code>chat.update</code> (using <code>channel_msg_id</code> from <code>report_publications</code> table); attach new file via <code>files.upload</code> and reference.
Discord	Edit webhook message; attach updated file.
Teams	Post new Adaptive Card with <code>previousVersionRef</code> ; old card is left in place (no edit-in-place reliable).
Email	New message with <code>In-Reply-To: <previous-Message-ID></code> + References chain; full updated attachment bundle.
Diary	Append new entry referencing previous; the report file at <code>docs/herald/reports/<update_key>/main.md</code> (and siblings) is the source of truth (overwritten on each update).

35.3 Git linkage

If the report's source `.md` is committed in the consuming project's repo, Herald includes:

- `git_commit_sha` — short SHA from `git log -1 --pretty=format:%h <file>` at publish time.
- `git_commit_url` — derived from `git remote get-url origin` + the commit SHA, formatted for the host (GitHub blob URL, GitLab blob URL, GitFlic/GitVerse equivalents).
- `git_diff_url` — URL to the diff against the previous publish's commit SHA.

If the source `.md` is NOT committed yet (working-tree only), Herald posts with a warning: ⚠ This report has uncommitted changes – link will become live once the change is committed.

35.4 Persistence

```
CREATE TABLE report_publications (
  id                UUID PRIMARY KEY DEFAULT uuidv7(),
  tenant_id         UUID NOT NULL,
  update_key        TEXT NOT NULL,
  -- stable identifier for the report
```

```

channel          TEXT NOT NULL,
channel_address_id  UUID NOT NULL REFERENCES
    channel_addresses(id),
channel_msg_id    TEXT,                                -- so we
    can edit-in-place on next update
content_sha256    BYTEA NOT NULL,
    -- of the source .md
git_commit_sha    TEXT,                                --
    nullable for uncommitted
published_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
UNIQUE (tenant_id, update_key, channel, channel_address_id)
);
ALTER TABLE report_publications ENABLE ROW LEVEL SECURITY;
ALTER TABLE report_publications FORCE ROW LEVEL SECURITY;
CREATE POLICY rp_isolation ON report_publications
    USING (tenant_id = current_setting('app.tenant_id')::uuid)
    WITH CHECK (tenant_id =
        current_setting('app.tenant_id')::uuid);

```

The (content_sha256, git_commit_sha) pair is the dedup key: if a re-publish event arrives but the source .md is byte-identical AND the commit SHA is the same, Herald skips the publish (logs digital.vasic.herald.report.noop).

§36. Outbound multi-format attachments (.md + .html + .pdf + .docx)

Every report, technical-documentation page, or research material that Herald sends as a subscriber-visible attachment **MUST** be attached in **four formats** so subscribers can pick the one they prefer for their use case.

36.1 Format set

Default [attachments].out_format_set = ["md", "html", "pdf", "docx"]. Per-channel overrides allowed via channel-address query param format_set=md,pdf (e.g. for bandwidth-constrained ntfy push).

36.2 Generation pipeline

For each source .md file Herald wants to attach:

```

source.md    (canonical, source of truth)
  ↓
pandoc -f gfm -t html5 -s -o source.html
  ↓
pandoc -f gfm --pdf-engine=weasyprint -o source.pdf
  ↓
pandoc -f gfm -o source.docx

```

docx generation requires Pandoc (already a dependency for HTML/PDF per §11.4.65); no extra runtime needed. WeasyPrint stays the PDF engine.

36.3 Pre-generation caching

Generation is deterministic — given the same source `.md`, the outputs are byte-stable (modulo PDF embedded timestamps, mitigated via Pandoc `--reproducible` flag in V3 r1).

Cache at `<diary_root>/herald/format-cache/<sha256-of-md>/
{source.md, source.html, source.pdf, source.docx, manifest.json}`.
Cache hit avoids regeneration.

36.4 Per-channel delivery rules

Channel	Delivery
Telegram	Four <code>sendDocument</code> calls in sequence (Telegram's bot API has no native "file group"); the order is <code>.md</code> , <code>.html</code> , <code>.pdf</code> , <code>.docx</code> so the most-readable preview surfaces first.
Slack	Single <code>files.upload_v2</code> with all four files referenced in one message; preview thumbnail = PDF.
Discord	Multiple file attachments on one webhook message (up to 10 per message; we use 4); embed shows PDF preview.
MS Teams	Adaptive Card with four <code>Action.OpenUrl</code> links to per-format URLs in a Herald-hosted blob (channel doesn't support multi-file attachment natively).
Lark	Multi-file upload in one message.
WhatsApp Cloud	One message per file (WA Business doesn't support multi-attachment messages).
Viber	Carousel of links.
Email	One MIME multipart message with four <code>application/...</code> parts, each <code>Content-Disposition: attachment</code> and <code>filename="report.<ext>"</code> .
ntfy	Posts the <code>.md</code> inline; the other three via X-Attach URLs to Herald-hosted blobs.
Gotify	Same as ntfy.
Webhook (outbound)	CloudEvents payload with four <code>data_base64</code> entries OR four URLs depending on <code>attachment_mode</code> query param.
Diary	All four written to <code>docs/herald/diary/attachments/<message_id>/</code> and referenced from the diary entry.

36.5 Format-set override per subscriber

Subscribers MAY express format preferences via `PreferenceSet.metadata.attachment_formats` (per §7.2):

```
{  
  "attachment_formats": ["md", "pdf"]    // only these two; skip  
html + docx  
}
```

If a subscriber's preference excludes a format that another subscriber on the same channel-address wants, Herald sends the union (or the channel's max bundle if multi-attachment is supported). On channels that allow only one attachment per message (WhatsApp, Viber), Herald sends per-subscriber DM with that subscriber's preferred format.

36.6 Total size limits

Combined size of the four formats per attachment-bundle MUST stay under `[attachments].out_bundle_max_mib` (default 50 MiB). If a single source `.md` produces > 50 MiB of total output (rare — typically only image-heavy reports), Herald posts a single message with a link to a Herald-hosted blob carrying the full bundle.

§30. V2 self-review log

Added 2026-05-19 by a focused self-review pass on V2 r1. Each finding is tagged V2-R-NN to distinguish from V1's R-NN series. **All findings in this section have been applied in V2 r2** unless explicitly marked Deferred.

30.1 Gaps closed (applied this revision)

- **V2-R-01. Missing Go value types referenced by the Channel interface.** §11 previously declared `Channel` with `Capabilities`, `OutboundMessage`, `Receipt`, and `InboundHandler` as forward references without definitions. Applied: new §11.0 ("Channel adapter contract") defines all referenced types — `Capabilities`, `DeliveryEvidence` (enum), `OutboundMessage`, `Body`, `Attachment`, `Recipient`, `Action / ActionType`, `Priority`, `Receipt`, `InboundHandler`, `InboundEvent`. These live in `commons/types.go`; adapters are forbidden from inventing equivalents.
- **V2-R-02. Missing webhook_sources table schema.** §5.5 referenced the table but never defined it. Applied: full schema (`signature_kind`, `signature_header`, `secret_encrypted`, `secret_rotated_at`, `ip_allowlist`, `replay_window_s`) with RLS policy and `<flavor>herald webhook rotate` CLI.
- **V2-R-03. Missing channel_addresses table schema.** §6 referenced the table but never defined it. Applied: full schema with `address_url` (env-interpolated at load time so secrets stay out of the row), `tags` GIN index, `priority_floor`, `enabled` + health-check columns.
- **V2-R-04. Missing thread_refs table schema.** §12 prose-only definition. Applied: full schema with composite PK (`tenant_id`, `logical_thread_id`, `channel`), `last_activity_at` for time-window GC.
- **V2-R-05. Missing quarantined_messages table schema.** §15.2 referenced but never defined. Applied: full schema with `triage_status` enum, partial index on pending rows, 30-day auto-expiry policy, CLI verbs.
- **V2-R-06. subscribers.handle had no uniqueness constraint.** Original schema allowed duplicate handles within a tenant — operationally surprising. Applied: `UNIQUE (tenant_id, handle)`. Also added roles `TEXT[]` and metadata `JSONB` columns for the operator-mapped privilege model and extensible per-subscriber data, plus an explicit composite index on `tenant_id`.

- **V2-R-07. Go toolchain version not pinned.** §9.1 said "written in Go" without naming a minimum. Without slog and the OTel slog bridge, observability won't compile. Applied: **Go ≥ 1.22** mandatory; toolchain directive across all go.mod files; bump path documented.
- **V2-R-08. License not named.** Multiple references to "the LICENSE file" without spelling out the terms. Applied: §9.1 now points to the parent project's LICENSE and requires top-level license comments in every go.mod; license compliance check is a cheraId responsibility.
- **V2-R-09. SIGTERM / graceful-shutdown semantics undefined.** Both modes (send, serve) lacked a documented exit contract. Applied: §3.1 now specifies trap-SIGTERM, stop-ingress, drain with HERALD_SHUTDOWN_GRACE (default 30 s), flush OTel exporter, close Postgres + Redis, exit 0/1 distinction. Second SIGTERM forces exit.
- **V2-R-10. OpenTelemetry env vars not enumerated.** §17.1 said "OTLP/gRPC to a Collector" without naming the standard SDK env vars. Operators couldn't deploy without reading OTel spec. Applied: full table of OTEL_* env vars Herald honours, including resource attributes Herald sets by default and the messaging semantic-convention version pin (v1.30+).
- **V2-R-11. Data retention + privacy was unspecified.** No section on how long Herald holds onto subscriber data, dead letters, quarantined messages, or diary entries; no GDPR right-to-erasure / right-to-portability flow. Applied: new **§16.1** with per-class retention defaults (table-driven), <flavor>herald subscriber forget and subscriber export flows, diary-redaction opt-out, data-sovereignty note.
- **V2-R-12. Operator quickstart missing.** New operators couldn't get from clone to first delivery without reading the whole spec. Applied: **§26.5** with a complete Docker Compose YAML, 6-step bring-up procedure, and a verifiable end-to-end test (curl webhook → Telegram delivery + diary append + OTel trace).
- **V2-R-13. Disaster recovery posture unspecified.** Backups were mentioned but RPO/RTO targets, cold-start procedure, and tested scenarios were absent. Applied: **§26.6** with per-class RPO/RTO table, cold-start procedure, and named integration-test scenarios that MUST live under tests/dr/.
- **V2-R-14. Cost considerations missing.** Operators had no order-of-magnitude TCO guidance to choose between self-hosted SMTP vs Resend/Postmark, between Grafana Cloud vs self-host Prometheus, etc. Applied: **§27.3** with infrastructure / ESP / messaging-platform / supply-chain-tooling / observability-backend pricing tables (with verify-at-deploy caveat).

30.2 Deferred (out of scope for this self-review)

These findings were noted but require their own focused pass:

- **V2-R-15. ASCII architecture diagram alignment.** Current diagram in §2.3 has minor box-drawing misalignment at the └ corners under proportional-font renderers. Deferred — fixing requires Mermaid or PlantUML adoption + a renderer pipeline; not blocker for V2.
- **V2-R-16. Heading-depth normalisation.** Some §18.X flavor subsections go 4 levels deep (heading → field → subfield → list). Considered but rejected — depth tracks meaningful structure; flattening loses navigation cues.
- **V2-R-17. "V1 minimum" vs "V1 (MVP)" wording inconsistency.** Cosmetic. Deferred to a docs-only polish PR.

30.3 Method

The pass was a single-author read-through immediately following V2 r1 authoring, focused on internal consistency (prose [?] formal definitions), operational completeness (can someone deploy this?), and compliance gaps (GDPR, license, version pinning). Findings rated High when a downstream reader would file a bug, Medium when a future implementer would have to invent a missing detail, Low for stylistic. Only High + Medium findings were applied.

30.4 Audit trail (r2)

- Pre-review commit: V2 r1 at 96b7cc6.
- r2 commit: 9648545 (one logical commit covering V2-R-01..V2-R-14).
- Inheritance gate before and after: 12 PASS / 0 FAIL. Meta-test: ✓.
- All four Herald mirrors targeted on push.

30.5 V3 review log (r3, this revision)

A second self-review pass following r2. Targeted a *different cut* than r1→r2: where r1→r2 closed missing tables and added operational content, r2→r3 closes the second-layer Go types (referenced by §11.0 but not defined) and adds the implementer-grade operational detail that turns "we have a spec" into "we have a buildable spec".

30.5.1 Gaps closed (applied this revision)

- **V3-R-01. Remaining undefined Go types in §11.0.** Subscriber, CloudEventEnvelope, TraceContext, Branding (as Go struct, cross-ref §6.3), ChannelID (typed string constants), SubscriberAlias, PreferenceSet/CategoryPref/WorkflowPref/QuietHours (Go-decoded forms of the JSON in §7.2) — all referenced by OutboundMessage / InboundEvent but never defined. **Applied:** all defined in §11.0 with package + file home pins (commons/types.go, commons/preferences.go, etc.). Closes the "Channel interface compiles in isolation" gap.
- **V3-R-02. Database migration tooling unspecified.** §26.3 mentioned <flavor>herald migrate but never named the tool, file layout, or numbering scheme. **Applied:** new §9.6 picks golang-migrate/migrate (embedded), specifies the commons_storage/migrations/000001_..._up.sql layout, the runtime contract (migrate up/down/status/force/validate), the role split (herald_migrator BYPASSRLS vs herald_app), and the forward-compatibility rule (two minor versions).
- **V3-R-03. Worker-pool sizing absent.** No guidance on how many concurrent workers Herald runs. **Applied:** new §3.4 specifies three independently-sized pools (HTTP ingress, router/dispatch, River channel-delivery) with defaults keyed off NumCPU, env-var overrides, and sizing guidance by deployment tier (small / multi-tenant / high-burst).
- **V3-R-04. SIGHUP hot-reload semantics missing.** <flavor>herald serve long-running but no way to refresh config without restart. **Applied:** new §3.4 specifies SIGHUP trap, the safe-to-change vs requires-restart partition (channels/router-workers/log-level/allowlists vs ports/DSN/RLS-policies), the diff-and-validate cycle, and the digital.vasic.herald.system.config.reloaded audit event.

- **V3-R-05. Ingress HTTP URL paths never enumerated.** Operators couldn't write reverse-proxy / API-gateway rules without reading the source. **Applied:** new §5.7 tabulates every /v1/* and /webhooks/* and /livez/readyz/startupz/metrics/admin/* endpoint with method + auth mode, plus the rate-limit defaults and the /v1/ versioning policy.
- **V3-R-06. Workable-item lifecycle missing.** §8 defined the prefix algorithm but never explained how an HRD-NNN actually moves from Bug: command → Issues.md row → Fixed.md resolution. **Applied:** new §8.3 with full lifecycle ASCII flow, the workable_items schema (lightweight pointer table — canonical record stays in Markdown per Universal §6 human-edit-ability), per-tenant advisory-lock sequence allocation, and the reopen flow that composes with Universal §11.4.55 Reopens.md history.
- **V3-R-07. null:// sandbox channel undocumented.** No first-class way to test routing without hitting real channels. **Applied:** new §11.14 specifies the URL grammar (null://?fail_rate=...&latency_ms=...&ceiling=...), the in-memory ring buffer, the /admin/null/* inspector API, the per-environment gate (CM-NUL-CHANNEL-DISABLED-IN-PROD), and a recommended test-fixtures pattern for every commons_messaging/channels/<name>/ package.
- **V3-R-08. Time / clock abstraction missing.** time.Now() called directly anywhere makes tests of quiet hours, batching, backoff, TTLs unreliable. **Applied:** new §3.5 with the commons/clock interface (RealClock + FakeClock), the Default package-global variable swap pattern, and the golangci-lint rule herald-no-direct-time-now that fails compilation if anyone calls time.Now() outside commons/clock.
- **V3-R-09. Machine-readable API specs not committed.** Spec referenced <flavor>herald openapi without describing where the specs live. **Applied:** new §24.1 specifies docs/api/openapi.v1.yaml (OpenAPI 3.1, hand-authored from Go handler signatures, with CM-OPENAPI-DRIFT parity gate) and docs/api/asynapi.v1.yaml (AsyncAPI 2.6, machine-generated from commons/events.go); also lists the CLI helpers (openapi, asynapi, schema dump).
- **V3-R-10. Outbound idempotency unspecified.** §4.3 covered ingress idempotency but not the case where the channel send itself succeeds-but-loses-response and gets retried. **Applied:** new §5.4.1 with the (tenant_id, event_id, channel, channel_address_id) outbound key, the outbound_dedup table (24h TTL — narrower than ingress 7d), and the rule that adapters supporting upstream idempotency (Slack/Stripe-style/Email Message-ID) MUST forward the key while those that don't (raw SMTP/Telegram/Discord) MUST consult outbound_dedup in the same River-job transaction.
- **V3-R-11. Per-channel SLO budgets missing.** §17.4 aggregate SLOs hid one bad channel behind good channels. **Applied:** new §17.4.1 publishes per-channel success and p95-latency targets for each of the 12 channels with a multi-window / multi-burn-rate alert rule (Google SRE Workbook pattern: 2h/14× AND 6h/6× = page; either alone = warn).
- **V3-R-12. AI-agent subscribers conflated with humans.** Spec assumes "subscriber" without distinguishing the runaway-loop risk of agents. **Applied:** new §7.5 introduces a subscribers.kind enum (human/agent/service), the agent_tokens table (argon2id-hashed bearer with per-token rate limit + expiry + revocation), the default throttles (60 req/min/token, 3-strike auto-cool-down at 30min), the audit-span attribute (herald.subscriber.kind="agent" + herald.agent.token_id), and the rule that quiet-hours are ignored for agents by default.

30.5.2 Deferred (out of scope for r3)

- **V3-R-13. Migration-from-existing-systems operator guide.** Mentioned briefly in §24 doc layout but no actual content for migrating from Apprise/Gotify/Mattermost-bridge/etc. Deferred — needs its own dedicated docs/migration/ content cycle.
- **V3-R-14. Multi-region / global deployments.** What if one tenant spans regions? Out-of-scope; treated as schema-per-tenant + region-pinned operator concern; will resurface when first cross-region customer appears.

30.5.3 Method

Same as r2: single-author read-through immediately following r2 commit, focused on three categories — (a) prose²formal-definition gaps in the type contract layer that §11.0 introduced, (b) operational completeness for an implementer who hasn't read the constitution, (c) correctness boundaries (outbound dedup composition, agent throttling) the architecture document needs to nail down before code lands. Findings rated as before; only High + Medium applied this round.

30.5.4 Audit trail (r3)

- Pre-review commit: V2 r2 at 9648545.
- r3 commit: covered V3-R-01..V3-R-12 in one logical commit on top of r2.
- Inheritance gate before and after: 12 PASS / 0 FAIL. Meta-test: ✓.
- All four Herald mirrors targeted on push.

Statistical context: r1→r2 added ~33 KB to the Markdown source closing 14 findings. r2→r3 closed 12 findings — the curve is flattening, which is the expected pattern (the easier high-leverage gaps go first). A fourth pass would likely yield only stylistic findings; the next high-value pass should come *after* first-implementation cycle surfaces real-world spec gaps that desk-review alone can't find.

30.6 V3 r1 review log (this revision)

This is **not** a desk-review of V2 — it is a *requirements-driven* expansion. V2 was architecturally complete; V3 r1 adds the operator-product story the user described in their V3 brief (project integration, inbound pipeline, LLM dispatch, reply protocol, versioned reports, multi-format attachments) and folds in the **explicit project-side requirements** the user introduced during V3 authoring: 30 s poll cadence, FIFO inbound queue, anti-spam, Investigation-before-Fixing, Claude Code project-named session, issues/users/attachments/WORKABLE_ITEM_ID/, multi-format outbound attachments (.md + .html + .pdf + .docx).

30.6.1 Findings applied (V3-R1-NN)

- **V3-R1-01. Project integration contract was implicit.** Spec assumed a consuming project existed; never specified the contract. Applied: §31 with 6-point integration checklist, configuration template, and test_project_integration.sh audit gate composing with 11 Universal Constitution mandates.
- **V3-R1-02. Inbound pipeline had no defined cadence.** §3 mentioned "daemon mode listens" but no polling rule. Applied: §32.2 mandates ≤ 30 s upstream

check rate; per-channel table maps to underlying mechanism (long-poll vs Socket Mode vs Gateway vs webhook vs IMAP IDLE).

- **V3-R1-03. No FIFO guarantee for inbound processing.** V2 mentioned River queue but no per-tenant per-channel ordering rule. Applied: §32.3 specifies advisory-lock-per-(tenant_id, channel_address_id) to guarantee strict arrival order.
- **V3-R1-04. Anti-spam unspecified.** Spec referenced "security validation" but no anti-spam layer. Applied: §32.5 with four-layer anti-spam (per-sender rate / burst detection / reputation / channel frequency); spam_audit Redis stream for tuning.
- **V3-R1-05. Investigation-before-Fixing pattern missing.** Spec went straight from "Bug:" command to workable item. Applied: §18.2.1 mandates intermediate investigation item (HRD-INV-NNN); LLM analyzes reproducibility before final item is allocated; rejects duplicates/out-of-scope/non-reproducible.
- **V3-R1-06. LLM/agent dispatch had no concrete first integration.** V2 left LLM dispatch as future work. Applied: §33 wires Claude Code as the V3 r1 default: resolve_session(project_name) algorithm, <<<HERALD-DISPATCH-v1>>> envelope, JSON reply schema, pluggable Dispatcher interface.
- **V3-R1-07. Claude Code session was assumed to support --session-name.** It doesn't. Applied: §33.2 documents the gap and specifies the resolution algorithm using the project-name as an anchor-file key, with --resume <UUID> as the actual CLI invocation.
- **V3-R1-08. Reply protocol was single-shot.** V2 said "Herald replies" without staging. Applied: §34 specifies three replies (queued / processing / result) with per-channel edit-in-place mechanics, criticality-driven SLAs, and verbose error reporting that complies with Universal §11.4 anti-bluff.
- **V3-R1-09. Reports vs one-off messages were conflated.** V2 had no separate semantic for documents that change over time. Applied: §35 introduces report-aware event types, report_publications dedup table, Git commit SHA + URL + diff URL embedding, edit-in-place where supported.
- **V3-R1-10. Outbound attachments were single-format.** V2 sent the rendered file in whatever format the channel preferred. Applied: §36 mandates four-format bundle (.md + .html + .pdf + .docx) so subscribers choose. Pandoc generation pipeline (already a dependency) + cache + per-channel delivery rules.
- **V3-R1-11. Attachment storage path for user uploads was implicit.** V2 said "validate and store" without naming the path. Applied: §18.2.4 mandates issues/users/attachments/<WORKABLE_ITEM_ID>/ with attachments_index.md as source-of-truth + 8-step validation pipeline (download → ClamAV → MIME → extension → size-per-file → size-total → move-on-pass → quarantine-on-fail).
- **V3-R1-12. Criticality classification had no SLA mapping.** V2 had a 4-level scale but no concrete SLA per level. Applied: §18.2.2 table maps each criticality to specific Reply A/B/C SLA windows + investigation deadline.

30.6.2 Deferred to V3 r2 (the user's follow-up scope)

- **V3-R1-13. Other flavors (§18.3–§18.10) not yet refined for richer channel interaction.** sherald/bherald/dherald/aherald/scherald/iherald/rherald/cherald still carry V2 subscriber-command vocabulary. V3 r2 will add: interactive buttons (Slack Block Kit action buttons / Telegram inline-keyboard callbacks / Discord components / Teams Adaptive Card actions), reaction-based ack/silence, slash-command discovery via /help, per-flavor command palettes.

- **V3-R1-14. Full re-export of V1+V2+V3 + push EVERYTHING.** Deferred to V3 r3 as the final polish/release pass.

30.6.3 Audit trail (r3 → V3 r1)

- Pre-V3 commit: V2 r3 at f4ebba1.
- V3 r1 commit: covers V3-R1-01..V3-R1-12 in one logical commit on top of V2 r3.
- Inheritance gate before and after: 12 PASS / 0 FAIL. Meta-test: ✓.
- All four Herald mirrors targeted on push.
- V2 metadata bumped to Status=superseded; V2 re-exported to keep §11.4.65 invariant green.

Statistical context: V1 was 594 lines; V2 r1 was 1745 lines (+1151); V2 r3 was ~3000 lines (+1255); V3 r1 adds another ~1100 lines. The spec has grown by ~5× in two days of iteration — about half of that is the V3 first-version requirements (§31–§36 + §18.2 expansion), the rest was architectural maturity in V2's three revisions.